Universidade Federal do Piauí
Centro de Educação Aberta e a Distância

REQUISITO DE SOFTWARE

Pedro de Alcântara dos Santos Neto



Ministério da Educação - MEC
Universidade Aberta do Brasil - UAB
Universidade Federal do Piauí - UFPI
Centro de Educação Aberta a Distância - CEAD

Requisito de Software

Pedro de Alcântara dos Santos Neto

UFPI
2013

PRESIDENTE DA REPÚBLICA	*Dilma Vana Rousseff Linhares*
MINISTRO DA EDUCAÇÃO	*José Henrique Paim*
GOVERNADOR DO ESTADO	*Wilson Nunes Martins*
REITOR DA UNIVERSAIDADE FEDERAL DO PIAUÍ	*Luiz de Sousa Santos Júnior*
PRESIDENTE DA CAPES	*Jorge Almeida Guimarães*
COORDENADOR GERAL DA UNIVERSIDADE ABERTA DO BRASIL	*João Carlos Teatini de S. Clímaco*
DIRETOR DO CENTRO DE EDUCAÇÃO ABERTA E A DISTÂNCIA DA UFPI	*Gildásio Guedes Fernandes*

COORDENADORES DE CURSOS

ADMINISTRAÇÃO	*Francisco Pereira da Silva Filho*
CIÊNCIAS BIOLÓGICAS	*Maria da Conceição Prado de Oliveira*
FILOSOFIA	*Zoraida Maria Lopes Feitosa*
FÍSICA	*Miguel Arcanjo Costa*
MATEMÁTICA	*João Benício de Melo Neto*
PEDAGOGIA	*Vera Lúcia Costa Oliveira*
QUÍMICA	*Davi da Silva*
SISTEMAS DE INFORMAÇÃO	*Arlino Henrique Magalhães de Araújo*

EQUIPE DE DESENVOLVIMENTO

TÉCNICO EM ASSUNTOS EDUCACIONAIS	*Ubirajara Santana Assunção*
EDIÇÃO	*Roberto Denes Quaresma Rêgo*
PROJETO GRÁFICO	*Samuel Falcão Silva*
DIAGRAMAÇÃO	*José Luís Silva*
REVISÃO	*Beatriz Gama Rodrigues*
REVISÃO GRÁFICA	*Carmem Lúcia Portela Santos*

CONSELHO EDITORIAL DA EDUFPI

Prof. Dr. Ricardo Alaggio Ribeiro (Presidente)
Des. Tomaz Gomes Campelo
Prof. Dr. José Renato de Araújo Sousa
Profª. Drª. Teresinha de Jesus Mesquita Queiroz
Profª. Francisca Maria Soares Mendes
Profª. Iracildes Maria de Moura Fé Lima
Prof. Dr. João Renór Ferreira de Carvalho

S237 Neto, Pedro de Alcântara dos Santos
Sistemas de Informações / Pedro de Alcântara dos Santos Neto - Teresina: EDUFPI/UAPI, 2011

160 p.

1- Requisito de Software

C.D.D. - 005.3

© 2013. Universidade Federal do Piauí - UFPI. Todos os direitos reservados.A responsabilidade pelo conteúdo e imagens desta obra é do autor. O conteúdo desta obra foi licenciado temporária e gratuitamente para utilização no âmbito do Sistema Universidade Aberta do Brasil, através da UFPI. O leitor se compromete a utilizar o conteúdo desta obra para aprendizado pessoal, sendo que a reprodução e distribuição ficarão limitadas ao âmbito interno dos cursos. A citação desta obra em trabalhos acadêmicos e/ou profissionais poderá ser feita com indicação da fonte. A cópia deste obra sem autorização expressa ou com intuito de lucro constitui crime contra a propriedade intelectual, com sansões previstas no Código Penal.

Apresentação

Um Sistema de Informação (SI) ou qualquer outro software tem seu início associado à ideia de desenvolvê-lo. A partir desse momento, é necessário iniciar uma série de reuniões para se aprofundar o conhecimento nas regras que envolvem a operação do sistema.

O conjunto de atividades que auxilia a obtenção de informações a respeito de como o sistema deveria ser criado, juntamente com uma representação desses conceitos em modelos que sejam utilizados para o desenvolvimento faz parte das atividades de Requisitos e Análise.

Nesta apostila apresentamos um detalhamento dessas atividades, guiadas a partir de vários exemplos práticos ilustrando sua execução.

Na Unidade I detalhamos os conceitos associados ao Fluxo de Requisitos, diretamente relacionado à obtenção de informações dos clientes para se elaborar um documento que descreve suas necessidades.

Na Unidade II apresentamos o Fluxo de Análise associado à modelagem das informações obtidas durante o levantamento de requisitos, porém transformando-as em um nível mais próximo dos desenvolvedores. É apresentado também um método para contagem do tamanho de um sistema, baseado na Especificação de Requisitos e Análise.

Na Unidade III exibimos os exemplos completos de diversos artefatos utilizados para exemplificarmos o levantamentos de requisitos, convocação e exibição dos resultados de uma reunião, além da revisão dos requisitos e revisão da análise.

Sumário

11 UNIDADE 1
O FLUXO DE REQUISITOS

1.1 O Fluxo de Requisitos 13
Qualidade dos requisitos 14
Correção 14
Precisão 15
Completeza 15
Consistência 15
Verificabilidade 16
Modificabilidade 16
Rastreabilidade 16
1.2 Atividades do Fluxo de Requisitos 16
Definição do Escopo 17
Identificação dos Requisitos 17
Detalhamento dos Requisitos 17
Detalhamento dos Casos de uso 17
Definição dos Protótipos de Interface 18
Revisão dos Requisitos 18
Exercícios 40
1.3 Técnicas de Apoio ao Levantamento de Requisitos 42
Oficinas de requisitos 42
Personalização 43
Sessões 44
Fechamento 45
Revisões 45
Participantes 47
Conduções 48
Exercícios 50

53 **UNIDADE 2**
ANÁLISE DE SOFTWARE

2.1 A Linguagem UML 55
A Origem da UML 55
Visões 56
Modelo de Elementos 57
Classes 11
Objetos 59
Estados 59
Pacotes 60
Componentes 60
Relacionamentos 61
Mecanismos Gerais 65
2.2 Diagramas 65
Diagrama de Casos de uso 66
Diagrama de Classes 66
Diagrama de Objetos 66
Diagrama de Estados 67
Diagrama de Seqüência 68
Diagrama de Colaboração 69
Diagrama de Atividades 69
Diagrama de Componentes 70
Diagrama de Implantação 71
Considerações Finais 72
Exercícios 72
2.3 O Fluxo de Análise 73
Atividades do Fluxo de Análise 73
Identificação das Classes 74
Organização das Classes 81
Realização dos Casos de Uso 82
Revisão da Análise 84
2.4 Análise de Pontos de Função 85
Introdução a Análise de Pontos de Função 88
O Processo de Contagem 90
Contagem de Funções de Dados 95
Contagem de Funções de Transação 96
Contagem Estimativa de Pontos de Função 101
Contagem de funções de dados 102
Contagem de funções de transação 104
Planejamento e Gestão de Projetos com PF 108
Visão crítica da APF 109
Exercícios 111

Exemplos de Artefatos 115

UNIDADE 1

O fluxo de Requisito

Resumo

Nesta unidade apresentamos o Fluxo de Requisitos que está diretamente associado à obtenção de informações junto aos clientes sobre o problema a ser tratado. Em muitos projetos esse fluxo é pouco explorado, o que normalmente resulta no desenvolvimento de um software que não atende aos anseios dos usuários finais. No Capitulo 1 apresentamos as atividades prescritas no fluxo com a exemplificação de como realizar cada atividade, a partir da explicação de como executá-la no sistema exemplo. No Capítulo 2 apresentamos algumas técnicas utilizadas como forma de apoiar o levantamento de requisitos, mais especificamente técnicas para se conduzir uma reunião com o objetivo de se obter informações e construir um conceito em conjunto, além de técnicas para revisar artefatos produzidos durante o desenvolvimento de software.

1 O FLUXO DE REQUISITO

Um requisito pode ser definido como um desejo, necessidade, restrição ou expectativa de um cliente com relação a um produto. Ou seja, ao pensar em um produto, um cliente possui muitos aspectos em sua mente que necessitam ser capturados, definidos, organizados, verificados e validados para que se chegue a uma Especificação de Requisitos. Esse documento é o principal artefato gerado no início do desenvolvimento de software. Seu principal objetivo é prover um enunciado completo, claro e preciso dos requisitos de um produto de software.

Um requisito pode ser resumido como algo desejado por um usuário em relação a um produto.

Logicamente, a geração de uma especificação de requisitos para um produto novo é bem mais complexa que para produtos existentes. Em muitos casos, os próprios clientes não sabem ao certo o que querem. Na verdade, a maioria dos clientes consegue identificar bem o que não quer. Nesses casos, o uso da prototipação é algo muito recomendado. Mais adiante vamos detalhar essa técnica que pode auxiliar bastante o fluxo de requisitos.

É importante detalhar de forma precisa o que é o Fluxo de Requisitos. Esse fluxo reúne as atividades que visam obter o enunciado completo, claro e preciso dos requisitos de um produto de software. Os requisitos devem ser levantados pela equipe do projeto, normalmente denominados Analistas de Requisitos (ou Engenheiros de Requisitos) em conjunto com representantes do cliente, usuários chaves que contam com a presença de especialistas da área de aplicação, uma vez que o projeto pode envolver conhecimentos não triviais que exijam a presença de profissionais altamente especializados com a área de negócio do produto a ser construído.

O conjunto de técnicas empregadas para levantar, detalhar, documentar e validar os requisitos de um produto compõe a Engenharia de Requisitos

O conjunto de técnicas empregadas para levantar, detalhar, documentar e validar os requisitos de um produto compõe a Engenharia de Requisitos. O resultado principal do fluxo dos requisitos é o documento de Especificação de Requisitos (ER).

1.1 Qualidade dos requisitos

Os requisitos de um software correspondem a uma parte muito importante do desenvolvimento. Por conta disso, é necessário que eles possuam diversas características de qualidade, permitindo assim seu uso de forma adequada e eficiente. A seguir apresentamos uma lista de características de qualidade de requisitos, baseadas nas características de qualidade de requisitos existentes segundo Paula Filho (2003).

As características citadas nesta seção podem ser utilizadas como critérios para se realizar uma revisão de requisitos, conforme será apresentado mais a frente. Mas para isso, é necessário entender o que está relacionado a cada uma das características.

Correção

Uma Especificação de Requisitos é correta se todo requisito presente nela realmente é um requisito do produto a ser construído. Não existe ferramenta que garanta a correção de uma Especificação de Requisitos. Para verificá-la, deve-se checar a coerência da Especificação dos Requisitos do Software com outros documentos da aplicação como manuais, protocolos, regras de negócio, etc. Outra forma de tentar garantir a correção é solicitar a aprovação formal da ER, por parte do cliente, sem a qual o projeto não poderá prosseguir.

Por conta disso é que normalmente ao final de um levantamento de requisitos é feita uma revisão do trabalho executado. A ideia por trás disso é encontrar falhas na qualidade dos requisitos que podem estar associadas a qualquer uma das características apresentadas nessa seção.

Precisão

Uma ER é precisa se todo requisito presente possuir apenas uma única interpretação, aceita tanto pelos desenvolvedores quanto pelos usuários chaves. Em particular, uma ER deve ser compreensível para todo o seu público alvo e deve ser suficiente para a especificação dos testes de aceitação do produto. Recomenda-se a inclusão no glossário da ER de todos os termos contidos no documento que possam causar ambigüidades de entendimento.

Uma forma de verificar a precisão de uma ER é solicitar sua leitura por pessoas que não tenham participado da sua elaboração, para analisarmos se é possível entender o que foi especificado de forma precisa.

Completeza

Uma ER é completa se reflete todas as decisões de especificação que foram tomadas, não contendo cláusulas de pendências. Uma ER completa deveria conter todos os requisitos significativos relativos à funcionalidade, desempenho, restrições de desenho, atributos e interfaces externas, além de definir as respostas do software para todas as entradas possíveis, válidas e inválidas e em todas as situações possíveis.

Também é fundamental para uma ER possuir um glossário de todos os termos técnicos e unidades de medida, facilitando assim seu entendimento por todos.

É bastante comum que organizações criem suas pseudo ERs contendo apenas um pedaço da especificação do comportamento do software desejado, normalmente ignorando os requisitos não funcionais.

Consistência

Uma ER é consistente se não há conflitos entre nenhum dos subconjuntos de requisitos presentes, tais como conflitos com o mundo real como, por exemplo, formatos de relatórios ou cores de sinalização; conflito lógico ou temporal entre ações, quando, por exemplo, um requisito diz que a ação A deve ser realizada antes da ação B, e outro diz o contrário; e uso de termos diferentes para designar o mesmo objeto do mundo real, como, por exemplo, “lembrete” versus “aviso”.

Mais uma vez notamos que uma revisão é fundamental para o fechamento de uma ER, pois apenas a partir de uma ação como essa é que poderemos descobrir certas incorreções.

Priorização

Uma ER é priorizada se cada requisito é classificado de acordo com a respectiva importância e estabilidade. A estabilidade estima a probabilidade de que o requisito venha a ser alterado no decorrer do projeto, com base na experiência de projetos correlatos. A priorização classifica o requisito de acordo com graus de importância atribuídos pelos clientes.

A priorização é algo importante, pois normalmente os custos e prazos podem ser bastante limitados, sendo importante descrever o que é mais importante e deve ser tratado primeiro. Da mesma forma, aquilo que ainda está em processo de mudança não pode ser considerado para implementação, pois ainda não é estável e qualquer ação aplicada pode ser trabalho perdido

Verificabilidade

Uma ER é verificável se todos os seus requisitos são verificáveis. Um requisito é verificável se existir um processo finito com custo compensador, que possa ser executado por uma pessoa ou máquina mostrando conformidade do produto final com o requisito.

Modificabilidade

Uma ER é modificável se sua estrutura e estilo permitirem a mudança de qualquer requisito, de forma fácil, completa e consistente. A modificabilidade geralmente requer que haja uma organização coerente, com índices e referências cruzadas; ausência de redundância entre requisitos; definição separada de cada requisito.

Essa característica está diretamente relacionada ao uso de padrões e convenções, de forma que o trabalho seja feito através de formatos pré-definidos e adequados ao uso.

Rastreabilidade

Uma ER é rastreável quando permite a fácil determinação dos itens que antecederam o surgimento deste, os quais foram gerados por conta da existência do mesmo. Isso normalmente está associado a dois tipos de rastreabilidade:

1. Rastreabilidade para trás, na qual deve ser possível localizar a origem de cada requisito. Deve-se sempre saber por que existe cada requisito e quem ou o que o originou. Isso é importante para que se possa avaliar o impacto da mudança daquele requisito e extinguir dúvidas de interpretação.
2. Rastreabilidade para frente, na qual deve ser possível localizar quais os resultados do desenvolvimento que serão afetados por cada requisito. Isso é importante para garantir que os itens de análise, desenho, código e testes abranjam todos os requisitos; para localizar os itens que serão afetados por uma mudança nos requisitos.

1.2 Atividades do Fluxo de Requisitos

O Fluxo de Requisitos é composto por várias atividades que devem ser executadas dentro de um processo de desenvolvimento. Diversos processos de software possuem atividades de requisitos em suas definições. As atividades que serão explanadas neste texto representam as atividades mais comuns relacionadas à Engenharia de Requisitos. No

entanto, a análise de um processo de software específico pode conter diversas diferenças para as atividades aqui apresentadas.

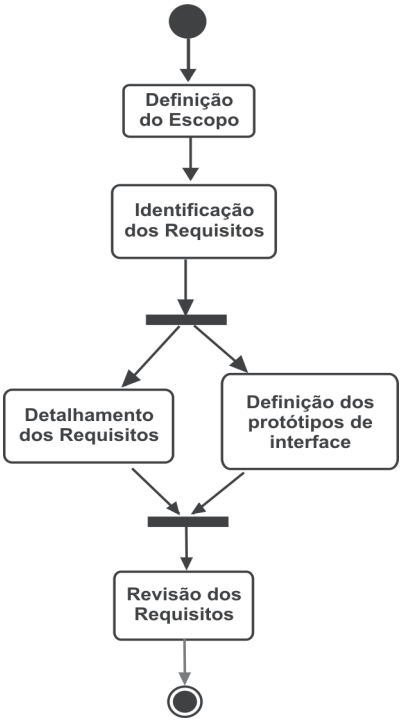


Figura 1: Atividades do Fluxo de Requisitos

As atividades do Fluxo de Requisitos são exibidas na Figura 1. A figura exibe um diagrama de atividades. O primeiro elemento na figura (uma circunferência preta) é conhecido como atividade inicial, representando o ponto de partida do fluxo. Os demais retângulos com os lados arredondados representam atividades. Existem duas barras de sincronização na figura, representando que as atividades posteriores só iniciam quando todas as atividades com ligação à sincronização tenham encerradas. Assim, conforme exibido, a Revisão dos requisitos apenas tem inicio quando o Detalhamento dos Requisitos e Detalhamento dos Protótipos de Interface tenha sido concluído.

A Definição do Escopo é a atividade onde o escopo do projeto vai ser identificado. De forma geral, a partir do escopo deve ser possível identificar o que faz parte do projeto e o que não faz.

A Identificação dos Requisitos é a atividade relacionada à obtenção de tudo o que os clientes desejam com relação ao produto. Isso inclui a definição do comportamento esperado, bem como outros aspectos que deverão ser identificados.

O Detalhamento dos Requisitos é a atividade em que os desenvolvedores com a ajuda dos clientes desdobram os requisitos em

funções do produto, de forma que o atendimento seja completo.

A Definição dos Protótipos de Interface é a atividade em que versões iniciais das interfaces do produto são criadas com o intuito maior de deixar claro o que se deseja, reduzindo assim problemas com requisitos questionáveis ou difíceis de serem entendidos.

Por fim, a Revisão dos Requisitos é a atividade em que é feita uma revisão geral do trabalho realizado, com o intuito de remover problemas com relação aos requisitos identificados e todos os seus desdobramentos executados.

Definição do Escopo

O Escopo de um Projeto é algo fundamental para o seu sucesso. Sua definição é algo considerado simples, porém deve ser feita de forma prudente e com a participação dos principais envolvidos.

Em muitos momentos, o escopo do projeto deverá ser reanalisado, pois define o que está e o que não está incluído no projeto em questão.

Uma boa forma de evitar problemas com os clientes de um projeto é definir bem o escopo e, para evitar falsas expectativas, detalhar o que não faz parte dele. Por isso, é tão importante termos uma seção com os Limites do Produto.

Um escopo mal estruturado poderá levar a falhas de cronograma e de orçamento, uma vez que tarefas acima do previsto podem ser consideradas, ao invés daquilo o que realmente deveria ter sido incluído.

De forma geral, o escopo de um projeto pode ser simplesmente um texto que define o que deve fazer parte do projeto. Neste material, utilizaremos um exemplo para demonstrar todas as atividades aqui descritas. Será usado como exemplo prático algo muito apropriado ao tema: uma ferramenta para Gerência de Requisitos. Esse projeto será denominado ReqG. Para isso, precisamos definir o escopo de um projeto cujo objetivo é produzir uma ferramenta para gerenciamento de requisitos. Uma sugestão de tal escopo pode ser visualizada a seguir:

> *Gerenciar os requisitos de um projeto de software, registrando todos os dados associados à sua definição, de forma a cobrir todas as atividades previstas no fluxo de requisitos.*

Na definição acima, podemos notar que o trabalho a ser realizado foi definido. Certamente, nem todos os que chegarem a ler o enunciado poderá entender por completo o que deve ser feito. Isso acontece por que apenas as pessoas com conhecimento no problema e no que está sendo definido poderiam entender por completo essa definição. No entanto, ela poderá ser revista em diversos momentos do projeto.

Algo comum à definição do escopo é definir também o que está

fora dele. Isso normalmente ajuda aos clientes entenderem de forma mais precisa o que está e o que não está incluído no projeto. A definição do que não está incluído de forma explícita evita falsas expectativas. Isso normalmente favorece o processo de comunicação com o cliente.

É importante ressaltar que o fluxo de requisitos é um fluxo com grande participação do cliente. Ele é quem define praticamente tudo o que será produzido nessa fase do desenvolvimento. Assim, quanto mais mecanismos utilizarmos para facilitar a comunicação com o cliente, melhores resultados serão obtidos com sua execução.

Continuando com nosso exemplo, uma forma interessante de definir o escopo seria detalhar o que não está incluído no projeto. Essa seção é normalmente conhecida como Limites do produto. Uma sugestão para tal seção seria a seguinte:

1. O ReqG não controlará a execução de tarefas no projeto, apenas registrará os dados relacionados a requisitos.
2. O ReqG não gerará documentos editáveis com a especificação de requisitos. Os dados ficam registrados no sistema e só podem ser alterados por ele.
3. O ReqG não controlará qualquer aspecto do custo ou do cronograma de execução.
4. O backup e a recuperação das bases de dados do sistema ficam a cargo da administração de dados e não serão providas pelo ReqG.
5. O ReqG não terá ajuda on line.

Uma dúvida que deve pairar naqueles que iniciam a elicitação de requisitos é justamente saber quem deveria participar dessa definição.

Importante: para se conhecer bem o que deve ser feito por um produto, devemos começar a conversa com aqueles que entendem um pouco de tudo (diretoria) para depois iniciarmos um aprofundamento nos detalhes (gerência, chefias).

Normalmente, qualquer organização possui alguns níveis hierárquicos em sua constituição. As pessoas no nível hierárquico mais alto geralmente possuem um bom conhecimento sobre tudo o que é realizado na organização. Isso acontece por que eles devem ser comunicados e devem acompanhar o que se passa dentro da instituição. Essas pessoas formam um grupo ideal para se utilizar na definição de um produto, pois conhece o todo. Ao aprofundarmos nas definições dos requisitos, necessitaremos da participação de outras pessoas, com conhecimentos mais aprofundados em detalhes específicos.

Assim, para levantar requisitos para uma aplicação que controle uma empresa, idealmente deveríamos conversar com os diretores da organização, uma vez que eles devem saber exatamente como a organização funciona. Em seguida, devemos conversar com os gerentes, pois normalmente existe um gerente para cada área prioritária da

organização. Ele conhece tudo o que se passa em seu setor. Para que seja possível detalhar o que realmente é feito em cada setor, deveríamos conversar com os funcionários que executam as atividades ou com os chefes de setores em um nível inferior aos gerentes.

Daremos início a um trabalho prático que deverá ser feito por cada aluno da disciplina, o qual acompanhará este material: a definição de um Software de Controle de Empréstimos Pessoais. Esse software será definido por você leitor, ao longo deste material. De início, procure definir o escopo e os limites do produto, de forma similar ao que fizemos com o software de gestão de requisitos. Utilize os modelos e o exemplo de ERSs que estamos apresentando como exemplo e que está contido na página da disciplina.

Requisitos funcionais estão associados a comportamento. Requisitos não funcionais estão ligados a características que o comportamento deve possuir.

Identificação dos Requisitos

A Identificação dos Requisitos é a atividade que prescreve a criação de uma lista dos requisitos para a aplicação a ser desenvolvida. Cada requisito nada mais é que uma descrição textual de algo que deveria ser considerado durante o desenvolvimento de software.

Os requisitos podem ser divididos em dois tipos: requisitos funcionais e não funcionais. Os funcionais estão diretamente ligados ao comportamento que a aplicação deve conter. Por exemplo, em um sistema de controle de uma biblioteca, o Empréstimo de Livro exige que o usuário a tomar um livro emprestado esteja cadastrado e ativo; que o livro desejado esteja disponível, não reservado e não seja cativo; que o usuário não esteja com cinco livros emprestados, considerando que esse seja o número máximo de empréstimos simultâneos permitido. Tudo isso que foi comentado são detalhes do comportamento que o software deve ter.

Outro tipo de requisito é o não funcional. Nesse caso, eles não expressam comportamento, mas características que o comportamento deve ter. Por exemplo, supondo o mesmo requisito Empréstimo de Livro, podemos exigir como requisito de desempenho que ele funcione com até 100 usuários simultâneos, gerando respostas em até 8s. Outro requisito não funcional associado ao empréstimo seria que ele fosse simples de usar, permitindo que uma pessoa conseguisse utilizá-lo apenas lendo o manual associado. Observe que tudo o que relatamos neste parágrafo não trata de comportamento, mas de características desejadas ao comportamento especificado no parágrafo anterior.

Conforme comentado, a lista dos requisitos é a expressão que

resume aquilo que os clientes desejam. É importante a participação dos Analistas de Requisitos para que seja possível organizar esses pedidos e estruturar o texto que os descreve de forma que seja possível analisar e desdobrar esses pedidos em funções do produto. A Tabela 1 exibe a lista de requisitos para a ferramenta de gestão de requisitos que queremos construir.

ID	Requisito	Descrição
RF1	Requisitos	O sistema deve permitir cadastrar e controlar todos os aspectos relacionados aos requisitos de um projeto, permitindo visualizá-lo e acompanhar sua evolução, incluindo as pessoas que trabalharam no projeto, analistas e gerente do projeto.
RF2	Casos de uso	O sistema deve possibilitar a especificação dos casos de uso, registrando sua descrição, atores, protótipos de tela associados, relacionando os requisitos que deram origem ao caso de uso.
RF3	Revisão	Deve ser possível realizar uma revisão dos requisitos e casos de uso, utilizando critérios definidos pelos próprios gerentes de projetos, de forma independente aos demais projetos.
RF4	Acompanhamento dos clientes	Deve ser possível que os clientes possam acompanhar a evolução do projeto a qualquer momento, consultando tudo o que foi feito.
RF5	Liberação de acesso por projeto	Deve ser possível liberar o acesso ao sistema por projeto, indicando o seu gerente. Assim, para se ter acesso ao sistema, deverá ser adquirido uma licença para um projeto. A partir disso, o gerente ficará responsável por definir quem deverá usar o sistema, seja para trabalhar na especificação de requisitos como um dos Engenheiros de Requisitos, seja como cliente consultando o resultado do trabalho realizado.
RF6	Geração da documentação	Deve ser possível gerar a especificação na forma de um documento não editável, contendo todos os dados registrados do projeto.

Tabela 1: Exemplo de definição de requisitos

Podemos notar que os requisitos exibidos na Tabela 1 são simples e exibem as necessidades de alguém que trabalha com requisitos. Podemos notar também que muito precisa ser detalhado para que tenhamos algo pronto para implementar.

A Tabela 2 exibe a lista de requisitos não funcionais identificados para o produto. Observe que também são definições simples, mas que retratam o que se deseja com relação a certas características ligadas ao comportamento do software. São exemplos disso a definição do ambiente (Web), o uso de uma tecnologia específica (MySQL) ou a definição de valores para atributos de qualidade como a quantidade de acessos simultâneos e o tempo máximo de resposta.

É importante ressaltar que requisitos não funcionais necessitam da definição de valores que permitam sua verificação. Dizer que o sistema deve ser rápido não ajuda muito aos testadores que devem verificar se o produto atende à ER. Mas especificar um tempo máximo de resposta em um contexto pré-definido ajuda bastante.

ID	Requisito	Descrição
RNF1	Ambiente	O sistema deve funcionar em ambiente Web, sendo compatível com os principais navegadores do momento: Internet Explorer, Firefox, Safari e Chroma.
RNF2	Linguagem	O sistema deve ser desenvolvido utilizando-se a linguagem Java, com as tecnologias JSF e Hibernate.
RNF3	Banco de dados	O sistema deve utilizar o banco de dados MySQL.
RNF4	Desempenho	O sistema deve ser construído para funcionar com 100 usuários simultâneos, com respostas de até 5s quando utilizado em rede local.
RNF5	Segurança	O sistema deve restringir o acesso por meio de senhas. Deve-se ter um controle no registro de senha, de forma a impedir o uso de senhas consideradas fáceis.
RNF5	Segurança	O sistema deve restringir o acesso por meio de senhas. Deve-se ter um controle no registro de senha, de forma a impedir o uso de senhas consideradas fáceis.

Tabela 2: Exemplo de requisitos não funcionais

O desenvolvimento de software pode ser entendido como uma série de transformações que nos levam aos desejos dos clientes até um produto executável que atende a tais desejos. O início das transformações acontece quando iniciamos uma série de reuniões para entender o que os clientes desejam. Essa lista, normalmente é abstrata, devendo ser melhor detalhada sob diversos aspectos, até que possa ser utilizada como guia para a implementação.

Na identificação dos requisitos, todos os aspectos levantados pelos clientes devem ser registrados, da forma mais parecida com o que foi relatado. Nas próximas atividades esses aspectos serão transformados em elementos que permitem seu detalhamento de forma estruturada. No entanto, isso não acontece nesse momento.

Mais uma vez ressaltamos que essa atividade necessita ser realizada com pessoas que possuam uma boa visão geral do todo. A ideia é obter uma descrição de alto nível do que precisa ser feito, deixando para depois um detalhamento aprofundado de cada questão.

Nesse momento você deverá fazer o levantamento dos requisitos relacionados ao Software de Controle de Empréstimos Pessoais que iniciamos anteriormente. Crie uma lista de requisitos desejados por você com relação a esse produto. Utilize o exemplo que disponibilizamos na página para iniciar o trabalho. Dessa forma, você deve ir alterando as seções que estão sendo descritas.

Detalhamento dos Requisitos

O Detalhamento dos Requisitos é a atividade em que cada requisito identificado deve ser desdobrado em funções do produto. Cada função do produto pode estar ligada a um único requisito, assim como pode estar relacionado a mais de um requisito.

O desdobramento de requisitos gera a identificação dos Casos de Uso. Esse é um termo comum na Engenharia de Requisitos e que precisa ser entendido. Um Caso de Uso é uma fatia de funcionalidade do sistema que representa algo de valor para seus usuários e que não apresenta nem lacunas nem superposição com outros casos de uso.

Os casos de uso são acionados por atores. Um ator é a representação de um papel no sistema. Atores podem representar um grupo de usuários, outro sistema, com o qual o sistema sendo especificado deve interagir. Um caso de uso normalmente interage com apenas um ator, mas pode interagir com mais de um. Segundo Pádua Filho (2003), atores podem ser identificados através dos seguintes critérios:

Casos de uso podem ser considerados as funções que um produto deve oferecer.

1. Quem está interessado em certo requisito;
2. Quem se beneficiará diretamente do produto;
3. Quem usará informação do produto;
4. Quem fornecerá informação ao produto;
5. Quem removerá informação do produto;
6. Quem dará suporte e manutenção ao produto;
7. Quais os recursos externos usados pelo produto;

8. Quais os papéis desempenhados por cada usuário;
9. Quais os grupos de usuários que desempenham o mesmo papel;
10.Quais os sistemas legados com os quais o produto deve interagir;
11. Quando casos de uso são disparados periodicamente, o tempo é de forma automática.

Atores são usados para representar também sistemas externos. Estes podem incluir sistemas legados, produtos comerciais de software e outros componentes de um sistema maior. Podem incluir recursos de hardware e comunicação que devam receber um tratamento específico por parte do produto (por exemplo, dispositivos de hardware que normalmente não fazem parte do ambiente operacional). Um ator especial é o Tempo, usado para acionar casos de uso que são disparados periodicamente, de forma automática.

Cada diagrama de casos de uso especifica os relacionamentos entre casos de uso e atores. Os relacionamentos indicam a existência de comunicação entre atores e casos de uso. Um caso de uso pode estar associado a mais de um ator quando a sua execução requer a participação de diferentes atores.

Normalmente, a comunicação será representada como ligação sem direção. Nesses casos, é convencionado que a iniciativa de comunicação parte do ator. Quando a iniciativa parte do caso de uso (por exemplo, alarmes, mensagens, dados enviados para outros sistemas, etc.), a comunicação deve ser direcionada para o ator.

Atores modelam papéis associados ao uso do produto. Papéis e não pessoas!

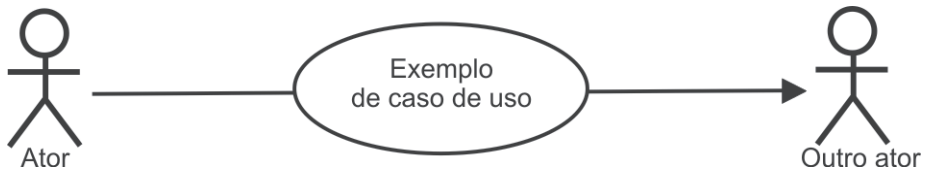


Figura 2: Exemplo de ator e caso de uso.

A Figura 2 exibe um exemplo de como é representado um caso de uso e atores em UML. O caso de uso Exemplo de caso de uso está associado a dois atores: o Ator e o Outro ator. Observe que o relacionamento entre o caso de uso, o ator e Outro ator possuem uma seta com indicação de direção. Isso mostra o fluxo de dados no relacionamento. Dessa forma, as informações devem fluir do caso de uso para o ator.

Assim, após a identificação dos requisitos, é necessário detalhar quais casos de uso serão necessários para atender aos desejos identificados pelos clientes. Nesse momento inicia-se uma tarefa um pouco mais nobre dos Engenheiros de Requisitos que é justamente

ajudar aos clientes a pensar como estruturar o software, em termos de funções para atender aos anseios identificados.

Como exemplo, iremos analisar os requisitos contidos na Tabela 1. Inicialmente, utilizaremos o RF1. O texto que o descreve está detalhado no quadro a seguir:

> *O sistema deve permitir cadastrar e controlar todos os aspectos relacionados aos requisitos de um projeto, permitindo visualizar isso e acompanhar sua evolução, incluindo as pessoas que trabalharam no projeto, os analistas e o gerente do projeto.*

De acordo com o texto do requisito, o sistema deve possibilitar o cadastro e controle dos requisitos. Com isso, podemos identificar uma função do produto: Cadastro de requisitos. Esse então já é um dos casos de uso do sistema. Temos que lembrar que existem requisitos funcionais e não funcionais. Assim, teremos que decidir se iremos realizar esse cadastro em um único caso de uso ou se serão necessários dois casos de uso: um para o cadastro de requisitos funcionais e outro para requisitos não funcionais. Nesse caso em específico, vamos utilizar um único caso de uso para registro dos requisitos. Vamos nomear esse caso de uso como

Gestão de requisitos.

Os requisitos são textos que descrevem os desejos dos clientes. Simples assim!
Os casos de uso são as traduções dos requisitos em funções do produto, feita por Engenheiros de Requisitos, a partir de um entendimento do que foi solicitado e o que pode ter em um produto.

Observe que ainda no texto do RF1 existe a menção ao registro das pessoas que trabalharam no projeto. Isso significa que teremos que ter um cadastro dos envolvidos no projeto. Cadastraremos o gerente, os analistas e os clientes envolvidos. Com isso, é possível identificar que teremos que cadastrar os membros do projeto. Isso dá origem a mais um caso de uso: Gestão de membros.

Continuando a análise, podemos concluir que para atender o RF2 é necessário termos um caso de uso para a Gestão de Casos de Uso. No entanto, teremos também que registrar os atores associados ao caso de uso. Isso nos remete a um novo caso de uso: Gestão de atores.

O RF3 nos remete diretamente a novos casos de uso: Revisão. No entanto, foi descrito que as revisões serão guiadas por critérios independentes para cada projeto. Isso nos leva a identificar um novo caso de uso: Gestão de critérios.

O RF4 solicita que exista uma forma de acompanhar o projeto. Isso pode ser resumido a partir de uma função do produto que gerasse a especificação em um formato contendo todos os dados do projeto,

incluindo requisitos, casos de uso, atores, etc. Por conta disso, o RF4 nos levou a identificar mais um caso de uso: Geração da especificação.

O RF5 refere-se ao modelo de negócio do sistema. Para que alguma pessoa possa utilizá-lo, é necessário que seja realizado o cadastro de um projeto. Esse projeto terá um gerente associado que a partir de então poderá cadastrar seus analistas, clientes, requisitos, casos de uso, etc. Dessa forma, é necessário que o sistema tenha um caso de uso Cadastro de Projeto, que deve ser feito por um administrador, responsável por liberar acesso ao software. No entanto, um projeto possui diversos dados adicionais como seu escopo com datas de execução, limites do produto que também precisam ser cadastrados. Isso nos leva a ter outro caso de uso para que seja possível registrar tais dados, mas utilizado pelo gerente do projeto e não pelo administrador do sistema. Esse caso de uso será chamado de Controle do projeto.

A exibição dos casos de uso e os requisitos que foram utilizados para sua geração, nos dão uma idéia da rastreabilidade dos requisitos.

Analisando o RF6 que trata da geração da documentação, encontramos algo interessante: a necessidade relatada já está contemplada com a proposição do caso de uso Geração da documentação, identificado quando analisamos o RF4. Dessa forma, ao propormos o caso de uso Geração da documentação, não só atendemos ao requisito RF4 como também ao RF6.

Pronto! Acabamos de identificar todos os casos de uso do sistema, a partir de uma leitura e interpretação dos requisitos expostos pelos clientes. Mais uma vez é importante ressaltar que embora tenhamos feito isso de forma direta neste documento, isso normalmente exige reuniões, discussões e um amadurecimento por parte dos clientes e envolvidos para que se chegue a uma determinação do produto a ser desenvolvido. A Figura 3 apresenta um diagrama de casos de uso com todos os casos de uso identificados, além dos atores associados.

O diagrama de casos de uso nos dá uma visao ampla do sistema, exibindo as funções existentes e os papéis associados.

Outro ponto importante de ressaltar é que isso não representa ainda o detalhamento dos requisitos, sendo necessário um maior aprofundamento dos requisitos. Isso inclui a determinação das regras de negócio associadas a cada um dos casos de uso que será feito na próxima subseção.

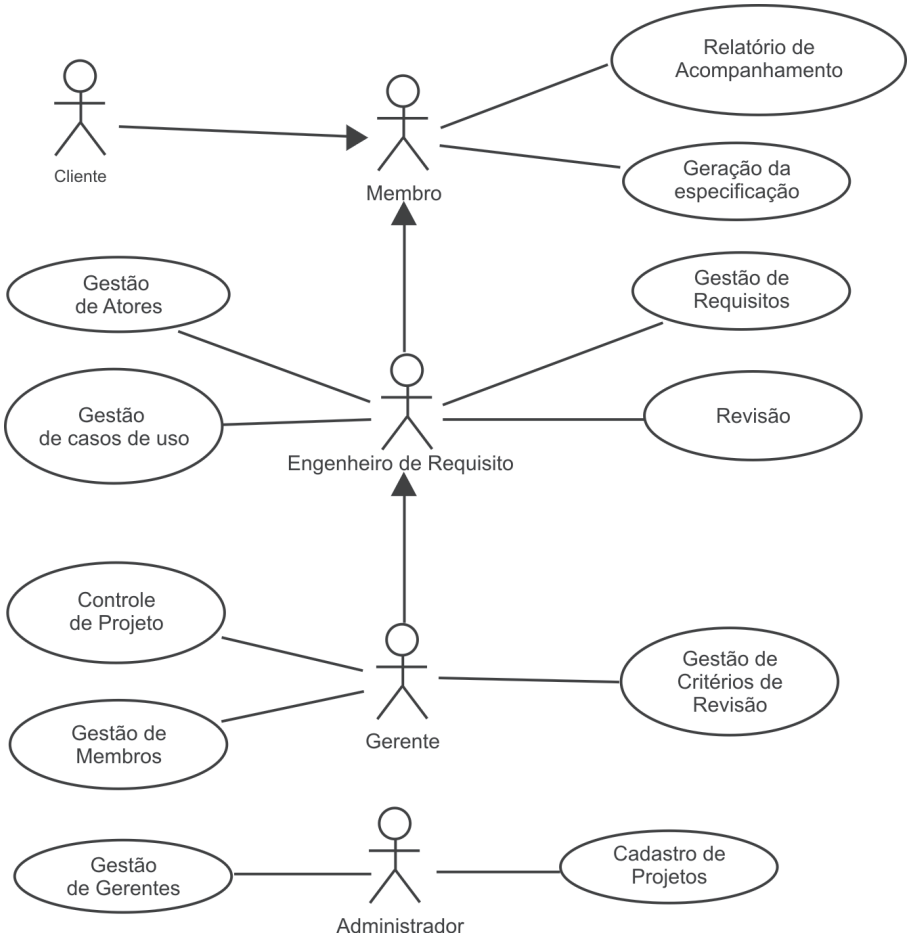


Figura 3: Diagrama de casos de uso para o ReqG.

A Tabela 3 exibe a lista de casos de uso identificados no sistema, com uma identificação dos requisitos associados. Essa ligação entre caso de uso e requisito é conhecida como rastreabilidade. Isso nos permite saber quem deu origem a quê. Assim, é possível saber que um requisito deu origem a um determinado caso de uso.

ID	Caso de uso	Requisito associado	Descrição
UC1	Gestão de Requisitos	RF1	Cadastro de requisitos funcionais e não funcionais associados ao projeto.

A exibição dos casos de uso e os requisitos que foram utilizados para sua geração, nos dão uma idéia da rastreabilidade dos requisitos.

ID	Caso de uso	Requisito associado	Descrição
UC2	Gestão de Membros	RF1	Cadastro de todos os envolvidos no projeto.
UC3	Gestão de Casos de Uso	RF2	Definição dos casos de uso do projeto.
UC4	Gestão de Atores	RF2	Definição dos atores que irão interagir no projeto.
UC5	Revisão	RF3	Revisão de um requisito ou de um caso de uso, observando os critérios pré-estabelecidos no projeto para a revisão.
UC6	Gestão de Critérios de Revisão	RF3	Cadastro de critérios a serem utilizados em uma revisão.
UC7	Cadastro de Projeto	RF5	Cadastro de um projeto, com definição do seu gerente, feito pelo administrador do sistema.
UC8	Gestão de Gerentes	RF5	Cadastro de um gerente de projeto, feito pelo administrador do sistema.
UC9	Controle de Projetos	RF5	Controle do projeto, com detalhamento de informações sobre o mesmo, feito pelo gerente do projeto.
UC10	Geração da especificação	RF6, RF4	Geração da especificação de requisitos, utilizando um formato pré-definido, contendo todos os dados registrados no projeto.
UC11	Relatório de Acompanhamento	RF6, RF4	Emissão de um relatório contendo uma indicação do estado do projeto, a partir do estado de cada caso de uso que o compõe.

Tabela 3: Lista de casos de uso identificados.

A Tabela 4 exibe a lista de atores identificados. É importante ressaltar que a identificação de atores muito nos facilita o entendimento do produto. Por isso, é fundamental que ao se pensar em uma função do sistema, haja também uma reflexão sobre que grupo deveria utilizar tal função.

Nº	Ator	Descrição
1.	Cliente	Clientes de um projeto, normalmente responsável pelo fornecimento de informações para moldagem do produto.
2.	Administrador	Responsável pelo controle do uso do sistema, liberando acesso aos gerentes a partir do cadastramento de um projeto.
3.	Gerente	Responsável pelo controle de um projeto, definindo a equipe e suas tarefas.
4.	Membro	Pessoa que faz parte da equipe que trabalha no projeto.

Tabela 4: Lista de atores do Projeto

Dando prosseguimento ao nosso trabalho relacionado ao Software de Controle de Empréstimos Pessoais, é necessário identificar os casos de uso e atores associados ao sistema.

Detalhamento dos Casos de uso

Conforme demonstrado na Seção anterior, o Detalhamento dos Requisitos inicia pela identificação dos casos de uso relacionados às necessidades relatadas pelos fornecedores de requisitos. No entanto, a identificação dos casos de uso é apenas o início do detalhamento. A partir da identificação de um caso de uso, sabemos que uma determinada função deverá existir no sistema. Mas será necessário detalhar como será essa função. Isso inclui a descrição desses casos de uso, especificando como eles deverão funcionar.

A atividade de Detalhamento dos Casos de Uso é uma subatividade do Detalhamento dos Requisitos. Existem muitas formas de ser descrever um caso de uso. Uma forma muito utilizada e adotada neste trabalho é utilizar o conceito de fluxos dos casos de uso.

O detalhamento dos fluxos dos casos de uso é uma tarefa onerosa e muito importante para a correta especificação do funcionamento de parte de um produto. Cada fluxo detalha passo a passo o que deve acontecer em determinada parte do produto. Essa descrição é geralmente feita por meio de textos seguindo formatos pré-estabelecidos. Notações muito informais são chamadas histórias de usuários (user stories) e são comumente adotadas por metodologias ágeis.

Cada caso de uso deve possuir ao menos a descrição de suas pré-condições, assim como um fluxo principal que representa um caminho de execução que normalmente é o mais utilizado para o caso de uso.

Um exemplo de fluxo principal é apresentado a seguir. Nele, é possível observar que existem passos relacionados com ações do sistema (ReqG) e passos relacionados a ações do ator (Administrador).

Fluxo Principal

*1. **O ReqG** exibe a Tela de Gestão de Gerentes.*
2. O Administrador informa os dados para pesquisa por Gerentes.
3. O Administrador aciona o comando Pesquisar.
4. O ReqG recupera e exibe na lista Gerentes recuperados os Gerentes que atendem aos parâmetros de pesquisa informados, ordenados pelo Nome em ordem crescente.

Os fluxos são comumente descritos em linguagem natural na forma de sequência de passos. Cada passo corresponde a uma ação de um ator ou do produto que devem aparecer explicitamente como sujeitos da frase. Outros atores podem aparecer como objetos verbais de uma ação. Nesse caso, provavelmente tais atores estejam ligados ao caso de uso utilizando setas que indicam a direção da comunicação no diagrama de casos de uso. Condições e interações podem aparecer nas descrições dos casos de uso (se alguma coisa, para cada coisa faça isso).

Um detalhe importante é que a descrição dos casos de uso apresentados nesse trabalho necessitará de um protótipo de interface com o usuário. Isso será descrito na próxima seção. Por enquanto, iremos nos ater à descrição do caso de uso. No entanto, utilizaremos parte de um exemplo que será completamente incluído como anexo deste trabalho. Uma leitura detalhada no Anexo I permitirá um bom entendimento dos conceitos apresentados, uma vez que o anexo descreve um sistema real em que foi necessário aplicar todos os conceitos apresentados neste trabalho.

Além do fluxo principal que descreve uma parte do caso de uso que provavelmente seja a mais utilizada, existem fluxos alternativos e subfluxos. Os fluxos alternativos ou fluxos excepcionais são descrições de alternativas de execução que podem ser iniciadas sempre que suas pré-condições forem atendidas. Um exemplo disso é apresentado a seguir.

Fluxo alternativo: Inclusão de um Novo Gerente

Précondições	*1 - O Administrador acionou o comando Novo.*
Passos	*1 - O Administrador preenche os Dados do Gerente.* *2 - O Administrador aciona o comando Salvar.* *3 - O ReqGverifica que não existe um Gerente com email e login informados.* *4 - O ReqGsalva os Dados do Gerente.*

O levantamento de requisitos deve detalhar o desejo dos clientes. Não devemos introduzir especificidades de desenho ou implementação durante essa atividade. Por isso não é aconselhado detalhar mensagens ao usuário, tecnologias, etc.

Observe que na descrição do fluxo alternativo acima existe a especificação de pré-condições que detalham o que deve acontecer para que o fluxo entre em execução. No caso, para que a inclusão de um novo gerente aconteça, é necessário que o ator associado ao caso de uso acione o comando novo, pois essa é a indicação que se deseja que o fluxo seja executado.

Nesse fluxo também temos algo muito interessante e fundamental na descrição dos casos de uso: a especificação de restrições no seu funcionamento. No passo 3 do fluxo é especificado que o sistema (ReqG) verificará se uma determinada condição é atendida. Isso significa que o fluxo só continuará se ela for verdade. Caso não seja, o fluxo não continuará sua execução. Nesse caso, foi especificada uma condição simples relacionada à verificação da existência de um gerente com determinadas informações. No entanto, poderíamos ter especificado condições bem mais complexas que serão traduzidas nas diversas regras de negócio de um produto durante sua implementação.

Outro detalhe importante que devemos ressaltar é que não precisamos definir mensagens ao usuário neste momento. A maioria dos iniciantes na descrição de casos de uso é tentado a descrever a verificação contida no passo 3 do fluxo alternativo exibido anteriormente contendo uma mensagem ao usuário, normalmente da seguinte forma: O ReqG verifica se não existe um gerente com email informado. Se existir o ReqG emite a mensagem Gerente já cadastrado! Mas qual o problema em especificar mensagens durante o detalhamento dos casos de uso? Mensagens ao usuário fazem parte do projeto (design) do sistema, uma etapa posterior aos requisitos e análise. As mensagens devem seguir convenções e padrões de usabilidade, não sendo adequado defini-las em um momento em que isso não é o foco. Assim, é bem melhor apenas identificar as restrições e deixar para o momento apropriado a definição completa e correta das mensagens.

Os subfluxos são utilizados para descrever conjuntos de passos que foram extraídos de algum fluxo por serem grandes e complexos ou com potencial de serem reutilizados em outros fluxos. Seria algo equivalente a extrair um método de outro método. Para acionar a execução de um subfluxo é necessário especificar isso de forma direta: O ReqG executa o Subfluxo X.

Em casos de uso do tipo CRUD (Create, Read, Update, Delete) que descrevem um cadastro, normalmente contendo funcionalidades para se cadastrar, pesquisar, alterar e excluir algo, uma dúvida comum é: qual dessas funcionalidades deve ser especificada no fluxo principal do caso de uso? A resposta é: qualquer uma. Mas lembre-se que normalmente utilizamos no fluxo principal a funcionalidade que é mais comumente utilizada. Uma convenção utilizada neste trabalho foi sempre utilizar as pesquisas como funcionalidade descrita no fluxo principal de casos de uso do tipo CRUD.

Em geral, os casos de uso não possuem pré-condições e alguns tem apenas o fluxo principal. Embora existam diversos formatos utilizados para sua descrição, conforme o formato apresentado nos exemplos contidos neste texto, o mais importante é que as descrições utilizadas sejam inteligíveis para quem as lê. Dessa forma, o bom senso é o maior guia para a descrição dos casos de uso: se uma pessoa consegue ler e entender o que tem descrito com condições de criar um programa que programe as descrições, então o caso de uso está bem descrito. No entanto, é importante ressaltar que os iniciantes na descrição de casos de uso nem sempre deixam lacunas na descrição que impossibilitam o seu entendimento. Isso é muito comum e tende a ser reduzido com a experiência. Para finalizar essa seção, apresentamos a descrição de um caso de uso um pouco mais complexo que detalha as regras para geração de um relatório associado ao nosso exemplo, contido no Anexo I deste texto.

Fluxo Principal

O ReqG exibe a Tela de Relatório de Acompanhamento.
O Membro informa os dados para pesquisa por Projetos.
O Membro aciona o comando Pesquisar.
O ReqG recupera e exibe os Projetos que atendem aos parâmetros de pesquisa informados na lista Projetos Recuperados e que tenham como Membro no projeto o usuário corrente, ordenados pelo nome do Projeto em ordem crescente.

O Membro aciona o comando Gerar Relatório.
Para cada requisito contido no projeto:
O ReqG imprime uma linha com o ID, nome, descrição, tipo e estado do requisito, calculado a partir do estado ou a partir dos casos de uso associados.
Para cada caso de uso associado ao requisito:
O ReqG imprime uma linha com o ID, nome, descrição e estado do caso de uso.
O ReqG soma o percentual de conclusão de cada caso de uso, de acordo com o seu estado, sendo Identificado (10%), Detalhado (25%), Implementado (75%) e Homologado (100%).
Se o requisito possui casos de uso associados:
O ReqG calcula o percentual de conclusão do requisito a partir da soma de todos os percentuais dos casos de uso, dividido pela quantidade de casos de uso.
Se não:
O ReqG calcula o percentual de conclusão do requisito a partir do estado do requisito, ou seja, a partir da soma de todos os percentuais dos requisitos divididos pela quantidade de requisitos existentes.

Esse exemplo possui alguns pontos interessantes. Ele possui claramente definido em cada passo o seu responsável, que normalmente é o sistema ou o ator. Existe a descrição de diversas regras de negócio, detalhando como calcular alguma coisa. Para isso, foram utilizados condicionais (se) e interações (para). Ressaltamos que apenas a leitura do caso de uso não nos permite gerar o relatório detalhado, pois temos que analisar também uma sugestão de formato para tal relatório. Isso está descrito no Anexo I e será comentado na próxima seção que trata da definição dos protótipos de interface.

Nesse momento é necessário o detalhamento dos casos de uso identificados ao Software de Controle de Empréstimos Pessoais. Crie a descrição dos casos de uso, sempre levando em consideração a necessidade de termos restrições para algumas regras envolvidas no sistema.

Definição dos Protótipos de Interface

A Definição dos Protótipos de Interface especifica, de forma detalhada, os requisitos relacionados às fontes de entrada e saída de dados no produto.

Nas interfaces gráficas de usuário, existem questões que claramente representam requisitos dos produtos, tais como formatos de

dados e comandos. Outros detalhes, como formatos de telas e janelas, são aspectos de desenho da interface de usuário e não devem ser tratados durante a fase de Requisitos.

No entanto, a criação de um esboço da interface auxilia bastante a identificação de regras de negócio, dados a serem utilizados e formatos de campos. É extremamente importante definir que tecnologia utiliza para geração desses esboços, uma vez que eles não devem demandar muito esforço e nem deveriam focar em tecnologias específica, visto que isso pode limitar o espaço da solução sem que haja essa necessidade. Os esboços são apenas sugestões, mas que não deveriam ser seguidos rigorosamente na construção no produto. Eles servem muito mais para detalhar os dados envolvidos nos fluxos que propriamente para especificar formatos de tela. Exemplos de esboços podem ser construídos utilizando as seguintes tecnologias:

Os protótipos de interfaces, durante o levantamento de requisitos, devem ser focados em se descobrir as informações e restrições importantes ao requisito. Nenhum aspecto de execução ou usabilidade deveria ser tratado nesse momento.

1. Desenhos à mão livre, em papel;
2. Layout alfanumérico feitos com um editor de texto como o Word;
3. Layout feitos em um editor HTML, como o DreamWeaver;
4. Desenhos feitos com uma ferramenta de desenho técnico como o Pencil;
5. Telas desenhadas em um ambiente de desenvolvimento rápido como Delphi;
6. Telas desenhadas no ambiente definitivo de implementação, utilizando Java Swing.

Os esboços devem ser descritos de forma que o usuário consiga entender seu objetivo e entenda seu funcionamento, independente da tecnologia a ser utilizada.

Os campos e comandos existentes nos protótipos devem representar requisitos relacionados aos dados necessários para se implementar uma determinada função. É importante utilizar formatos independentes de tecnologia, para que sua definição final fique apenas para o Projeto (Design). Não é interessante tentar definir esse formato durante o levantamento de requisitos.

Neste trabalho iremos utilizar uma convenção para especificação de protótipos criados utilizando-se um editor de texto. Essa alternativa é bastante viável por conta da sua facilidade de manipulação, expressividade e, além disso, pelo fato de estar completamente dissociada de qualquer tecnologia de implementação.

Geração da Especificação		
Informações do Projeto		
Projeto	*SystemG* (texto com até 30 caracteres)	
Gerente	Silio Silvestre Ferreira Freitas (texto com até 100 caracteres)	
<Pesquisar>		
Projetos recuperados		
Projeto	**Descrição**	**Gerente**
SystemG	Criação de um sistema X	Silio Silvestre Ferreira Freitas
Frigo	Projeto muito interessante	Alberto Sobrinho Araújo
TecnoComp	Manutenção de algo	Silio Silvestre Ferreira Freitas
<Gerar Especificação>		

Figura 4: Exemplo de protótipo simples.

A Figura 4 exibe um exemplo de um protótipo simples para uma interface de usuário. Nela existem diversas convenções que guiarão a criação de outras interfaces. A primeira convenção está relacionada às cores utilizadas nas interfaces. Cada cor possui um significado, conforme detalhado na Tabela 5. Um campo com fundo branco indica que ele é editável, ou seja, o usuário poderá realizar alterações nos valores existentes. Campos com uma tonalidade cinza clara são campos com valores dinâmicos, preenchidos pelo sistema, mas que não podem ser alterados pelo usuário. As demais partes do protótipo que possuem uma tonalidade cinza mais acentuada são partes fixas e rótulos, que normalmente não são mutáveis.

	Campo alterável.
	Campo não alterável.
	Título de interface, rótulo de campo ou comando.

Tabela 5: Exemplo de protótipo simples.

Na Tabela 5 é mostrado que o formato de cada campo alterável é descrito na forma de um texto junto ao próprio campo. Isso pode ser visto no campo projeto, o qual contém textos com até 30 caracteres. Quaisquer outras restrições associadas podem ser especificadas nesse texto independente de sua complexidade. Podemos especificar máscaras, condições para que ele seja ocultado ou exibido, valores inicialmente exibidos, etc. Tudo o que for necessário pode ser especificado no próprio campo. Isso torna muito simples o entendimento do seu formato e do seu funcionamento.

Na mesma figura existe ainda a especificação de diversos comandos descritos a partir dos delimitadores <>. Um comando é uma entidade que dispara alguma ação. Note que não definimos se o comando será um botão hiperlink, comando de voz ou qualquer outro tipo. O importante é deixar claro que existe um comando na tela e que ao ser acionado é responsável por alguma ação. Essa é a vantagem de se utilizar protótipos independentes de tecnologia: não temos aderência a qualquer formato, facilitando o desenho definitivo da interface sem limitar o conjunto de alternativas existentes.

Outro ponto interessante na figura é a existência de uma tabela com uma lista de valores (Projetos recuperados). Uma tabela é algo comum na maioria dos sistemas. Frequentemente necessita-se especificar tabelas em nossos protótipos, pois elas são muito úteis para agrupar informações correlacionadas.

Também é importante ressaltar algo que geralmente gera muita confusão nos iniciantes em criação de protótipos para o levantamento de requisitos: essas telas nunca vão executar! Assim, fique tranquilo quanto à usabilidade, pois elas não serão usáveis! As telas definitivas feitas com base nesses protótipos é que vão executar. Mas por enquanto, temos apenas um esboço daquilo que será feito em uma fase posterior do desenvolvimento.

Quando temos um protótipo e o detalhamento do caso de uso, o entendimento de parte de um produto se torna bastante simples. Veja por exemplo, a descrição do fluxo principal associado ao protótipo exibido na Figura 4.

O ReqG exibe a Tela de Geração da Especificação.

O Membro informa os dados para pesquisa por Projetos.

O Membro aciona o comando Pesquisar.

O ReqG recupera e exibe na lista denominada Projetos Recuperados aqueles Projetos que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo nome do Projeto em ordem crescente.

O Membro aciona o comando Gerar Especificação.

O ReqG gera um documento no formato especificado no arquivo Modelo-ERSw.doc.

Alguns protótipos possuem campos com formatos mais específicos, conforme apresentado na Figura 5. O campo projeto possui um asterisco que indica que ele é obrigatório. Além disso, seu conteúdo contém a especificação que conterá uma lista de projetos que atendem a uma determinada restrição, significando que essa lista será carregada dinamicamente durante a execução do produto e que seus valores serão trazidos de uma entidade do produto.

DADOS DA REVISÃO	
Projeto*	[Lista de Projetos que possuem o usuário corrente como membro]
Identificador*	Revisão preliminar de requisitos (texto até 50 caracteres)
Descrição*	Revisão preliminar de requisitos (texto até 50 caracteres)
Data*	12/12/2010 (data no formato dd/mm/aaaa)
Participantes dos desenvolve-dores*	[Lista de Membros do Projeto que não sejam clientes]
	...
	[Lista de Membros do Projeto que não sejam clientes]
Participantes dos clientes	[Lista de Membros do Projeto que sejam clientes]
	...
	[Lista de Membros do Projeto que sejam clientes]
Situação*	[Aberta; Fechada]

Figura 5: Protótipo com campos mais específicos.

Ainda no protótipo exibido na Figura 5, podemos notar que o campo Participante dos desenvolvedores também possui uma especificação de uma lista, mas poderá ter mais de um valor, o qual será mapeado por diversas formas em uma interface definitiva: vários campos do tipo check box, um campo list, uma tabela, etc.

Tipos de Ingresso	[Lista de Tipos de Ingresso pré-cadastrados no sistema]
	...
	[Lista de Tipos de Ingresso pré-cadastrados no sistema]
Effects	[Lista de Effects pré-cadastrados no sistema]
	...
	[Lista de Effects pré-cadastrados no sistema]
Classificação	[Matrícula; Nome]

Figura 6: Exemplo de especificação de protótipo usando editores de texto.

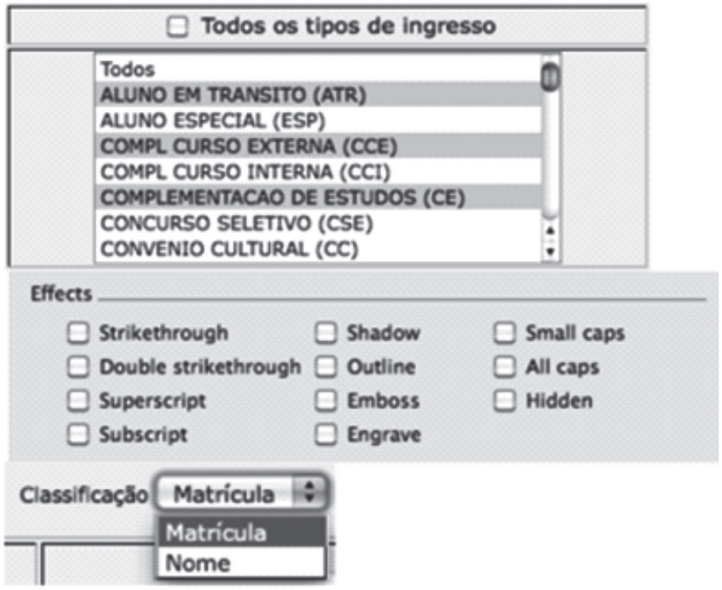


Figura 7: Exemplo de interface definitiva similar ao protótipo da figura anterior.

Uma modelagem do protótipo, feita utilizando nossas convenções, pode ser mapeada para diferentes formatos em tecnologias específicas

Na Figura 6 temos a especificação de três campos utilizando nossas convenções. Os dois primeiros campos são multivalorados enquanto que o terceiro pode ser apenas um de dois possíveis valores. Na Figura 7 temos um exemplo de como esse protótipo pode ser construído em um ambiente definitivo (HTML). Note que embora os dois primeiros campos possuam a mesma representação em nosso formato independente de tecnologia, este poderá ter diferentes implementações em um ambiente definitivo.

Mais uma vez ressaltamos que existem diversos exemplos no Anexo I deste documento, que possui a especificação completa de um sistema real, o qual deve servir de base para a criação do nosso trabalho prático.

É chegada a hora de realizar a criação dos protótipos para os casos de uso identificados. Lembre-se do formato independente de tecnologia apresentado aqui e fique atento às convenções definidas.

Revisão dos Requisitos

A revisão dos requisitos é a atividade realizada para garantir que o padrão prescrito pela organização foi realmente seguido e que os requisitos identificados atendam aos critérios de qualidade solicitados, permitindo o seu correto entendimento e, por conseguinte, a realização do projeto adequado bem como da implementação apropriada.

Para a revisão é necessário inicialmente estabelecer os critérios a serem utilizados. Na Seção 2 foram expostos diversos critérios de qualidade para requisitos. Um projeto pode determinar que critérios devam ser utilizados para a realização das revisões, para então analisar cada requisito à luz dos critérios selecionados.

Dessa forma, podemos facilmente visualizar que uma revisão nada mais é que uma leitura e posterior análise dos requisitos, tendo em mente aspectos bem pontuais a serem avaliados. De modo geral, pessoas que não sejam os autores da especificação de requisitos seriam mais adequadas para a realização da revisão que os próprios autores. Isso acontece por que normalmente os autores podem ficar cegos quanto a certos problemas.

Um exemplo de revisão para parte dos requisitos contidos no Anexo I é apresentado na Tabela 6. Nela podemos notar a análise de alguns requisitos e casos de uso com base em critérios pré-definidos. A partir dessa análise serão registrados os eventuais conflitos e acompanhado os passos para sua resolução.

Nr.	Requisito	Caso de Uso	Ambigüidade	Clareza	Comple-tude	Conflitos
1	RF1	UC2	Aprovado	Aprovado	Não aprovado	Não foi especificada a ordem em que os resultados devem ser exibidos.
2	RF2	UC3	Aprovado	Aprovado	Aprovado	-
3	RF5	UC8	Não aprovado	Aprovado	Aprovado	O caso de uso parece que poderia ser agrupado com o caso de uso Gestão de Membros, não havendo necessidade de criação de um caso de uso adicional.

Tabela 6: Fragmento da revisão de uma ER.

No próximo capítulo apresentaremos de forma detalhada uma forma de se conduzir reuniões para levantamento de requisitos e para realização de revisões.

No Anexo IV existe a revisão por completo contendo os critérios

utilizados e sua explicação, assim como o resultado da avaliação executada. Você deve criar algo similar para o Software de Controle de Empréstimo Pessoais. Lembre-se de definir os critérios, detalhando como eles serão utilizados, além de registrar os problemas e as formas de resolução dos mesmos.

Com a apresentação do formato para revisão de requisitos, encerramos o detalhamento do Fluxo de Requisitos. Após uma execução completa desse fluxo, teremos boas informações sobre como desenvolver um produto que atenda às necessidades do cliente, porém ainda serão necessárias várias transformações até que o produto seja construído.

Algo que devemos ressaltar bastante para os iniciantes no levantamento de requisitos é uma frase contida segundo o autor Pádua Filho (2003, p. ?) “para desenvolver uma especificação de requisitos que custa tempo e dinheiro; não fazê-la geralmente custa mais tempo e dinheiro ainda”!

Exercícios Propostos

1. Qual a definição de requisito?

2. Qual o objetivo de uma Especificação de Requisitos?

3. Por que a geração de uma Especificação de Requisitos para um produto novo é mais complexa que para produtos existentes?

4. O que é o fluxo de requisitos?

5. Quem participa do levantamento de requisitos?

6. O que é a Engenharia de Requisitos?

7. Cite e explique 3 características de qualidade de requisitos.

8. Quais são as atividades do fluxo de requisitos? Descreva-as brevemente/

9. O que é o escopo de um projeto?

10. Por que é importante definir os limites do produto?

11. Como deve ser a abordagem em uma instituição para se levantar requisitos junto aos clientes?

12. Qual a diferença entre requisitos funcionais e não-funcionais?

13. Cite 3 exemplos de requisitos funcionais para um Sistema Acadêmico.

14.Cite 3 exemplos de requisitos não-funcionais para um Sistema Acadêmico.

15. O que é um caso de uso, segundo requisitos do software?

16. O que são os atores, segundo requisitos do software?

17. Como identificar atores?

18. O que devemos fazer para identificarmos casos de uso?

19. Identifique casos de uso e atores para os requisitos descritos na questão 13.

20. Crie um diagrama de casos de uso para os itens identificados na questão anterior.

21. O que é um fluxo principal?

22. Qual a diferença entre fluxo principal e fluxo alternativo?

23. Como podemos especificar regras de negócio nos casos de uso?

24. Qual a importância de um protótipo de interface?

25. Quais são as tecnologias possíveis utilizadas para construção de protótipos?

26. Qual a convenção de cores utilizadas para construção de protótipos? O que elas significam?

27. Crie um protótipo de tela utilizando as convenções prescritas no capítulo para modelar alguma tela real de algum sistema que você utilize.

28. Para que serve a revisão dos requisitos?

29. Qual a importância dos critérios para uma revisão de requisitos?

1.3 Técnicas de Apoio ao Levantamento de Requisitos

Durante o levantamento de requisitos são necessárias diversas reuniões que tem como objetivo básico entender bem as necessidades dos clientes, além de avaliar se os dados coletados estão adequados e consistentes com as necessidades.

Por conta disso são necessárias técnicas que facilitem a execução dessas reuniões. Neste capítulo apresentaremos justamente técnicas apropriadas para os dois casos citados anteriormente. Boa parte do material deste capítulo foi baseada do livro Engenharia de Software de autoria do Paula Filho (2003).

Oficinas de requisitos

Uma oficina de requisitos é uma forma de identificar o que se deseja de forma conjunta com os usuários.

As oficinas de requisitos são reuniões estruturadas para definição conjunta dos requisitos, envolvendo desenvolvedores, usuários e demais especialistas. O tipo de oficina que será aqui discutido é baseado nas técnicas de JAD, embora existam variantes na literatura.

As oficinas de requisitos usam uma técnica estruturada de condução de reuniões de desenvolvimento, aplicável a diversas atividades do ciclo de vida do software, sendo especialmente útil no levantamento de requisitos. Nessas reuniões, denominadas oficinas de requisitos, o levantamento e detalhamento dos requisitos são feitos em conjunto com a participação de desenvolvedores e usuários chaves, assim como gerentes, todos unidos para termos uma melhor qualidade no resultado do trabalho.

O levantamento de requisitos utilizando JAD normalmente apresenta resultados muito interessantes.

As oficinas de requisitos tendem a apresentar melhores resultados que os levantamentos baseados em reuniões individuais com alguns usuários. Isso acontece por uma série de razões, dentre elas:

1. Elas facilitam o comprometimento dos usuários com poder de decisão e os requisitos;

2. Elas reduzem o prazo de levantamento de requisitos, visto que a reunião com todos os envolvidos tende a ser mais direta que conversas individuais setorizadas;

3. Eliminam requisitos de valor questionável, uma vez que todos são discutidos nas reuniões;

4. Reduzem diferenças de interpretação dos requisitos entre usuários e desenvolvedores, uma vez que as explicações acontecem ao vivo e com diferentes pontos de vista;

5. Produzem um primeiro esboço das interfaces de usuário, a partir do

direcionamento dado pelos próprios usuários no momento das reuniões; **6.** Trazem à tona, o mais cedo possível, problemas políticos que possam interferir no projeto. Essas oficinas possuem três partes bem distinta: a Personalização, onde são feitas adaptações do método para a aplicação; as Sessões propriamente ditas, na qual as reuniões são realizadas com a participação de todos os envolvidos; e o Fechamento com a produção dos resultados obtidos.

Personalização

A personalização da oficina de requisitos é uma parte simples do processo. As equipes são selecionadas e organizadas para que haja a participação dos usuários chave para as reuniões sejam produtivas, além de uma orientação sobre como os trabalhos serão conduzidos.

Além do que foi exposto, são feitas as particularizações necessárias ao projeto, bem como a preparação das instalações a serem utilizadas para as reuniões.

Um ponto importante a se discutir nesse momento é: quais são os usuários adequados a participarem de uma reunião de levantamento de requisitos? Não existe uma resposta genérica e adequada para tal questão, mas existem diretrizes importantes a serem seguidas. Uma dessas diretivas é: nas reuniões iniciais, onde são determinados o escopo e os requisitos, é mais importante a participação de usuários com um bom conhecimento do todo, embora não tenha conhecimentos aprofundados sobre as partes. Tipicamente, esses usuários são as pessoas da organização com maior nível hierárquico. Normalmente chefes, coordenadores, gerentes, diretores possuem um bom conhecimento do todo e pouco das partes. Eles são pessoas importantes para as reuniões iniciais de levantamento de requisitos.

A seleção dos participantes das reuniões é algo que pode facilitar muito o levantamento de requisitos.

Uma vez levantados os requisitos iniciais, torna-se importante ter contato com as pessoas que possuem mais conhecimentos detalhados nas partes que compõem o produto. É chegada a hora de termos o detalhamento dos requisitos, onde teremos que propor protótipos para as interfaces, além de detalhar as regras de negócios do produto. Para isso, os funcionários com mais conhecimento dos detalhes e provavelmente menos conhecimento do todo, são mais importantes.

Sessões

As sessões correspondem à realização efetiva das reuniões. Todos os integrantes da equipe da oficina devem participar das sessões em tempo integral para que não se perca tempo em recapitulações para os ausentes em sessões anteriores. Idealmente, as sessões devem ser conduzidas em local afastado das organizações dos participantes, visto que é muito comum que haja diversas interrupções quando são realizadas nas instalações do cliente. Assim, uma boa dica é levar as sessões para longe do ambiente dos clientes.

Sem o controle adequado nas sessões, o risco de insucesso aumenta bastante!

Interrupções são altamente prejudiciais, sendo importante avisar todos os participantes sobre essas regras. Telefonemas não devem ser atendidos, ou seja, preferência desligar.

Deve-se planejar a disponibilidade e necessidade dos materiais, tais como:

1. Computadores: normalmente são necessários pelo menos 2 computadores (ou notebooks), sendo um para projeção da ER que está sendo criada e outro para redação da ata da reunião;

2. Projetores: pelo menos um projetor é necessário para que seja possível exibir o resultado do trabalho e permitir uma discussão de todos;

3. Copiadoras: eventualmente pode ser necessário compartilhar material com os demais participantes;

4. Blocos de escrever, flip-charts, quadros brancos, lápis e canetas: em muitos momentos será necessário escrever, sendo importantes termos materiais para tal fim;

5. Lanches: as sessões podem durar um turno inteiro, sendo importante um momento para repor as energias e evitar a desatenção nas discussões.

Existem papéis importantes nas sessões. Cada um possui uma atribuição específica e pode ser decisivo para o sucesso das reuniões. Dentre os papéis destacamos:

1. O líder da sessão, responsável por manter o bom clima e o foco das discussões, que deve dominar técnicas de condução de reunião e ser treinado na condução destas oficinas;

2. Os patrocinadores, representantes do cliente, responsáveis pela resolução de conflitos entre grupos de usuários, com poder de decisão para tal;

3. Os representantes dos usuários têm autoridade para tomar decisões em nome da respectiva comunidade;

4. Desenvolvedores: fornecem informação sobre a viabilidade das idéias levantadas;

5. O relator, participante da equipe do projeto, encarregado de redigir as atas de reunião.

Durante a discussão, as ideias devem ser registradas e simultaneamente exibidas a todos os participantes, seja através de quadros brancos, seja através do projetor. O líder deve estimular a participação de todos, mantendo o foco, encaminhando a resolução de conflitos e identificando questões que não possam ser resolvidas durante a sessão. Deve ficar claro para todo o comprometimento que deverá existir em relação às conclusões da oficina.

Fechamento

No Fechamento são produzidos os artefatos resultantes da reunião. Isso normalmente inclui partes da ER e atas. Deve-se ter um cuidado especial para as pendências registradas na reunião e que devem ser resolvidas em uma data acertada por todos.

Normalmente, o fechamento é feito apresentando o material produzido incluindo a Ata para posterior envio e aprovação por parte dos envolvidos.

No Anexo II exibimos um exemplo de uma Agenda para uma reunião de levantamento de requisitos e no Anexo III um exemplo de uma ata para uma reunião. Esse modelo pode ser utilizado para servir de base para criação desses artefatos em um projeto.

De modo geral, uma ata deve conter um espaço para detalhar os participantes (clientes e desenvolvedores), assuntos discutidos e pendências. As atas são importantes, pois além de registrarem os assuntos discutidos e envolvidos na discussão, detalha o tempo em que tais discussões apareceram, mostrando profissionalismo nas relações, algo que não é comum em todas as empresas que trabalham com o desenvolvimento de software.

Uma revisão é um mecanismo de garantia da qualidade normalmente aplicada para verificar artefatos não executáveis.

Revisões

O Teste de Software é algo muito importante no desenvolvimento. É usado para avaliar se o produto construído está de acordo com os requisitos identificados. Mas se construímos uma ER, como fazer para testá-la? É nesse momento que se torna necessária a realização de atividades de garantia da qualidade que tenha como objetivo avaliar o trabalho realizado. Para os casos em que o produto do trabalho não é executável, as revisões são adequadas.

As revisões de software são técnicas eficazes de garantia da qualidade. Segundo um grande pesquisador na área da Engenharia de Software, Watts Humphrey, as Revisões são mais eficazes que os testes porque se encontra defeitos em um produto nessa fase.

As revisões aplicadas no desenvolvimento de software geralmente são de responsabilidade de um grupo de garantia da qualidade. Como boa parte das organizações é pequena e não consegue ter um grupo dedicado somente a isso, é necessário ter no processo uma previsão dessa atividade. O fluxo de requisitos apresentado neste texto contém a previsão de uma revisão no final de sua execução, devendo ser liderada pelo Gerente do Projeto, caso não haja um responsável direto.

Existem diversos tipos de revisão. A revisão técnica tem como objetivo avaliar artefatos específicos para verificar se eles estão conformes com os respectivos padrões e se modificações foram efetuadas de maneira correta.

A inspeção é mais formal que a revisão técnica. Tem como objetivo principal identificar e remover defeitos. Para isso, seu resultado deve gerar uma lista de defeitos com classificação, exigindo dos autores do artefato revisado providência para a resolução dos problemas relatados.

Na revisão de apresentação o autor demonstra o material em ordem lógica, sem limite de tempo a um grupo que verifica o material à medida que ele vai sendo apresentado. Este tipo de revisão não exige preparação prévia e pode ser feito com maior número de participantes por terem este papel mais passivo. Podem ser usadas nos marcos de projeto em que são necessárias apresentações ao cliente.

A revisão gerencial é conduzida pelo gerente de um projeto com o objetivo principal de avaliar os problemas técnicos e gerenciais do projeto, assim como o seu progresso em relação aos planos.

Além das revisões formais, existem diversos tipos de revisão informal que podem e devem ser usadas. Alguns métodos ágeis se utilizam muito desse tipo de revisão, uma vez que essa prática de revisar é uma boa prática no desenvolvimento de software. Dentre os tipos de revisões informais, destacam-se:

1. A programação em pares adotada no XP e outros processos ágeis. Essa é uma revisão contínua, onde uma dupla trabalha programando em uma mesma máquina. Um dos integrantes da dupla é responsável por “pilotar” o equipamento e o outro por uma revisão permanente do trabalho sendo executado.

2. A revisão individual é realizada pelos autores, seguindo formalmente

os roteiros pertinentes, eventualmente com a ajuda de pares.
3. A revisão preliminar é realizada por um ou mais pares dos autores, para eliminar defeitos mais simples do material como ortografia, numeração, estilos e consistência.

Participantes

Uma revisão deve ser feita com a participação de diversas pessoas. No entanto, é aconselhável que esse grupo tenha de 5 a 8 membros, sendo: 1 líder; 1 relator; 1 ou 2 autores; 2 a 4 revisores pares dos autores; 0 a 2 representantes dos usuários, dependendo do material em revisão.

O líder da revisão deve possuir algumas características, conforme detalhamento a seguir:
1. Ele deve compreender o propósito das revisões em geral e entender seu funcionamento;
2. Ter conhecimentos técnicos de alto nível sobre o material a ser revisado;
3. Ter participado de alguma outra revisão como revisor e também, de preferência, como autor;
4. Não ter com qualquer um dos revisores alguma dificuldade pessoal que possa interferir em sua habilidade de liderar a revisão.

Um número grande de participantes inviabiliza a produção de bons resultados em uma revisão.

Assim como o líder, o relator da revisão também deve possuir algumas características:
1. Compreender o propósito das revisões em geral e entender seu funcionamento;
2. Compreender o jargão e os formatos utilizados neste material; •ser capaz de se comunicar com as pessoas presentes na revisão;
3. Ter participado de alguma outra revisão como revisor ou como autor.

Os demais revisores devem levar em conta os seguintes aspectos de comportamento:
1. Estar preparado, lendo cuidadosamente o material antes da reunião;
2. Ter conhecimento técnico sobre parte do material da revisão;
3. Ser cooperativo;
4. Ser franco em relação ao material da revisão, mas polido em relação aos autores;
5. Compreender perfeitamente os pontos discutidos;
6. Usar um comentário positivo e outro negativo;
7. Apontar defeitos, mas não discutir sua resolução, pois esta não

faz parte da revisão;
8. Evitar discussões sobre detalhes não-pertinentes à qualidade do material em revisão;
9. Limitar-se aos assuntos técnicos;
10. Não avaliar os produtores, apenas o resultado do projeto.

Normalmente, os revisores são desenvolvedores, ou seja, pares dos autores. Em alguns casos, pode ser desejável a participação de usuários como revisores, por exemplo, em revisões das especificações de requisitos, dos desenhos das interfaces de usuário e da documentação de usuário. Nesse caso, os usuários devem estar cientes de que estarão analisando a qualidade do material sob revisão.

Condução

É muito importante deixar bem claro nas reuniões que o que está sendo revisado é um trabalho e não seus autores. Isso deve ser frisado por que é comum que haja certas disputas em revisões que trazem à tona certos problemas de relacionamento.

É preciso deixar muito claro que em uma revisão estamos analisando o trabalho produzido e não seus produtores.

Outro ponto chave é manter sempre em mente que o objetivo da revisão não é detalhar soluções, mas apenas apontar problemas e, eventualmente, dar sugestões sobre possíveis melhorias ao material em revisão. A descoberta por soluções pode levar um tempo muito grande, inviabilizando a própria revisão.

Normalmente, a reunião de revisão não deve ultrapassar duas horas. Deve-se garantir que não sejam feitas interrupções externas à reunião e que os membros da revisão não sejam solicitados por telefonemas ou trabalhos externos. O líder da revisão deve certificar-se de que todos os telefones celulares estejam desligados.

Nas revisões técnicas, o material é normalmente apresentado na ordem em que está documentado por um autor ou pelo líder da revisão. Nas inspeções, é comum utilizar a ordem por item analisado. Desta maneira, em cada etapa a equipe de inspeção terá a atenção focalizada em um aspecto específico.

A reunião deve ser iniciada pontualmente; nenhum participante poderá mais entrar após o seu início. Se a reunião tiver que ser interrompida ou se algum dos participantes estiver ausente à revisão deverá ser cancelada, e nova data para a revisão deverá ser marcada pelo líder.

Uma revisão técnica é realizada de acordo com o seguinte procedimento:

1. O gerente do projeto envia o material para o Grupo de Garantia da Qualidade (GQ) ou para o grupo responsável pela realização das revisões, caso não haja o grupo de garantia da qualidade. O material consiste dos resultados de uma ou mais atividades de fase do desenvolvimento, como uma ER, pré-revisados, se necessário, para eliminar problemas menores de estilo, ortografia, etc.
2. O GQ convoca os membros da equipe de revisão, escolhe o líder e o relator desta equipe e distribui os resultados para o grupo de revisores.
3. O grupo de revisores estuda o material e faz a preparação da revisão, possivelmente registrando suas observações.
4. O líder comanda a realização da reunião de revisão e envia os relatórios para o GQ contendo o resultado do material analisado.
5. O GQ faz o pós-processamento da revisão.

Em alguns estilos de revisão, a detecção de defeitos é realizada principalmente durante a reunião, tendo a preparação o objetivo de estudo e análise preliminar do material. Em outros estilos, solicita-se que os revisores procurem localizar o máximo de defeitos durante a preparação, tendo a reunião o objetivo principal de eliminar duplicações e padronizar a classificação dos defeitos e o fraseado das observações.

O Relatório de Revisão deve conter uma lista com os defeitos encontrados. É importante o uso de um projetor durante a geração do relatório para permitir que todos vejam o que está sendo escrito. Ao final da reunião, deve-se produzir uma versão impressa com listagem das anotações, o qual será conferido por todos os participantes e que receberá as assinaturas destes.

Os autores devem agir em relação a essa lista de defeitos, anotando as providências tomadas. Uma recomendação é que para cada defeito, o autor deve inserir a providência correspondente, indicando se o defeito foi corrigido ou expondo as razões pelas quais discorda da necessidade de correção. O autor deve também indicar o tempo gasto para remoção do defeito, que será posteriormente usado para alimentar uma base de dados históricos.

Após a revisão, o GQ deve encaminhar o resultado para o Gerente do Projeto, para que se façam as correções necessárias no material revisado. Isso depende muito do resultado da revisão que pode ser:
1. Aceitação: indicando que o material foi aceito, embora ainda devam existir outros procedimentos de controle.
2. Revisões menores: o gerente do projeto solicita à equipe do projeto que

providencie os acertos necessários. O GQ deve verificar se as correções devidas foram feitas.

3. Revisões maiores: o gerente do projeto solicita à equipe do projeto que corrija o material e pede ao GQ uma nova revisão ao final da correção.

4. Reconstrução: o gerente do projeto investiga as causas dos problemas detectados, agendando a reconstrução do material, indicando as prováveis falhas para serem observadas.

É importante ressaltar que as sugestões apontadas no relatório de revisão não precisam ser necessariamente implementadas. Cabem ao gerente do projeto determinar, entre as alterações propostas, quais devem ser realmente executadas, ponderando os aspectos técnicos, gerenciais e políticos ouvindo o GQ.

Os Relatórios de Revisões devem ser arquivados permanentemente, sendo importante que cada projeto mantenha seu repositório.

Exercícios Propostos

1. O que são oficinas de requisitos?
2. Por que as oficinas de requisitos tendem a apresentar melhores resultados que as reuniões individuais?
3. Como são divididas as oficinas?
4. O que é feito na personalização de uma oficina?
5. Por que as sessões de levantamento deveriam ser feitas longe do ambiente dos participantes dos clientes?
6. Que materiais são necessários durante uma sessão de levantamento de requisitos?
7. Quais são os papéis associadas a uma sessão de levantamento de requisitos?
8. O que é feito no fechamento de uma sessão?
9. O que deve existir em uma ata de uma reunião de levantamento de requisitos?
10. Qual a relação existente entre o teste e a revisão?
11. Quem é o responsável pela condução de uma revisão?
12. Quais são os tipos de revisão existentes?
13. Qual a forma de revisão existente no processo XP?

14. Quais características devem estar associadas ao líder de uma revisão?
15. Quais características devem estar associadas ao relator de uma revisão?
16. Quais aspectos de comportamento são importantes para os demais revisores?
17. Como deve ser conduzida uma reunião de revisão?
18. Quais são os possíveis resultados de uma revisão?

REFERÊNCIAS DA WEB

Página da Universidade Aberta do Piauí - UAPI
http://www.ufpi.br/uapi

Página da Universidade Aberta do Brasil- UAB
http://www.uab.gov.br

Instituto de Engenharia de Software do MIT
http://www.sei.cmu.edu/

Sítio de apoio ao livro de Roger Pressman
http://www.rspa.com/

Sítio de apoio ao livro de Ian Sommerville
http://www.comp.lancs.ac.uk/computing/resources/IanS/

Sítio de apoio ao livro de Wilson de Pádua
http://homepages.dcc.ufmg.br/~wilson/

Sítio da Ferramenta Astah
http://astah.change-vision.com

UNIDADE 2

Análise de Software

Resumo

Nesta unidade apresentamos o Fluxo de Análise. Tem como objetivo modelar conceitos importantes, identificados durante o levantamento de requisitos. Essa modelagem dá origem a diversas classes e diagramas que apresentam a especificação de requisitos sob outra forma, mais próxima dos desenvolvedores e um pouco mais distante dos usuários finais. No Capitulo 3 apresentamos a Linguagem UML, um padrão de facto para a modelagem de sistemas e fundamental para execução do Fluxo de Análise. Todos os seus elementos serão descritos, juntamente com um breve detalhamento sobre como utilizar a ferramenta Astah para modelagem. No Capítulo 4 apresentamos o Fluxo de Análise que são detalhadas às atividades relacionadas à disciplina, apresentando em detalhes a Análise para nosso sistema exemplo, os quais detalharão os passos executados para cada atividade. No Capítulo 5 apresentamos a Análise de Ponto de Função, que é uma técnica para contagem de sistemas quando esses ainda não foram implementados e possuem apenas uma Especificação de Requisitos e um Modelo de Análise.

2 ANÁLISE DE SOFTWARE

A Linguagem UML

A modelagem é um dos conceitos importantes para o desenvolvimento de software. Um modelo é uma abstração de algo, em que nos concentramos nos aspectos fundamentais do que se está querendo modelar, ignorando os aspectos que não são relevantes.

Modelos são fundamentais no desenvolvimento, pois criam uma camada de abstração do que estamos querendo modelar, focando apenas no que é importante. Isso permite que os desenvolvedores consigam representar algo complexo de forma bem mais simples, explorando apenas o que se deseja explorar em um determinado momento.

A Unified Modelling Language (UML) ou Linguagem de Modelagem Unificada em Português é uma linguagem utilizada para a criação de modelos. Ela possui uma interessante característica que a fez ser tão difundida: uma linguagem baseada em diagramas, o que a torna muito mais simples de usar e entender.

Neste trabalho utilizaremos uma ferramenta gratuita para a modelagem UML. O objetivo é possibilitar o uso de uma ferramenta, aplicando os conceitos de forma prática. A ferramenta selecionada foi a ASTAH que pode ser gratuitamente obtida em http://astah.change-vision.com. Utilizamos a versão community para demonstrar a aplicação prática dos conceitos apresentados durante o capítulo.

A Origem da UML

Até o surgimento da UML existiam diversos processos de software e diversas linguagens de modelagem associadas aos processos. Isso gerava muitos problemas, devido à ausência de um padrão para se modelar conceitos de um sistema. Por conta disso, houve uma tentativa de padronizar os processos e as linguagens de modelagem, gerando uma unificação dos conceitos associados. Isso deu origem ao Processo

Unificado (Unified Process – UP) e a UML. Três processos deram origem ao UP e à UML:

1. Booch – Esse método foi criado por Grady Booch e baseada no fato de um sistema ter diversas visões, cada uma descrita por modelos e diagramas. O método possuía uma simbologia complexa de ser desenhada à mão.

2. OMT – O Object Modelling Technique foi desenvolvido pelo General Electric onde o pesquisador James Rumbaugh trabalhava. Os modelos prescritos pelo método são compostos por modelos de objetos, funcional e casos de uso.

3. OOSE/Objectory – Os dois métodos foram desenvolvidos por Ivar Jacobson. Ambos são baseados na utilização de casos de uso, a partir da modelagem dos requisitos iniciais do sistema vistos por um ator externo. O método Objectory foi adaptado para permitir seu uso também para a modelagem de processos de negócio, fundamentais no funcionamento de qualquer organização.

Os métodos citados anteriormente tinham ideias em comum e diferente. Da mesma forma, eles possuíam notações próprias. A partir disso, os "três amigos" resolveram criar uma notação única, baseada no que havia de melhor em cada notação. Isso deu origem à UML. De forma similar, eles criaram um processo único, também baseado na unificação das melhores ideias existentes nos processos, criando assim o UP.

A UML, atualmente, é um padrão de facto. O mundo inteiro utiliza a UML. Ela é uma forma de comunicação padrão que é aceita e exigida por muitas organizações. Para entendermos a UML é necessário saber como ela é dividida. Nas próximas seções apresentaremos cada uma das partes da UML: Visões, Modelos de Elementos, Mecanismos Gerais e Diagramas.

Cada visão foca em um aspecto particular do produto sendo modelado.

Visões

Uma visão mostra um aspecto diferente do sistema que está sendo modelado, sendo normalmente composta por diversos diagramas que focam em aspectos particulares de um sistema. As principais visões da UML são:

1. Visão de Casos de Uso: descreve o comportamento do sistema, a partir da definição de como o sistema se comportará a partir da interação com os atores externos que o utilizarão. Isso está diretamente ligado ao diagrama de casos de uso.

2. Visão Lógica: descreve como o comportamento será implementado. Enquanto a visão de caso de uso está muito mais associada à visão externa do sistema, a visão lógica foca na visão interna, descrevendo os mecanismos que serão necessários para que o sistema funcione. Isso inclui a estrutura estática, composta por classes, objetos, relacionamentos, assim como a estrutura dinâmica composta por realizações. A parte estática da visão lógica é descrita por diagrama de classes e de objetos. A parte dinâmica é descrita por diagramas de estado, sequência, colaboração e atividade.

3. Visão de Componentes: descreve a divisão da implementação em módulos, demonstrando suas dependências. Essa visão está diretamente ligada ao diagrama de componentes.

4.Visão de concorrência: descreve o sistema sob a perspectiva de divisão em processos e processadores, indicando se haverá execuções em paralelo, detalhando a comunicação e a concorrência existente no sistema. Essa visão pode ser descrita pelos diagramas dinâmicos (estado, seqüência, colaboração e atividade), além dos diagramas de desdobramento.

5.Visão de Organização: descreve a organização física do sistema, indicando os computadores, periféricos e suas conexões entre si. Essa visão é ligada aos diagramas de desdobramento.

Modelo de Elementos

Os conceitos utilizados nos diagramas são modelos de elementos. Eles representam as definições comuns existentes na linguagem, tais como: classes, objetos, mensagens, atributos e relacionamentos. Esta seção detalha os principais modelos de elementos existentes na UML.

Classes

Uma classe é uma representação abstrata de um conceito. Nessa representação tentamos focar os dados relacionados à entidade que está sendo modelada, bem como nas possíveis operações que podem acontecer sobre esses dados.

Na UML as classes são representadas por um retângulo que pode ser dividido em até 3 compartimentos: o nome da classe, seus atributos e as operações possíveis.

A identificação de classes em um sistema é feito a partir de

uma análise da Especificação de Requisitos. A notação de classes da UML é independente da linguagem a ser utilizada. Assim, uma classe representada em UML pode ser gerada em Java, C++, Ruby, etc.

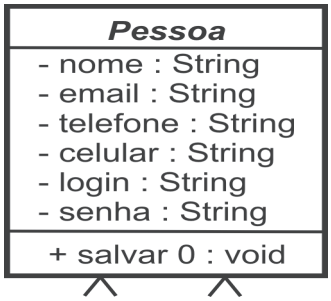


Figura 8: Exemplo de Classe.

A Figura 8 exibe um exemplo de classe, chamado Pessoa. Ela possui diversos atributos (nome, email, telefone, celular, login e senha). Na classe existe apenas uma operação (salvar). Para ser criada uma classe utilizando a ferramenta Astah, será necessário criar um diagrama de classes, uma vez que tal diagrama possibilita a criação de classes. Uma vez criado este diagrama, será exibida uma barra de ferramentas similar à mostrada na Figura 9. De forma geral, todas as ferramentas UML apresentam barras de ferramentas para facilitar a criação de elementos em diagramas. Para cada diagrama, uma barra de ferramentas com os elementos possíveis de integrar o diagrama são exibidos.



Figura 9: Barra de ferramentas para criação de diagramas de classe.

O primeiro item na barra de ferramenta é o item para seleção de objetos. O segundo item é o botão que permite criar classes. Basta clicar nele e clicar no diagrama para que uma classe seja criada. Após sua criação, é possível, via acionamento do botão direito do mouse sob a classe, criar atributos e operações, conforme exibido na Figura 10.

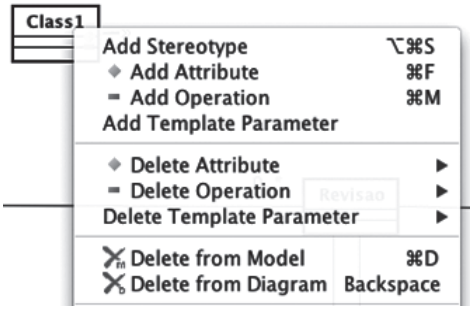


Figura 10: Criação de atributos e operações para classes.

Objetos

Os objetos representam instâncias de uma classe. De maneira simples, enquanto que as classes representam conceitos, objetos representam coisas reais associadas ao conceito modelado. Como exemplo, na Figura 9 tem a representação do conceito pessoa, indicando que é algo que contém nome, email, celular, etc. Na Figura 11 temos um exemplo de uma instância de Pessoa, que possui nome Pedro, email pedro@gmail.com, etc. Esse é apenas um exemplo de Pessoa, mas muitos outros podem ser criados.

Para criar um objeto, podemos utilizar a barra de ferramentas exibida na Figura 9. O botão para tal ação é o 15o elemento da esquerda para a direita. Ao se passar o ponteiro do mouse por cima dele será exibido o texto InstanceSpecification. Ao clicar no botão, para que ele fique selecionado, clica-se em qualquer local dentro do diagrama de classe, um objeto será apresentado. Depois disso, você terá que associar o objeto a uma classe, para que em seguida seja possível especificar os valores dos atributos.

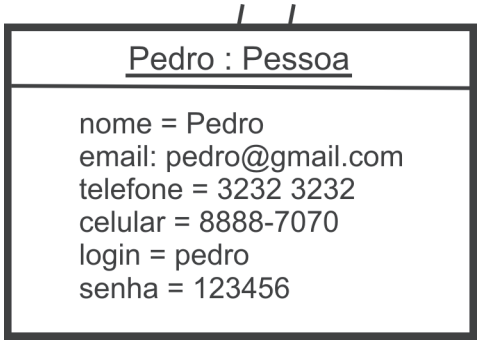


Figura 11: Exemplo de um objeto pessoa.

Estados

Os objetos normalmente possuem um estado que representa o resultado das operações executadas no objeto, sendo normalmente determinado pelos valores de seus atributos e ligações existentes com outros objetos. A Figura 12 apresenta dois estados relacionados a um objeto Livro. Um livro pode estar livre ou emprestado. Para mudar de livre para emprestado é necessário que o evento empréstimo seja realizado. Para mudar de emprestado para livre, é necessário que o evento devolução aconteça. Os estados de um objeto podem deixar claro seu funcionamento, sendo uma descrição bastante expressiva e simples de ser entendida.

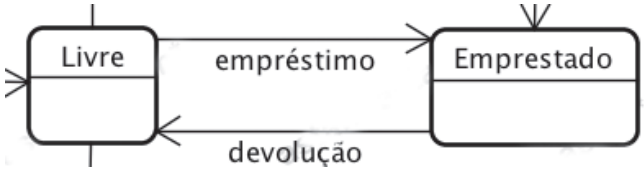


Figura 12: Exemplos de estados para o objeto Livro.

Pacotes

Um Pacote é um agrupamento de itens utilizado para fins de organização. De forma geral, esses agrupamentos são feitos para se manter juntos elementos que possuem algum tipo de relação entre si. Os pacotes podem ser criados dentro dos modelos UML, a partir do acionamento do botão direito do mouse em cima do local desejado. De forma simples, um pacote pode ser comparado a uma pasta dentro do modelo. A Figura 13 apresenta um exemplo de criação de um pacote em um modelo UML.

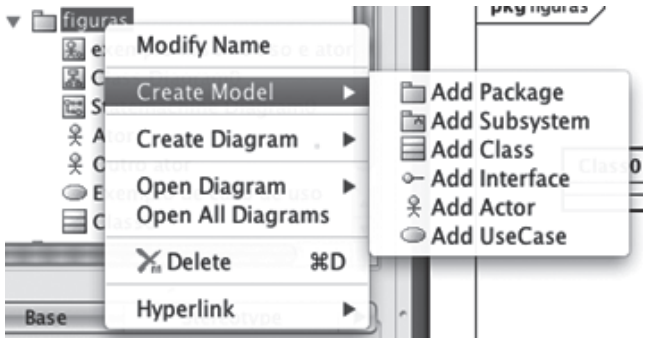


Figura 13: Criação de um pacote na ferramenta Astah

Componentes

Um componente é a representação de uma parte do sistema. Isso pode ser um código na linguagem fonte ou código executável. Imagine um sistema feito em Java. Cada arquivo .class pode ser considerado um componente, assim como qualquer arquivo .java. A Figura 14 exibe exemplos de componentes criados na Astah.

O uso desse tipo de elemento está diretamente associado ao projeto de um software. Por conta disso, ele será pouco explorado neste material, uma vez que estamos tratando de requisitos e análise.



Relacionamentos

Os relacionamentos permitem a ligação de classes e objetos entre si, estabelecendo uma relação entre eles. Essas ligações podem assumir os seguintes tipos: associação, generalização e dependência.

Associação

Uma associação cria uma conexão entre as classes ou entre objetos das classes. Dessa forma, uma associação entre classes indica que qualquer objeto de uma classe terá uma conexão com um objeto da outra classe.

A associação simples é a forma mais comum de uma associação. É representada por uma linha ligando duas classes, o qual pode possuir uma seta indicando direção. Isso significa que ela só pode ser usada para o lado em que ela aponta. Não havendo seta, significa que os dois lados são navegáveis. A Figura 15 exibe um exemplo contendo associações com e sem direção. Isso significa que podemos obter os Membros a partir de um Projeto, assim como a partir de Membro podemos obter os Projetos, uma vez que a associação não possui direção. Já a outra associação indica que podemos obter facilmente os Clientes de um Projeto, mas o inverso não é verdadeiro.

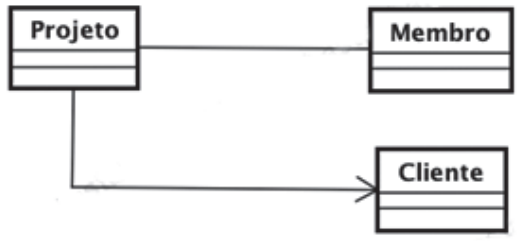


Figura 15: Exemplo de classes com associações com e sem direção.

A Figura 16 exibe as mesmas associações contidas na Figura 15, porém, com a especificação dos nomes dos papéis nos relacionamentos e multiplicidades das relações. Os nomes dos papéis dão origem ao nome dos atributos das classes quando fazemos geração de código. Dessa forma, ao gerarmos código para a classe Projeto, conterá dois atributos de coleção: os membros e os clientes. A maioria das ferramentas UML permite alguma geração de código. No entanto, isso não está diretamente ligado aos objetivos deste material, pois estamos ainda entendendo o problema e não projetando uma solução.

Para se inserir nome dos papéis, multiplicidade e nome nos

relacionamentos, você deve clicar na associação e visualizar a caixa de propriedades do item, que normalmente fica localizado no canto inferior esquerdo da tela da ferramenta Astah.

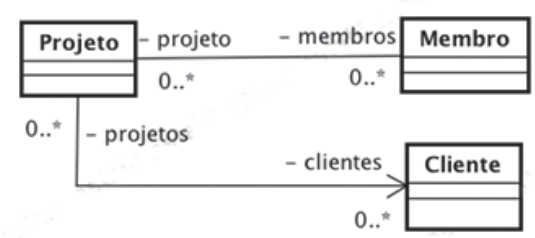


Figura 16: Associação com nomeação dos papéis e multiplicidades.

A multiplicidade das associações pode assumir diversos valores, tais como 0..1 (zero ou um), 2 (dois), 4..9 (de quatro a nove), etc. O uso de "*" indica que não há um limite estabelecido. Quando não é descrito multiplicidade utilizamos como padrão "1".

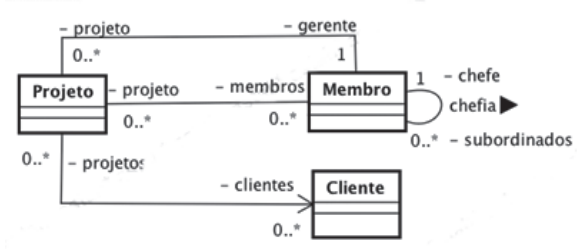


Figura 17: Exemplo com auto-associação e especificação do nome da associação.

A Figura 17 exibe um exemplo de associação recursiva entre Membro (também chamada de autoassociação). Conforme especificado, um membro, denominado chefe, chefia vários membros subordinados. Da mesma forma, um membro subordinado tem um membro que é seu chefe. Esse exemplo também mostra uma associação com nome (chefia).

Existem outros tipos de associação que não serão detalhados aqui como as associações com restrições, associações ternárias, associações qualificadas e classes de associação. No entanto, existe muito material na Internet disponível sobre o assunto, que não é tão comum de ser encontrão em modelos de análise, foco deste documento.

Ainda existem dois tipos particulares de associação que merecem comentários: agregação e composição. A Agregação é uma associação que indica a relação todo/parte. Isso significa que um dos itens relacionados é o todo na relação e o outro é uma parte. Normalmente os nomes das associações, caso fossem utilizados, deveriam ser "tem",

“contém“, “faz parte“. Uma agregação não possui uma representação específica na maioria das linguagens de programação nem necessitam de um comportamento especial da implementação associada. A Figura 18 exibe um exemplo que mostra que um Projeto contém Membros. Conforme comentado, a implementação da agregação entre projeto e membros deverá gerar a mesma implementação que uma associação normal. Observe que uma Agregação é representada por um losango não preenchido. É necessário termos muita atenção com a simbologia associada à UML: o uso de um símbolo errado pode passar uma ideia completamente diferente do que se desejava!

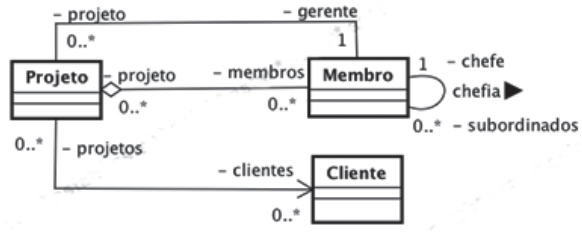


Figura 18: Exemplo de agregação.

A Figura 19 exibe uma associação do tipo Composição. Nesse tipo de associação existe um relacionamento forte entre os integrantes: a parte não vive sem o todo. Dessa forma, a implementação de uma Composição exige que esse comportamento seja implementado. Caso contrário, não estaremos cumprindo o que foi modelado. No exemplo temos modelado que Sócio contém Dependentes. Dessa forma, está implícito que a exclusão de um sócio implica na exclusão de todos os seus dependentes.



Figura 19: Exemplo de Composição.

Observe que a composição também é representada por um losango, porém, preenchido. Observe também que a diferença entre estar ou não preenchido pode causar uma grande mudança no entendimento e na forma de implementar o que foi modelado!

Generalizações

Uma Generalização indica que um elemento herdará todos os atributos e operações de outro elemento, podendo ainda incluir

comportamento adicional. Nos locais em que é esperado um elemento do tipo geral pode-se utilizar um elemento específico sem qualquer problema.

A Figura 20 exibe um exemplo de generalização. Nela podemos notar que um Projeto possui um Cliente que é a classe genérica. Um Cliente pode tanto ser uma Pessoa Física como uma Pessoa Jurídica.

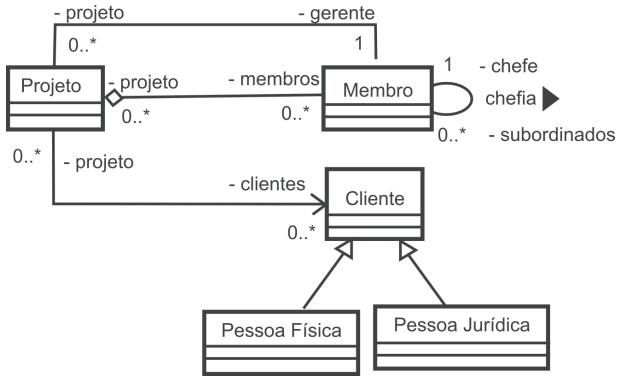


Figura 20: Exemplo de Generalização.

Dependências

A Dependência é uma relação mais fraca entre elementos, indicando que uma alteração em um elemento pode indicar uma mudança de comportamento em outro. A dependência é muito utilizada na especificação de arquitetura de um sistema, mostrando que um determinado sistema depende de tecnologia X, Y, Z. De forma geral, é uma relação mais fraca que a associação, mas indica uma conexão semântica entre os envolvido.

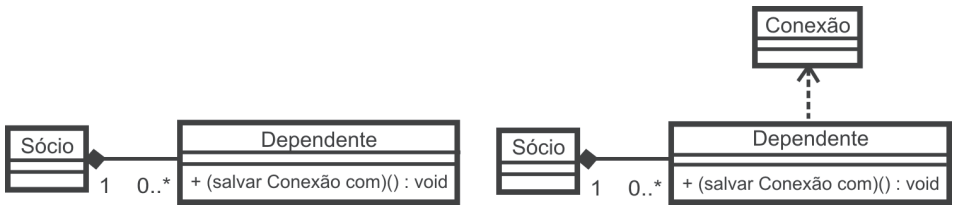


Figura 21: Exemplo de Dependência entre classes.

A Figura 21 exibe uma relação de dependência entre classes. Pode-se visualizar que a classe Dependente possui um método salvar que possui um parâmetro do tipo Conexão. Logo, uma alteração no tipo Conexão pode causar uma mudança no comportamento de Dependente. Isso causa a dependência entre as classes. A dependência não precisa ser exibida nos diagramas, a não ser que seja necessário expor essa dependência. Por conta disso, na figura existe a composição entre sócio

e dependente que é exibida duas vezes, sendo que parte do lado direito foi enfatizada a dependência entre a classe Dependente e conexão, uma vez que dentro da classe Dependente existe um método que usa um objeto do tipo Conexão.

Mecanismos Gerais

Na UML existem informações adicionais que são associados aos modelos. O grande exemplo disso são as notas. Embora a UML tenha muita semântica associada às representações, não é possível modelar tudo. Dessa forma, uma nota pode ajudar a detalhar certas informações. A Figura 22 exibe um diagrama contendo uma nota, especificando uma regra para a associação de sócios e dependentes, utilizando nossa linguagem natural.

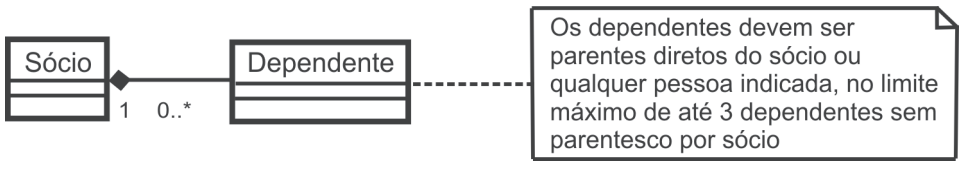


Figura 22: Exemplo de nota em um diagrama.

2.2 Diagramas

Na UML existem diversos gráficos que descrevem o que existe em uma visão. Esses gráficos são os diagramas, utilizados para criar as visões do sistema. Existem 9 diagramas na UML que serão apresentados nas próximas subseções. A criação de um diagrama na ferramenta Astah é feito a partir do menu Create Diagram, exibido quando se clica com o botão direito em algum pacote do modelo. A partir desse menu, é possível criar qualquer um dos diagramas existentes. A Figura 23 exibe o menu para criação de diagramas na ferramenta Astah.

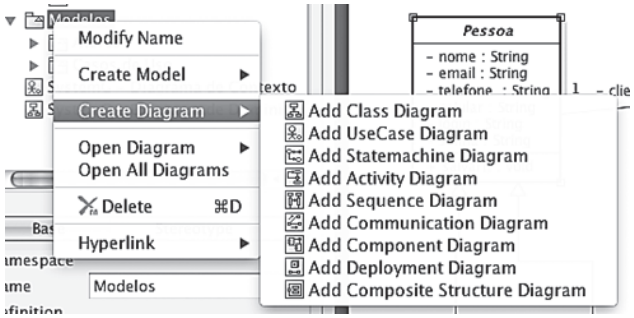


Figura 23: Menu para criação de diagramas na Astah.

Diagrama de Casos de uso

O diagrama de casos de uso exibe os requisitos funcionais de um sistema. Isso está diretamente ligado ao comportamento que o sistema deverá adotar. Nesse diagrama existem atores, que representam papéis no sistema e casos de uso, que representam as funcionalidades existentes. A ligação entre atores e caso de uso indica que há comunicação entre eles. Uma seta direcionada pode ser utilizada para definir o sentido da comunicação.

Diagramas de caso de uso são excelentes para se obter um entendimento rápido sobre um sistema!

A Figura 24 exibe um exemplo de diagrama de casos de uso. Temos o caso de uso Reserva de livro, associado ao ator Aluno. Nesse caso, pode-se concluir que um aluno pode reservar um livro, embora não tenhamos detalhes sobre quais são as regras específicas para a reserva. O ator Aluno, nesse caso, representa um papel relacionado a um grupo de usuários. O outro caso de uso da figura, Invalidação de reserva, está associado ao ator Tempo. Isso indica que o caso de uso é acionado automaticamente dentro de certos períodos de tempo pré-estabelecidos.



Figura 24: Exemplo de casos de uso.

Diagrama de Classes

O diagrama de classes está associado à visão estática de um sistema. Conforme visto neste capítulo, classes podem se associar a outras classes de diferentes formas (associação, composição, dependência, etc.). Um sistema real pode conter um número grande de classes, sendo necessária a criação de diversos diagramas, cada um focando em um aspecto específico. Uma mesma classe pode aparecer em diferentes diagramas. Nas figuras anteriores foram apresentados diversos diagramas de classe para se explicar os mais diferentes relacionamentos entre classes.

Diagramas de objetos podem facilitar o entedimento de classes complexas, a partir da demonstração de uma instância específica, onde podemos demonstrar alguns aspectos mais difíceis de serem entendidos.

Diagrama de Objetos

O diagrama de objetos descreve uma instância específica de um diagrama de classe. Sua notação é bem próxima do diagrama de classe, porém, é importante frisar que ele exibe as instâncias das classes com

sua configuração de execução momentânea.

Os diagramas de objetos não são tão utilizados nos modelos, mas servem para esclarecer aspectos importantes de diagramas complexos. A Figura 25 exibe um diagrama de objetos para o diagrama de classe apresentado na Figura 19.

Essa é apenas uma possível configuração para o diagrama de classe comentado. Nele podemos perceber que o Sócio Pedro possui dois dependentes: Ana Célia e Joana. Diversos outros diagramas de objetos são possíveis, desde que eles não violem as restrições existentes no diagrama de classe original.

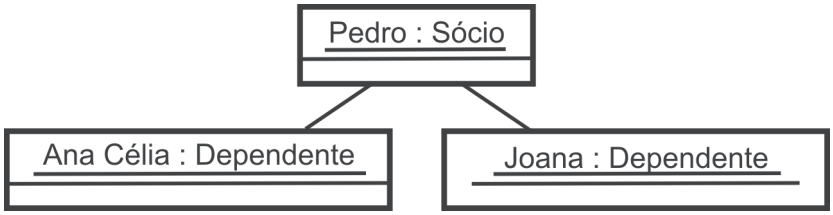


Figura 25: Diagrama de objetos.

Diagrama de Estados

O diagrama de estados descreve o comportamento dinâmico de uma instância de uma classe, normalmente representando seu ciclo de vida. Esse diagrama é importante para descrever classes que possuem objetos com ciclos complexos, uma vez que podemos especificar as ações e condições associadas às mudanças de estado.

Os diagramas de estados mostram a dinâmica das classes e são facilmente entendidos por usuários leigos, a partir de um mínimo de explicação.

A exibe um diagrama de estados para um objeto Livro. Ele inicia livre, pronto para ser reservado, emprestado ou tornado cativo e, dependendo da função utilizada, pode assumir outros estados. Um livro pode ser reservado, para depois ser emprestado e então devolvido para que seja feito novo empréstimo.

No diagrama, temos a indicação que o estado inicial de um objeto livro é livre e que furtado é um estado final (sinalizado pelo estado final ligado ao estado furtado). Um estado final indica que o ciclo de vida do objeto acabou não sendo possível mais nenhuma outra transição de estados.

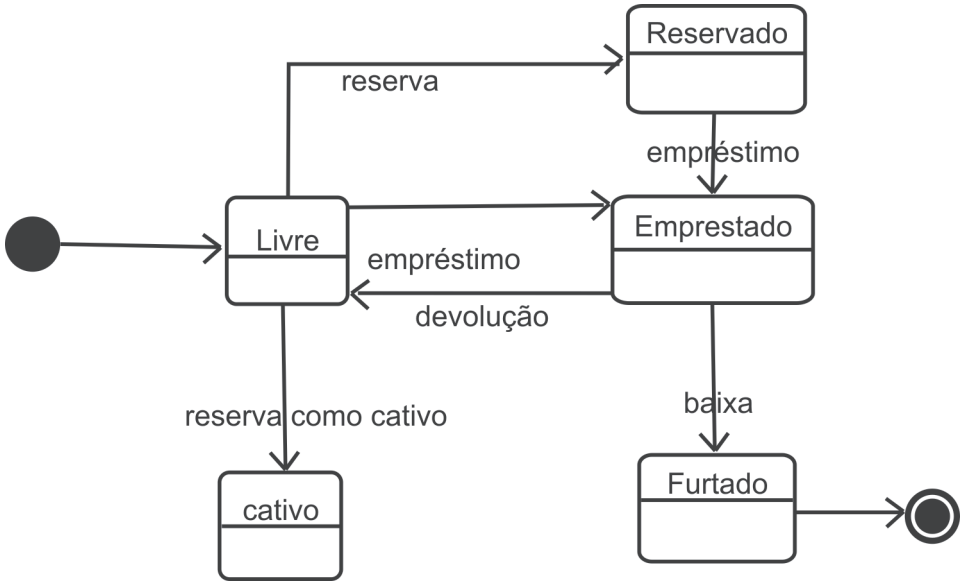


Figura 26: Diagrama de estados para o objeto Livro.

Diagrama de Sequência

O diagrama de sequência mostra a troca de mensagens entre objetos durante a implementação de certos comportamentos. O foco no diagrama de sequência é mostrar as interações entre objetos, ao invés de mostrar apenas o comportamento interno de um objeto. Ele ainda apresenta a ordem temporal das chamadas, destacando assim a sequência das ações invocadas, por isso seu nome.

A Figura 27 exibe um diagrama de sequência. Os objetos associados ao diagrama são exibidos na forma de um retângulo, com uma linha tracejada abaixo. Essa linha é chamada linha da vida do objeto, indicando quando o objeto está vivo durante a execução do roteiro que está sendo modelado. As setas entre objetos indicam o acionamento de algum método existente via troca de mensagens. Na figura podemos notar que o objeto Sócio invoca o método criar do objeto Dependente, acionando os métodos associar e salvar logo em seguida. O método salvar, do objeto Dependente, aciona o método validar, também interno a Dependente, que por sua vez aciona o método validarCPF do objeto Útil e salvarObjeto do objeto Persistência. Fica clara a ordenação temporal das chamadas.

Diagramas de sequência podem ser utilizados para se detalhar casos de uso. Essa é uma notação alternativa aos textos descrevendo os diversos fluxos que compõem os casos de uso. No entanto, essa alternativa é mais difícil de ser entendida pelos clientes, razão essa que dificulta sua adoção com essa finalidade.

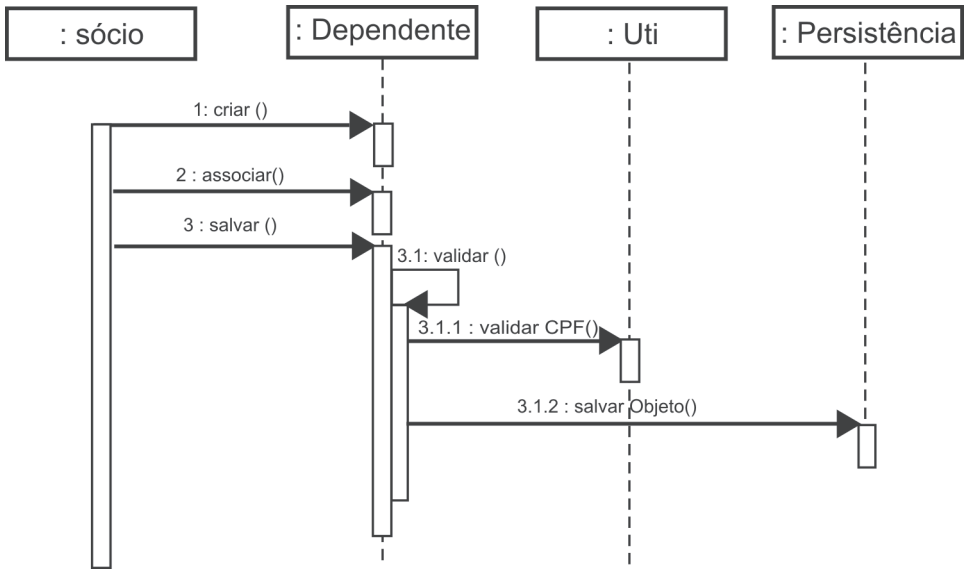


Figura 27: Exemplo de Diagrama de Sequência.

Outra aplicação para diagramas de sequência é exibir conceitos chaves de uma arquitetura, detalhando como determinados roteiros funcionam. Ele serve como uma explicação do que acontece em determinados casos, uma vez que detalha o comportamento dinâmico que parte de um software pode ter sob certas circunstâncias.

Diagrama de Colaboração

O diagrama de colaboração pode ser considerado como outra forma de exibição do diagrama de sequência. Um diagrama pode ser convertido para o outro, na maioria das ferramentas UML, a partir de um clique.

A diferença entre os diagramas está no foco: no diagrama de colaboração é enfatizada a troca de mensagens entre objetos e não sua ordem temporal. O diagrama de colaboração é pouco utilizado no desenvolvimento de software, sendo preferível o diagrama de sequência. Por conta disso, não daremos muita ênfase a ele neste trabalho.

Diagrama de Atividades

O diagrama de atividades é uma variante do diagrama de estados na UML. Porém, seu objetivo é completamente diferente, enquanto os diagramas de estados exibem o comportamento interno de um objeto, o diagrama de atividades exibe as ações associadas a um roteiro,

possibilitando detalhar os executores envolvidos e os produtos de trabalho gerados.

Diagramas de atividades podem ser usados para detalhar etapas de um processo, descrição de fluxos de um caso de uso e qualquer outro roteiro que exija a informação das ações associadas.

Neste trabalho, utilizamos diagramas de atividades para descrever os processos relacionados com requisitos e análise.

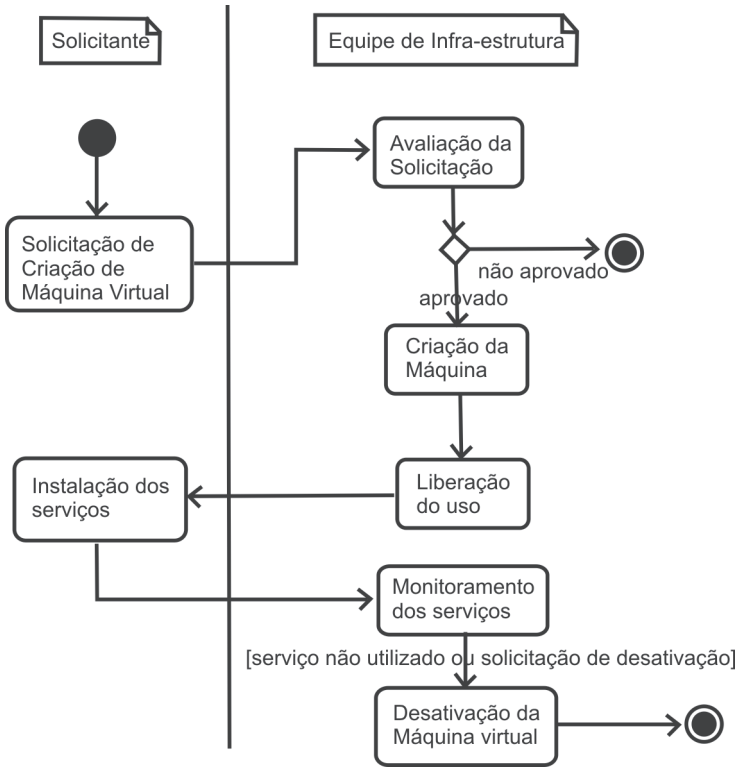


Figura 28: Descrição de um processo utilizando Diagrama de Atividades.

A Figura 28 exibe um diagrama de atividades criado com o intuito de descrever um processo para criação de máquinas virtuais em um Datacenter. Podemos notar que o diagrama exibe a ordem em que as ações devem acontecer, deixando claro quem são os responsáveis (exibidos na forma de divisão de colunas no diagrama). Existe no diagrama uma condição que indica para onde a execução deve ir, caso determinada situação aconteça.

Diagrama de Componentes

O diagrama de componentes exibe a relação entre as partes que compõem um sistema e a organização dos módulos. Eles podem

representar classes, pacotes e subsistemas. Esse tipo de diagrama é muito associado à exibição da arquitetura de um sistema, deixando claro como ele está estruturado, além de demonstrar sua organização e dependência.

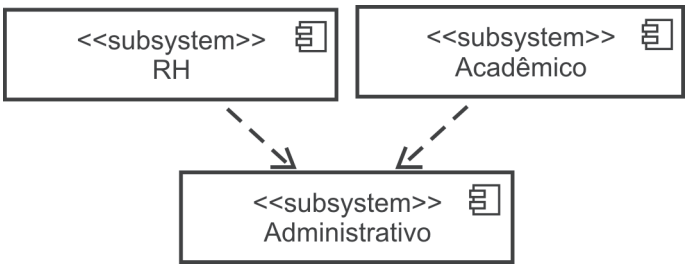


Figura 29: Diagrama de Componentes detalhando estrutura de um sistema.

A Figura 29 exibe um diagrama de componentes detalhando a estrutura de um sistema. Podemos notar que os subsistemas RH e Acadêmico dependem do sistema Administrativo. Da mesma forma, poderíamos detalhar dependências entre pacotes, classes e outros subsistemas. Isso descreve o agrupamento utilizado na construção de um sistema, exibindo ainda sua organização e dependências.

Diagrama de Implantação

O diagrama de implantação exibe a arquitetura de execução de um sistema, detalhando os componentes físicos que executam no ambiente de execução utilizado, sendo possível detalhar que módulo funcionará em qual dispositivo e como será sua comunicação com outros dispositivos.

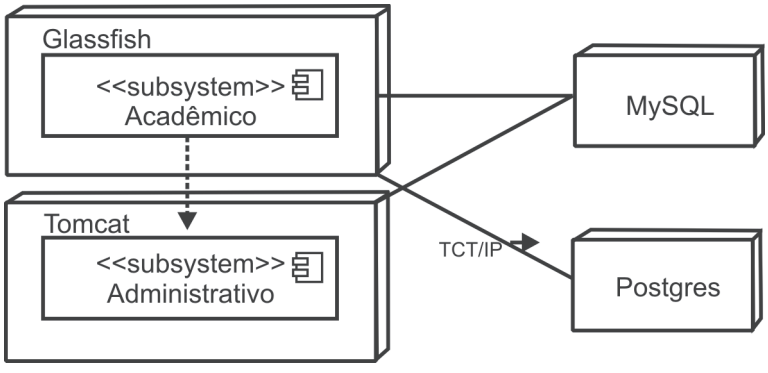


Figura 29.1

Esse tipo de diagrama também é muito associado à exibição da arquitetura de um sistema, deixando claro como será organizada sua execução.

Considerações Finais

Neste capítulo apresentamos a UML. Essa linguagem é atualmente um padrão de facto para a modelagem no mundo. Demonstramos sua divisão e como utilizá-la a partir de uma ferramenta gratuita, a Astah. No próximo capítulo iremos apresentar como usar a UML para modelar aspectos identificados em uma ER, embora isso já tenha iniciado com a criação de diagramas de caso de uso durante o levantamento de requisitos.

Exercícios Propostos

1. O que é um modelo?
2. O que é a UML?
3. Qual a origem da UML?
4. O que são as visões da UML?
5. Quais são as visões existentes na UML?
6.O que é uma classe? Como ela é representada em UML?
7. O que é um objeto? Como ele é representado em UML?
8. O que é um estado na UML?
9. Quando devemos utilizar estados da UML?
10. O que é um pacote na UML?
11. O que é um componente na UML?
12. Existem diversos tipos de relacionamentos entre classes na UML. Descreva a associação simples, apresentando um exemplo com nome dos papéis e cardinalidades.
13. O que é uma autoassociação?
14. Qual a diferença de uma agregação para uma composição?
15.Cite um exemplo prático de uma Generalização, demonstrando isso no formato prescrito pela UML.
16. O que significa o relacionamento de dependência?
17.Crie um diagrama de caso de uso para um sistema acadêmico, com pelo menos 4 casos de uso.
18. Crie um diagrama de estados para um sistema acadêmico com pelo menos 5 classes.
19.Crie um diagrama de estado para modelar algo que faça parte do seu dia-a-dia.
20. Para que utilizamos o diagrama de atividades?
21. Crie um diagrama de atividades para modelar algo em seu trabalho

ou relacionado ao seu dia-a-dia.
22. Para que serve o diagrama de componentes?
23. Qual o objetivo do diagrama de implantação?

2.3 O Fluxo de análise

O Fluxo de Análise tem como objetivo modelar os conceitos identificados na ER, tornando-os mais próximos do mundo computacional e um pouco mais distante dos usuários finais. Com isso, também é feita uma revisão dos requisitos identificados, visando avaliar sua qualidade.

Atualmente, a Análise Orientada a Objetos é o formato mais utilizado para modelagem dos conceitos existentes em uma ER. Por conta disso, utilizaremos bastante o conceito de classe e objeto. Também será bastante utilizada a linguagem UML, uma vez que a Análise se baseia na modelagem dos conceitos existentes na ER.

Atividades do Fluxo de Análise

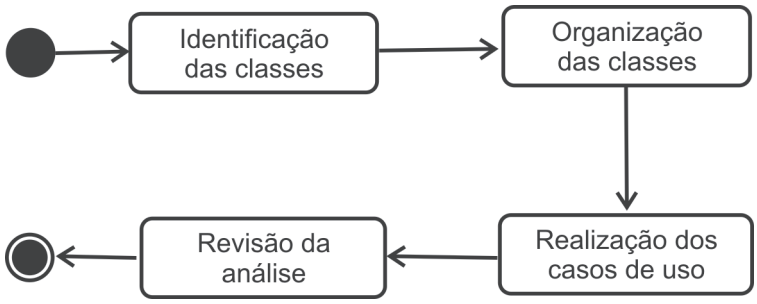


Figura 30: Atividades do Fluxo de Análise.

A Figura 30 exibe as atividades do Fluxo de Análise. Conforme foi ressaltado durante a apresentação do Fluxo de Análise, enfatizamos que essa figura exibe uma sugestão de atividades para o fluxo. Processos como o RUP prescrevem muito mais atividades enquanto processos ágeis prescrevem muito pouco. De forma geral, colocamos aqui apenas o essencial e que provavelmente seja interessante em qualquer processo.

O Fluxo de Análise inicia pela identificação das classes. Nessa atividade teremos que traduzir os conceitos relevantes, identificados na ER em classes, relacionamentos, atributos e operações.

Na atividade Organização das classes é feito uma organização das classes identificadas, criando pacotes (pastas) para melhor agrupamento dos itens e atribuindo estereótipos a todos os elementos identificados.

Não existe consenso sobre as atividades relacionadas à Análise. Neste trabalho detalhamos aquilo que é muito útil na prática e presente em muitos processos.

Na Realização dos Casos de Uso são exibidas as interações entre objetos para se implementar um determinado comportamento, expresso na descrição dos casos de uso da ER. A diferença está no formato dessa descrição, que será feita na forma de troca de mensagens entre objetos.

A Revisão da Análise é o momento em que o trabalho realizado no fluxo é revisto, no intuito de aumentar a qualidade e evitar que problemas se perpetuem no restante das atividades de desenvolvimento.

Identificação das Classes

O principal resultado obtido durante a Identificação das Classes é a criação de diagramas modelando os principais conceitos existentes no sistema.

Um conceito simples de entender e modelar está relacionado aos dados persistentes. Praticamente todas as aplicações existentes necessitam persistir dados. Um diagrama exibindo todas as classes associadas a dados persistentes, demonstrando os relacionamentos existentes, facilita bastante o entendimento do problema e até mesmo a criação de uma solução. Esse diagrama é normalmente denominado Diagrama de Classes de Entidade. Uma entidade é uma classe que está relacionada a dados persistentes.

Dando continuidade ao nosso exemplo com requisitos levantados na Unidade I, poderemos agora iniciar o levantamento das classes persistentes, para depois detalharmos tais classes. Para que isso seja feito, é necessário analisar cada caso de uso à procura de informações que devem ser persistidas.

Não existe uma técnica bem estabelecida e reconhecidamente efetiva para a identificação de classes. O entendimento do contexto descrito na ER e o bom senso apurado são as melhores técnicas existentes!

Iniciando essa análise, a partir do caso de uso Cadastro de Projeto, podemos notar a necessidade de termos uma entidade associada a Projeto. Pela descrição do caso de uso é fácil notar que um projeto deve possuir uma sigla, descrição e um gerente associado. Aqui cabe uma análise a mais: um gerente será outra entidade ou pode ser apenas um campo texto para informação do seu nome? Nossa ideia é cadastrar gerentes de projeto para que possamos associá-los a outros projetos e manter assim um histórico de execuções. Dessa forma é fundamental que os gerentes estejam associados a alguma entidade. Com isso já é possível criar as entidades Projeto e Gerente.

Continuando a análise, podemos notar que o caso de uso Gestão de Gerentes nos fornece mais informações sobre os dados associados a um gerente, tais como nome, email, telefone, login e senha. Assim, podemos completar a classe Gerente, já identificada, adicionando tais atributos.

O próximo caso de uso, Gestão de Membros, nos faz refletir sobre alguns pontos importantes. Um membro pode ser um Cliente ou um Engenheiro de Requisitos, conforme consta no caso de uso. Dessa forma fica a reflexão: será que necessitamos realmente de uma classe de entidade para modelar Gerente ou será que o Gerente de um projeto também não é um Membro? Não existe uma resposta precisa sobre a questão, pois o contexto deve ser analisado. No entanto, definimos nesse trabalho, por achar mais adequado, que devemos ter apenas a classe Membro. Assim, a classe Gerente, identificada no parágrafo anterior será excluída do nosso modelo, ficando apenas Projeto e Membro até o momento. Isso nos gera o modelo exibido na classe projeto que contém vários outros atributos identificados a partir da análise do caso de uso Controle de Projetos.

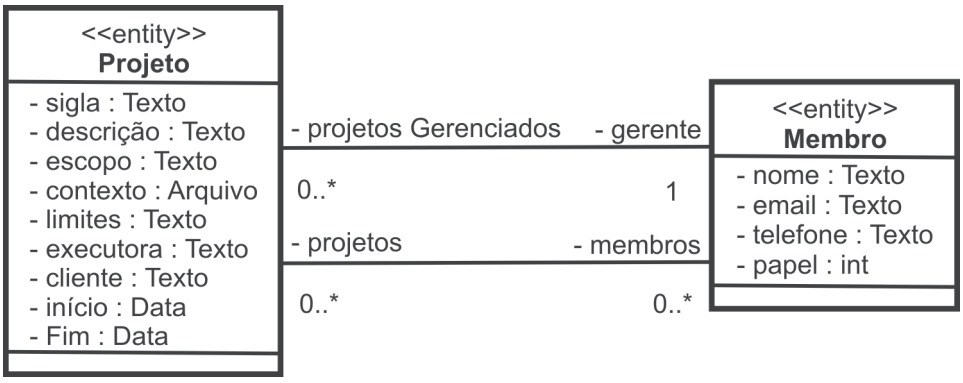


Figura 31: Modelo representando Projeto e Membro.

Outra constatação interessante é que os membros possuem login e senha para acesso ao sistema. Tomamos a decisão de modelar isso em uma classe separada, indicando que os membros são usuários do sistema. Observe que isso foi uma decisão de modelagem tomada pela equipe, sem que houvesse uma indicação na ER que pudesse nos levar a isso.

Poderíamos ter mantido os atributos de Usuário na classe Membro, não necessitando dessa classe extra. Porém, acreditamos que a modelagem utilizada é mais apropriada e permite uma maior inteligibilidade à representação. Por conta disso, resolvemos adotá-la. Isso é exibido na Figura 32.

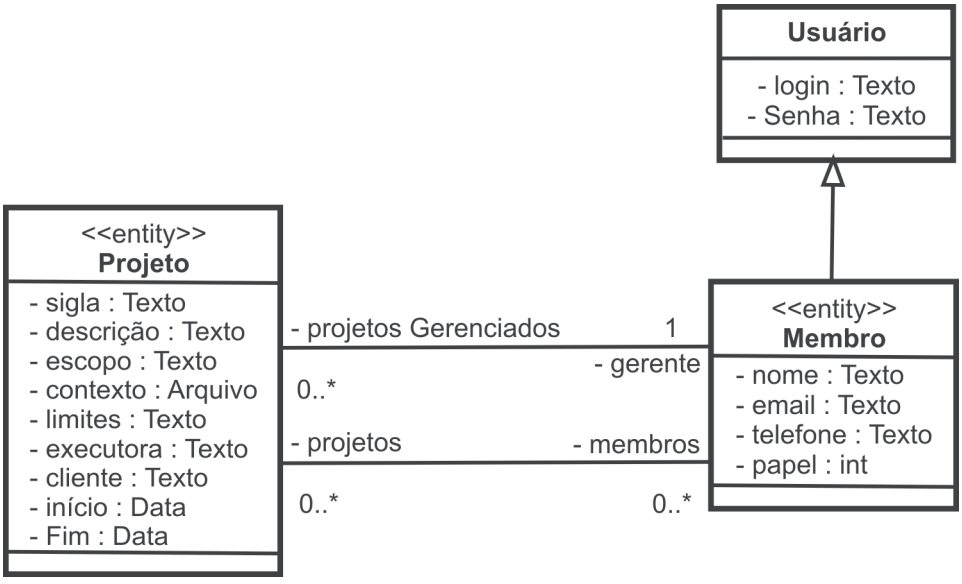


Figura 32: Extração da classe Usuário.

Continuando nossa análise, partirmos para o caso de uso Gestão de Requisitos. É fácil notar que um projeto pode ter vários requisitos, os quais possuem atributos simples: identificador, nome, descrição, tipo, prioridade, complexidade e situação. Os atributos relacionados a uma lista com valores fixos e pré-definidos, como é o caso de tipo, prioridade, complexidade e situação, podem ser modelados como tipo inteiro, onde cada valor pode ser representado por um número. No entanto, a análise desse caso de uso nos levanta duas questões interessantes:

1. Qual o tipo de associação entre projeto e requisitos? Ela não parece ser uma associação simples. Os requisitos fazem parte do projeto e parece não sobreviver sem a existência do projeto. Isso nos remete ao conceito de composição. Dessa forma, modelaremos a associação entre projeto e requisitos como uma composição.

2. No protótipo da tela de gestão de requisitos existe um histórico de alteração. Isso nos indica que será necessária a existência de uma entidade que modele as alterações de requisitos. Nesse caso, a classe requisito deverá ter uma associação com os históricos de alteração.

A modelagem das classes até as considerações acima, nos remete a um modelo conforme o exibido na Figura 33.

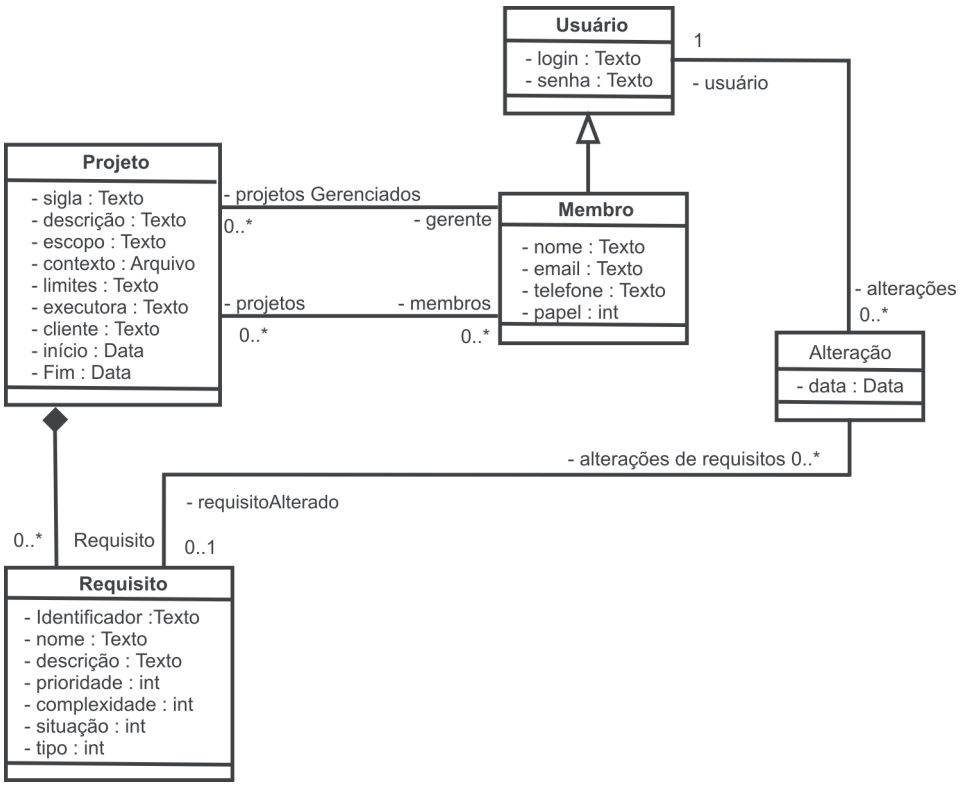


Figura 33: Modelo do sistema contendo requisitos e alterações.

Os próximos casos de uso: Gestão de Atores e Gestão de Casos de Uso estão bastante relacionados. A classe Ator possui atributos simples (nome e descrição) e pode estar relacionada a vários casos de uso, uma vez que é possível registrar vários casos de uso associado a um ator.

O caso de uso Gestão de Casos de Uso possui diversos atributos e relacionamentos com outras classes. É fácil perceber que um caso de uso pode ter vários protótipos de tela. Isso nos remete a uma nova classe (Protótipo) e o relacionamento com essa classe. Como um protótipo não consegue viver dissociado de seu caso de uso, isso indica uma composição entre os dois.

Algo bem mais interessante acontece com os fluxos. Observe que um caso de uso tem um fluxo principal, diversos fluxos alternativos e subfluxos. Um fluxo contém seu nome e passos. Fluxos alternativos contêm, além disso, pré-condições. Isso denota que existe uma generalização envolvida nesse relacionamento. Dessa forma, fluxo é algo que contém nome e passos; fluxo alternativo é um fluxo com pré-condições. Observe que na Figura 34 apresentamos um exemplo para tal modelagem. A classe Caso de uso contém dois relacionamentos com

Fluxo: uma para representar o fluxo principal e outro para representar os possíveis subfluxos existentes. É importante ressaltar que essa é uma boa modelagem para o problema, mas é difícil de ser percebida para iniciantes na execução da Análise.

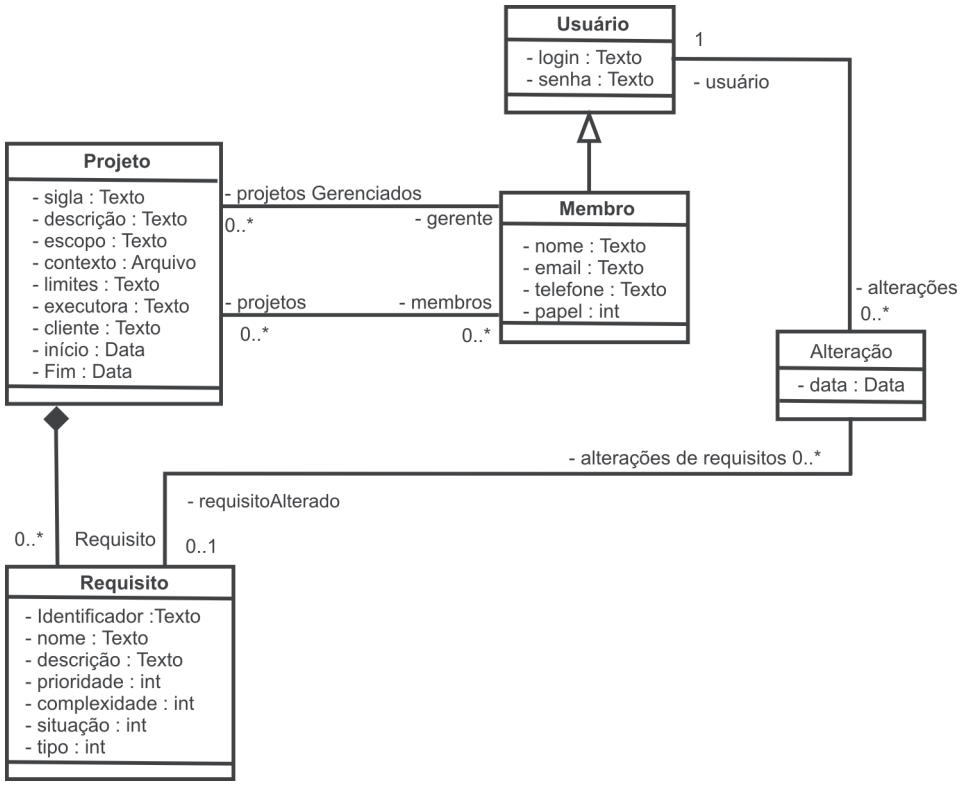


Figura 34: Modelagem para Ator e Caso de uso.

A última parte da ER a ser considerada para Análise é a parte relacionada à Revisão. Essa parte também possui uma complexidade associada, não sendo trivial sua análise e entendimento. Um projeto pode possuir diversas revisões, os quais têm critérios associados, com uma indicação de aprovação e uma observação, além da associação ao item sendo avaliado, que pode ser um requisito ou caso de uso. Mas como representar isso?

Nesse caso teremos que criar uma classe adicional, não visível pelos usuários diretamente, para agrupar todos os elementos necessários ao relacionamento. Assim, teremos um item de avaliação que estará relacionado ao critério utilizado para avaliação, à revisão em andamento e ao elemento sendo revisado (requisito ou caso de uso). É muito comum durante uma análise de um caso real, encontrarmos a necessidade de criação de itens para agrupamento como foi o caso da revisão.

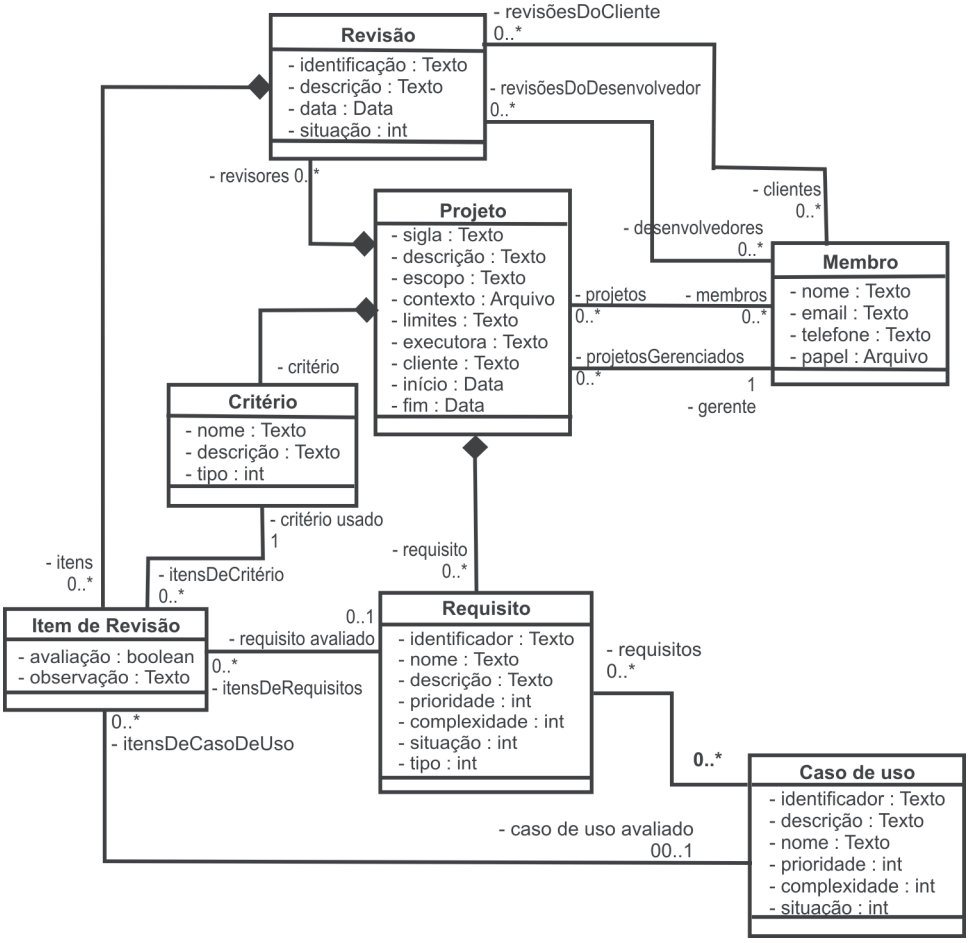


Figura 35: Modelagem dos aspectos relacionados a revisão.

Observe na Figura 35 o resultado da modelagem dos aspectos ligados a uma revisão. O projeto contém revisões que possuem diversos itens de revisão. Esses itens são os responsáveis pelo agrupamento que indica qual o critério utilizado, o que está sendo revisado (requisito ou caso de uso) e qual o resultado dessa avaliação de item em específico, dado pelos atributos avaliação e observações.

Ainda existem dois casos de uso na ER: Geração da Especificação e Relatório de Acompanhamento. Porém, nenhum dos dois acrescenta dados novos. Com isso então, finalizamos a identificação das classes de entidade. O resultado final dessa Análise é mostrado na Figura 36.

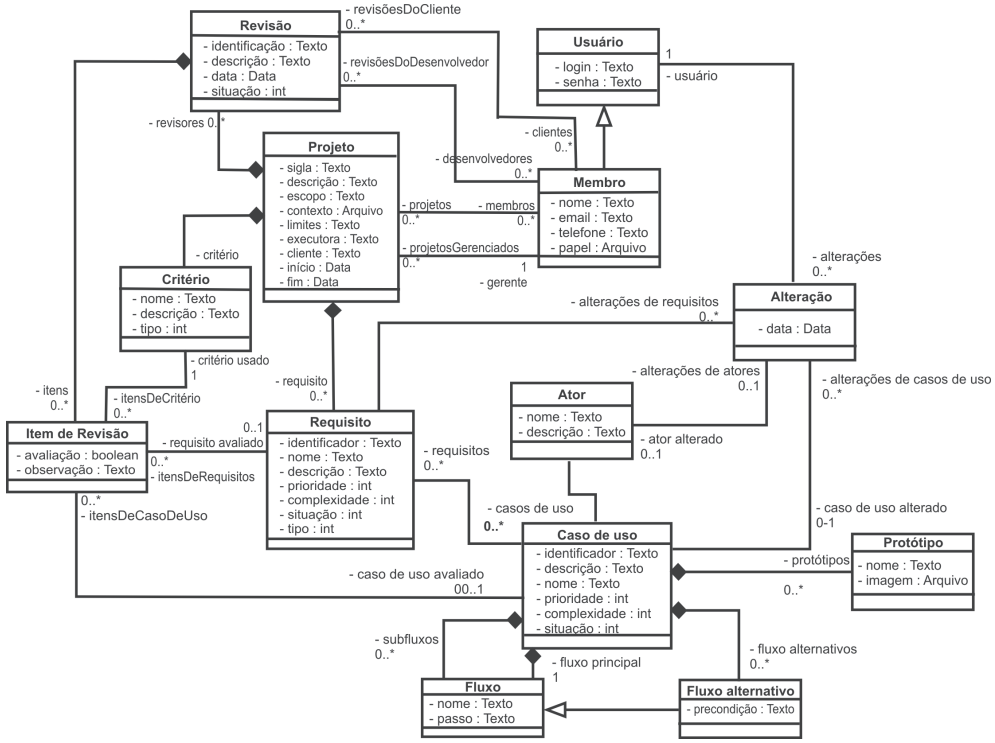


Figura 36: Modelagem final dos conceitos existentes na ER do ReqG.

No entanto, é preciso destacar que a Identificação das Classes ainda não foi concluída. Ainda devemos identificar todas as demais classes do projeto. Isso inclui as classes relacionadas às telas e relatórios denominados classes de fronteira, além das classes que devem conter as regras de negócio denominadas classes de controle. A identificação dessas classes permite a criação das realizações dos casos de uso. No entanto, a realização é algo que não é prescrito pelos métodos ágeis e será apenas demonstrado neste trabalho sem tanta profundidade.

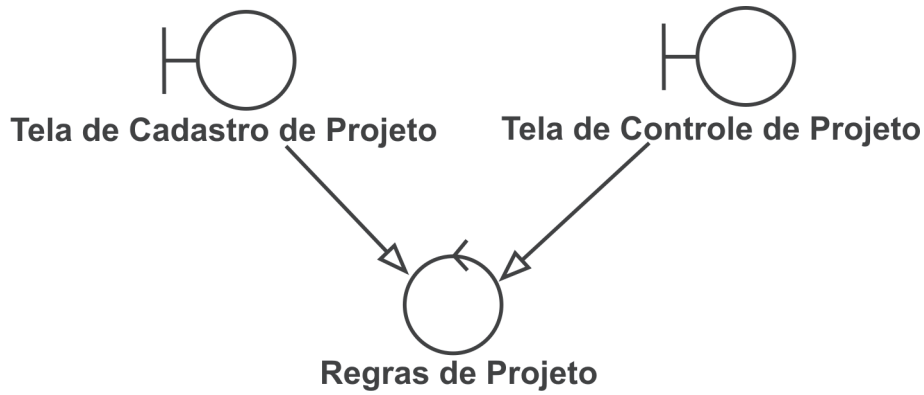


Figura 37: Exemplo de classes de fronteira e controle.

A Figura 37 exibe um diagrama contendo duas classes de fronteira e uma classe de controle. Essas classes foram criadas para representar as telas identificadas para os casos de uso Cadastro de Projetos e Controle de Projetos, além de uma classe para modelar as regras de negócio relacionadas a projeto, denominada Regras de Projeto. Essas classes foram estereotipadas com os estereótipos <<boundary>> e <<control>>. Isso possibilitou essa apresentação utilizando a forma icônica, onde as classes se apresentam com imagens não tradicionais.

Os estereótipos dão mais poder à UML, permitindo classificar elementos com alguma coisa em comum. Um exemplo para facilitar o entendimento seria, por exemplo, modelar hospital. Poderíamos ter símbolos específicos para representar médicos e pacientes. Com isso, estaríamos estereotipando um elemento. Assim, todos os médicos teriam o mesmo símbolo, já indicando algumas propriedades e comportamento associado. Esse símbolo poderia estar associado e uma imagem, destacando cada elemento em um diagrama.

A Figura 37 nos demonstra que as classes Tela de Cadastro de Projetos e Tela de Controle de Projetos representam classes de apresentação (uma vez que estão estereotipadas como fronteira) e possuem uma associação com a classe Regras de Projeto. Essa classe contém as regras de negócio relacionado a projetos, uma vez que ela está estereotipada como controle. Normalmente, ao identificarmos uma classe já definimos seu estereótipo, embora isso seja uma atividade diretamente relacionada à organização das classes, apresentada a seguir.

Organização das Classes

Uma vez identificadas todas as classes de entidade, devemos organizá-las, de forma a facilitar o trabalho e entendimento do projeto. A organização está associada à criação de pacotes com os agrupamentos necessários, além da atribuição dos estereótipos exigidos.

Devemos utilizar algum critério para agrupamento das classes. No entanto, não existe uma regra específica para isso. Geralmente, o bom senso é o melhor dos guias. Como exemplo, observe o agrupamento sugerido para o nosso exemplo, apresentado na Figura 38. Dividimos o projeto em quatro pacotes:

1. Administração: para conter os itens relacionados à administração e uso do sistema, composto pelas classes Alteração (registro de histórico de alteração) e Usuários (representação dos usuários do sistema).

2. Projeto: contendo a representação de um projeto e dos seus

membros.

3. Requisitos: contendo os elementos ligados aos requisitos como os casos de uso, protótipos, atores, fluxos e fluxos alternativos.

4. Revisão: contendo os itens diretamente ligados a uma revisão como os critérios, itens de revisão e a própria revisão.

Conforme comentado, o importante é criar uma organização no projeto que favoreça seu entendimento. Se isso acontecer, o resultado da execução da atividade foi bom.



Figura 38: Organização das classes para o ReqG.

Realização dos Casos de Uso

Uma realização de caso de uso mostra como um caso de uso será implementado, a partir da colaboração dos objetos que os implementam. Isso é feito a partir da criação de diagramas de interação (sequência ou colaboração) detalhando o caso de uso.

De forma geral, uma realização pode ser considerada como a descrição do caso de uso, similar à descrição textual, só que utilizando objetos e operações.

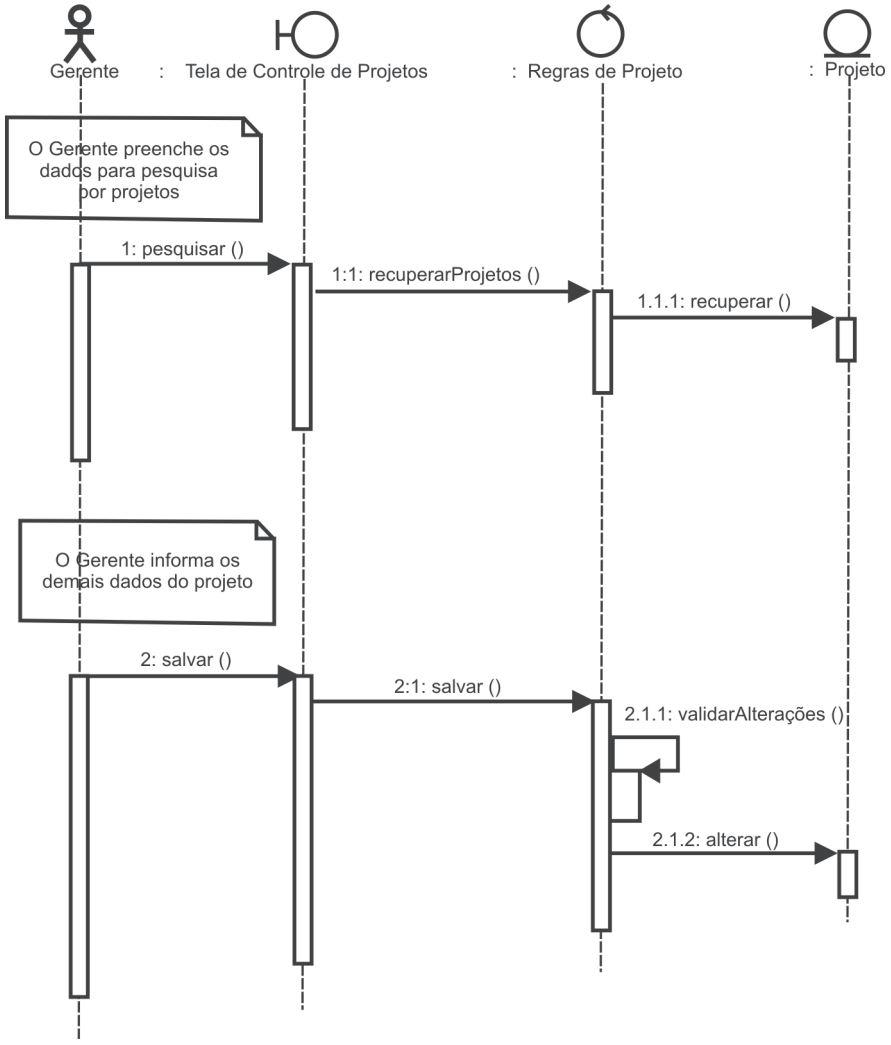


Figura 39: Realização para o caso de uso Controle de Projeto.

A Figura 39 exibe a realização para o caso de uso Controle de Projetos. Podemos observar que na realização estão todas as classes relacionadas ao caso de uso: sua tela, regras de negócio e as entidades associadas. No caso em específico, existe apenas uma entidade relacionada. Se esse caso de uso tivesse que criar relacionamentos com diversos objetos de outras classes, elas deveriam estar na realização, além de existir mensagens até esses objetos.

Conforme já comentado, alguns processos prescrevem a realização de todos os seus casos de uso, outros apenas dos mais importantes e, alguns, como os baseados em métodos ágeis, não prescrevem qualquer realização. No entanto, havendo a necessidade de criar realizações será necessário organizá-las. Um exemplo de tal organização pode ser

exibido na Figura 40. É importante mantermos um padrão para evitar dificuldades de entendimento.

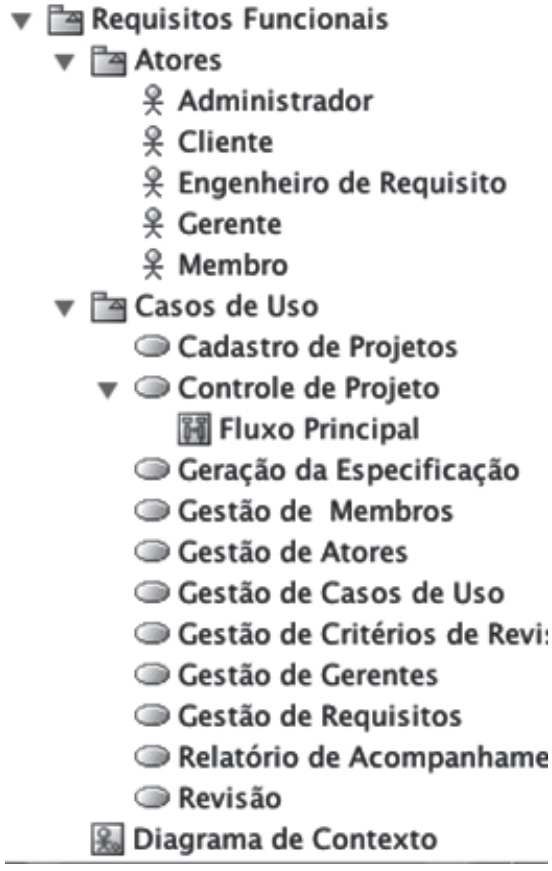


Figura 40: Organização das realizações de um caso de uso.

Uma vez identificadas todas as classes de entidade, devemos organizá-las, de forma a facilitar o trabalho e entendimento do projeto. A organização está associada à criação de pacotes com os agrupamentos necessários, além da atribuição dos estereótipos necessários.

Revisão da Análise

A revisão da análise é o momento de fazermos uma verificação geral nos diagramas e classes identificadas. A revisão da Análise pode ser feita da mesma forma que a revisão dos Requisitos. Os artefatos são exatamente os mesmos. Até mesmo os critérios podem ser iguais. A mudança está na forma de se avaliar o atendimento aos critérios. Por exemplo, o critério completude deveria verificar se todos os atributos existentes nos protótipos estão modelados nas entidades. Isso é um dos principais itens para a revisão da Análise e que precisa ser executado

com bastante atenção.

No Anexo V deste trabalho temos a planilha utilizada para a revisão do modelo de Análise, contendo a descrição dos critérios associados, bem como o resultado da própria revisão executada.

Exercícios Propostos

1. Qual o objetivo do fluxo de análise?
2. Descreva brevemente as atividades do fluxo de análise.
3. O que é uma classe de entidade?
4. Como devemos proceder para identificar classes a partir de um ER?
5. O que é um estereótipo na UML?
6. Por que devemos organizar as classes de um modelo de análise?
7. O que é uma realização de caso de uso?
8. Qual o principal objetivo da revisão da Análise?
9. Qual é um dos principais pontos a se verificar durante uma revisão da Análise?

2.4 Análise de pontos de função

A cada dia que passa, cresce a necessidade mundial por sistemas de informação, que estão cada vez mais presentes em nossas vidas, auxiliando as mais variadas tarefas. Essa abrangência dos sistemas causa também um aumento na sua complexidade, sendo sempre importante que eles sejam desenvolvidos o mais rápido possível, cumprindo sempre um cronograma pré-estabelecido para tal.

Para se estabelecer estimativas de custo e prazo para projetos de software, é necessário ter uma base histórica de custo e prazo de outros projetos!

Um problema frequente relacionado ao desenvolvimento de software está relacionado ao estabelecimento de estimativas de prazos e custos equivocadas, o que leva a dificuldade em cumprimento do que foi estabelecido. Fora isso, é provável que haja uma correria no final do projeto, possibilitando a perda de qualidade para que seja possível cumprir prazos irreais.

Tais problemas são muito comuns. Mas uma pergunta que pode ser feita é: como podemos evitar tais problemas? Uma possível resposta para isso é usar experiências passadas, com o apoio de técnicas associadas ao controle e gestão de projetos. Isso exige o uso de métricas

que permitam associar um trabalho a uma medida clara e objetiva de quanto foi gasto para fazê-lo.

A Mensuração em software é o processo de definir, coletar, analisar e agir sobre medidas que possam melhorar não só a qualidade dos produtos, como também o próprio processo de desenvolvimento. É fundamental que a mensuração gere dados quantitativos que possam ser utilizados para auxiliar o processo de tomada de decisões.

A produtividade é uma medida fundamental para realização de estimativas. Ela indica a capacidade de produção por unidade de tempo.

Muitas coisas podem ser medidas durante o desenvolvimento de software. São exemplos dessas possibilidades: o tamanho do produto, complexidade, esforço necessário para sua construção, cronograma, etc. Mas uma importante pergunta que devemos fazer é: para que necessitamos medir o tamanho de um produto de software? Várias são as razões. Apenas medindo é que podemos estimar algo. Isso viabiliza a determinação de preço do produto, além de facilitar o planejamento do projeto.

As medidas de tamanho de produtos de software possuem correlação com o esforço necessário para sua produção. Conhecendo tais medidas podemos ser capazes de realizar estimativas com muito mais chances de acertarmos, uma vez que tendo medidas podemos encontrar a produtividade da equipe. Com base nessa produtividade podemos fazer previsões para outras situações.

A Produtividade é uma medida importante para a realização de estimativas. É baseada na medida do produto do trabalho dividido pelo esforço para produzi-lo. Um exemplo simples disso é a produtividade de um pintor. Imagine que ele consegue pintar 20m2 de parede por dia. Essa medida, 20m2/dia, pode ser utilizada para nos guiar nas próximas estimativas. Se existir um serviço para pintar 200m2 de paredes, poderíamos estimar 10 dias úteis de trabalho.

Da mesma forma, a produtividade relacionada a software pode nos ajudar em estimativas. Para isso, é fundamental que o esforço dedicado para realizar um trabalho seja cuidadosamente medido, de forma precisa. De nada adianta termos o registro do esforço se ele não for preciso. Isso certamente vai gerar estimativas equivocadas. Imagine como exemplo a medida de produtividade do pintor comentado acima. Se houvéssemos errado na medida do esforço empregado para pintar uma parede, imaginando, por exemplo, que para pintar 20m2 o profissional gastou apenas 4h ao invés de 8h, qualquer medida feita com base nessa produtividade (20m2/4h) estará associada a uma grande taxa de erro.

Dessa forma torna-se claro a necessidade por mensuração, uma

vez que é base para encontrarmos a produtividade que nos permite realizar estimativas. No entanto, para a área de software temos um problema adicional: a mensuração na área de software não é nada trivial! É necessário utilizar uma medida que seja padronizada, uniforme para uma aplicação e inteligível para a população. Existem algumas medidas possíveis, como linhas de código, casos de uso e módulos implantados. Mas vamos ver que tais medidas não são as mais adequadas.

De forma geral, uma boa medida de tamanho deve possuir algumas características fundamentais para garantir sua adequabilidade:

1. Ela deve ser deduzida dos requisitos, ou seja, a partir da especificação de requisitos associadas ao produto deve ser possível estimar o tamanho do produto. Um exemplo disso é a medida de tamanho de uma casa, em que normalmente é utilizado o tamanho em metros quadrados. A partir da especificação de produto (casa), detalhando número de quartos, banheiros, salas, é possível deduzir o tamanho da edificação. Se for realizado um pré-projeto, incluindo uma planta baixa, essa estimativa tende a ser muito melhorada, uma vez que ela serve como um protótipo da construção a ser realizada;

2. Ser contável a partir de um procedimento definido. A medida deve permitir que a contagem do tamanho do produto seja facilmente mensurável quando o mesmo se encontra- completo. Continuando no exemplo de uma casa, podemos notar que a casa construída é fácil medi-la em m2, permitindo assim uma fácil comparação entre o que foi estimado e o que foi realmente feito.

Existem diversas medidas de tamanho que podem ser utilizadas para software. Dentre elas destacamos:

1. Linhas de código;
2. Pontos de função;
3. Pontos de caso de uso;
4. Pontos de objetos.

A medida em linhas de código é muito interessante por muitos aspectos, dentre eles, podemos destacar que ela é a mais simples e barata de se utilizar. Além disso, ela pode ser contada de forma totalmente automática, uma vez que possui as características comentadas acima.

Após as explicações acima, detalhando as vantagens associadas ao uso de linhas de código como medida de tamanho para software, podemos nos questionar: por que ela não seria a medida ideal para ser utilizada? Por que utilizar outra medida?

A resposta para as perguntas anteriores é, também, relativamente simples: linhas de código somente podem ser medidas DEPOIS do produto concluído. No entanto, essa medida não é adequada por que ela não pode ser obtida dos requisitos. Ou seja, ela não pode nos ajudar em estimativas, pois ela é adequada para contar o tamanho de produtos construídos e não de produtos a construir. Além disso, a medida em linhas de código não é independente de tecnologia: um produto desenvolvido em Java deve ter tamanho diferente de um desenvolvido em Delphi, que deve ser diferente de um desenvolvido em Ruby. Isso não é uma característica boa para uma medida.

A medida em linhas de código é muito boa: simples, baixo custo, bem definida. Mas ela só pode ser obtida DEPOIS do produto pronto! Isso impossibilita seu uso para estimar a construção do produto.

Por esses motivos, foram criadas outras medidas apropriadas para o desenvolvimento de software. Uma dessas medidas, denominada pontos de função é bastante utilizada no mundo e objeto de estudo deste capítulo.

1. Introdução a Análise de Pontos de Função

A Análise de Pontos de Função (APF) mede a funcionalidade entregue ao usuário. Uma característica interessante da APF é que ela é independente da forma de implementação, ou seja, se o produto vai ser desenvolvimento em Java, C, Ruby, Delphi, isso não influencia em nada a contagem realizada.

APF é independente de tecnologia. Isso permite comparar empresas e tecnologias de forma efetiva!

Essa característica de independência de tecnologia possibilita algo muito interessante: a comparação entre empresas e linguagens. Existindo uma medida independente de tecnologia, é possível analisar se a empresa X trabalha de forma mais rápida que a empresa Y, caso ambas iniciem o desenvolvimento do produto. Da mesma forma, podemos analisar se a linguagem A permite maior produtividade que a linguagem B.

Pontos de função foram inicialmente propostos por Allan Albrecht em 1979. A formalização das regras de contagem teve início em 1984, pela IBM, a partir da criação de manuais para guiar a contagem. Em 1986 foi criado o International Function Points Users Group (IFPUG). Essa organização tem como missão ser reconhecida como líder na promoção e encorajamento do gerenciamento do desenvolvimento de software a partir do uso da APF. Dentre outras atribuições, o IFPUG mantém o manual de contagem de pontos de função, que atualmente está em sua versão 4.3.1 e pode ser obtido na página oficial da organização (http://www.ifpug.org/).

A técnica de Análise de Pontos de Função foi concebida com dois

objetivos principais:

1. Possibilitar a mensuração de funcionalidades que um software oferece aos seus usuários, de forma independente da tecnologia utilizada para sua implementação;

2. Criar uma forma de contagem que gere um custo adicional pequeno, não sendo caro realizar a medição e que permita que seu uso seja feito entre diferentes projetos e organizações.

Após sua criação e uso por parte da comunidade, foi possível notar que a técnica de APF possuía não somente os benefícios inicialmente propostos, mas permitia também:

1. Determinar o tamanho de uma aplicação já construída, a partir da funcionalidade nela contida;

2. Determinar os benefícios de um pacote de aplicativos para uma organização, baseados em quantas funções eles atendem dentro do contexto de necessidades existentes, permitindo uma comparação quantitativa sobre quem atendeu mais em termo de tamanho.

Além disso, a APF permitiu que o controle de produtividade dos elementos que fazem parte de uma equipe de desenvolvimento fosse medido e acompanhado. Isso foi possível devido ao fato de existir um método simples para se medir o tamanho de uma funcionalidade, aliada ao tempo para se desenvolver tal funcionalidade. Isso gera a produtividade que conforme já foi dito, é a razão entre o tamanho de um produto de trabalho desenvolvido pelo esforço necessário para sua produção.

Fora a produtividade, foi possível estabelecer um parâmetro de comparação com outros projetos, permitindo estimar outros recursos necessários para uma empreitada. Isso permite um melhor planejamento, pois independe da tecnologia utilizada, pontos de função servem para normalizar os dados relativos a tamanho, permitindo a comparação entre empresas, tudo isso de forma independente do que usam para produzir seu trabalho.

De forma geral, as principais vantagens do uso de APF são:

1. Transparência para o usuário final. Com o uso de APF, o usuário consegue entender a medida de tamanho relacionada ao produto que ele deseja. Isso também permite o acompanhamento da evolução do produto. Isso também estimula o próprio usuário e o torna mais ativo no desenvolvimento.

2. Apoio à estimativa de tempo, recursos e custos. O uso de APF permite estimar o tempo necessário para desenvolver algo, baseado em uma comparação da produtividade de uma equipe em projetos

passados. A partir dessa base histórica é possível encontrar o tempo gasto para desenvolver um ponto de função, permitindo assim estimar qualquer trabalho que possa ser contado utilizando APF. É importante notar que, mesmo se tivermos um detalhamento do projeto, mesmo que ele esteja em estágios iniciais de desenvolvimento, é possível já iniciar as estimativas, pois a contagem pode ser feita de forma preliminar. Essa é outra característica muito interessante.

3. Facilitar contratos de terceirização. Muitas vezes, uma empresa terceirizada pode alegar ter trabalho por 100h em parte de um software sem ter conseguido muita evolução. Por isso a contagem em horas não é considerada uma forma eficiente para se avaliar contratos terceirizados. É fundamental acompanhar tais contratos pela medida de funcionalidade desenvolvida em um período de tempo. Por conta disso, APF é a técnica sugerida para acompanhar praticamente todos os contratos de desenvolvimento existentes nas instituições públicas.

De forma geral, a APF é uma técnica muito utilizada no mundo e bastante consolidada. Isso não significa que ela não tenha defeitos. Possuem vários contextos de uso bem definidos. Iremos tratar disso mais a frente. Por hora, vamos detalhar um pouco melhor como ela funciona.

A contagem de PF é baseada em 5 elementos: dados mantidos pela aplicação, dados mantidos por outra aplicação e usados pela aplicação sendo contada, funções que mudam dados, funções que consultam dados e realizam processamento nesses dados e funções que apenas consultam dados.

O Processo de Contagem

Muito já falamos sobre APF. Mas como ela funciona exatamente? Nesta seção trataremos disso. Uma representação dos principais elementos relacionados à contagem de pontos de função é exibido na Figura 41. Para a contagem temos que levar em consideração cinco elementos:

1. Arquivo Lógico Interno (ALI) que representa os dados mantidos pela aplicação.

2. Arquivo de Interface Externa (AIE) que representa dados utilizados pela aplicação, mas mantidos por outras aplicações.

3. Entrada Externa (EE) que representa as entradas de dados que causam mudanças nos ALIs.

4. Saída Externa (SE) que causa a exibição de dados relacionados à aplicação, normalmente exigindo algum tipo de processamento.

5. Consulta Externa (CE) que causa a simples exibição de dados relacionados à aplicação.

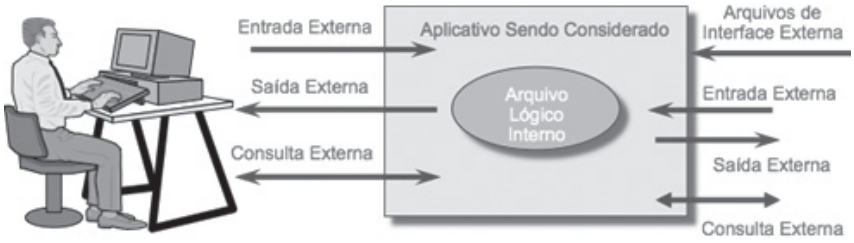


Figura 41: Representação dos principais elementos associados à APF.

A contagem de pontos de função envolve um procedimento simples, baseado em poucos passos:

1. Determinar o tipo de contagem;
2. Identificar o escopo de contagem e a fronteira da aplicação;
3. Contar funções de dados;
4. Contar funções de transação;
5. Determinar o fator de ajuste;
6. Calcular o total de pontos de função ajustados.

Cada um desses passos será detalhado nas próximas seções. Por enquanto, iremos detalhar mais alguns aspectos relacionados à contagem.

De forma geral, existem três tipos de contagem de PF:

1. Projeto de desenvolvimento: Nesse caso é medido o tamanho das funções oferecidas aos usuários, decorrentes do desenvolvimento de um software, que será criado no projeto, ou seja, ainda não existe ainda.

2. Projeto de evolução: É feita a medida das modificações de funcionalidades associadas a uma aplicação existente, ou seja, é a contagem das mudanças em um software que já foi desenvolvido, mas precisa ser alterado.

3. Aplicação. Está associado à medida das funcionalidades existentes em uma aplicação. Essa medida é inicializada quando a contagem do projeto de desenvolvimento é concluída. Cada vez que um projeto de evolução é realizado, a contagem da aplicação deve ser atualizada, incorporando a medida das mudanças efetivadas.

Para que uma contagem possa ser realizada é necessário definir o objetivo da contagem, deixando claro para que seja realizado o trabalho de mensuração. Existem diferentes objetivos possíveis. É necessário também estabelecer o escopo da contagem, determinando exatamente o que deve ser considerado. Além disso, é necessário estabelecer o limite existente entre o sistema e o mundo exterior, uma

vez que isso ajuda a guiar a contagem.

Podem existir diferentes objetivos para uma contagem. Um possível objetivo é fornecer insumos para a estimativa do esforço de desenvolvimento. Podemos contar o tamanho de um software, a partir da sua especificação, com o intuito de utilizá-lo para estimar o tempo de desenvolvimento. Isso é feito com base em outros projetos conjunto de aplicações instaladas. Nesse caso, realizamos uma contagem apenas para termos o inventário dos sistemas existentes. Outro objetivo é comparar o tamanho das funcionalidades oferecidas por produtos concorrentes. Imagine que estamos querendo selecionar o produto que mais atenda a certa especificação. Poderíamos contar o tamanho de funcionalidade atendida por um e pelo outro, facilitando assim saber quem mais atende.

Além do objetivo, é necessário definir o escopo da contagem. Isso significa que devemos definir um (sub) conjunto do software a ser medido, utilizando como base o objetivo definido. É necessário determinar exatamente o que deve ser incluído na contagem. Em alguns casos isso pode incluir mais de uma aplicação.

Em projetos de evolução, o escopo da contagem pode incluir todas as funções adicionadas, alteradas ou excluídas. Em projetos de desenvolvimento isso inclui todas as funções construídas ou customizadas pelo projeto. Em aplicações inclui toda a funcionalidade ou apenas a fração que tem valor para o usuário, como por exemplo, no caso de comparação entre ferramentas que mais atendem certas funcionalidades.

Para que a contagem aconteça, também é importante definir a fronteira da aplicação. Isso significa que devemos estabelecer uma divisão conceitual entre a aplicação e mundo externo, uma vez que temos que saber exatamente o que está dentro e o que está fora da aplicação, pois isso tem influência direta na contagem. Os dados mantidos pela aplicação possuem contagem diferenciada de dados apenas consultados, mantidos externamente. É fundamental saber quais são as funções que operam utilizando dados que cruzam a fronteira da aplicação a partir de transações.

A fronteira da aplicação deve ser definida com base na visão do usuário. Ele deve ser independente do escopo da contagem. Ela deve ser delimitada em áreas funcionais, como vistas pelo usuário e não baseada em considerações técnicas. A fronteira é dada por aquilo que o usuário percebe. Assim, analisando a aplicação pela visão do

usuário conseguimos dizer o que faz parte e o que não faz parte da aplicação.

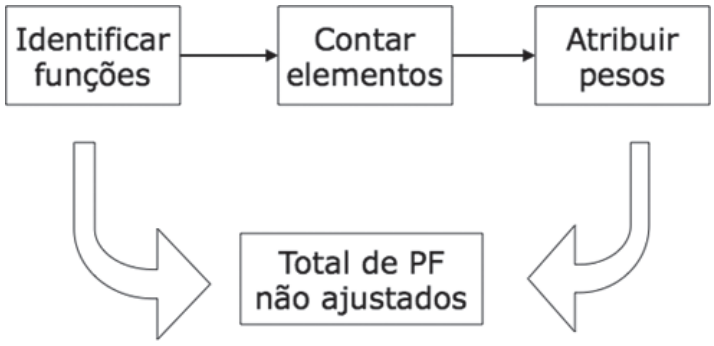


Figura 42: Ações para cálculo em PF.

A Figura 42 exibe as ações relacionadas à contagem em Pontos de Função (PF). Inicialmente, é necessário identificar todas as funções existentes na aplicação e em seguida, é necessário contar os elementos. Podem existir diversos elementos associados a uma função, sendo necessário contá-los adequadamente. Após essa contagem será possível estabelecer os pesos relacionados a cada parte a ser considerada, identificando o tamanho em PF.

A identificação de funções é o momento em que devemos listar todas as funções existentes na aplicação. Isso deve ser feito levando-se em consideração o escopo da contagem, uma vez que ele determina o que deve ser incluído. Cada função deverá contribuir para parte do tamanho total da aplicação. Basicamente, existem duas categorias de funções:

1. Funções de dados;
2. Funções de transação.

As funções de dados representam a funcionalidade oferecida ao usuário para cumprir requisitos de dados, ou seja, elas são a representação de todos os dados existentes na aplicação. Elas podem ser categorizadas em dois tipos:

1. Arquivo Lógico Interno (ALI);
2. Arquivo de Interface Externa (AIE).

É importante ressaltar que o termo “arquivo” refere-se a grupos de dados logicamente correlatos, independente de sua forma de implementação. Assim, um ALI pode representar um dado que foi implementado sob a forma de uma tabela em um banco de dados. No caso de um sistema de biblioteca, poderíamos ter como funções de dados:

1. Usuário da biblioteca;
2. Título;
3. Exemplar;
4. Empréstimo;
5. Reserva.

Os AIEs são similares aos ALIs, com a diferença que são mantidos por outra aplicação e não pela aplicação sendo considerada na contagem. Ou seja, os AIEs são ALIs em outras aplicações. Por exemplo, imagine que em um sistema de biblioteca, não tenhamos os dados de endereço (logradouro, bairro, cidade, estado) de um usuário. Ao invés disso, temos apenas o CEP e o número. Assim, os dados de endereço serão considerados como um AIE, pois não serão mantidos pela aplicação.

As funções de transação representam a funcionalidade oferecida ao usuário para processar os dados da aplicação. Elas podem ser de três tipos:

1. Entrada Externa (EE);
2. Saída Externa (SE);
3. Consulta Externa (CE).

A diferença entre um ALI e um AIE é simplesmente o fato que um ALI é mantido pela aplicação sendo contada e o AIE não!

Conforme já comentado, uma EE representa alguma função que causa algum tipo de mudança em dados mantidos pela aplicação. São exemplo de EE em um sistema de biblioteca o empréstimo de livro, reserva de exemplar, inclusão de novo usuário. Cada uma dessas funções deve causar uma alteração em um dado mantido em algum ALI da aplicação.

Uma SE representa a geração de dados na forma de um relatório, que envolva algum tipo de processamento associado. É exemplo desse tipo de função o quantitativo de empréstimos por período e livros com maior quantidade de reserva.

Uma CE representa algo parecido com uma SE, porém não envolve nenhum tipo de processamento complexo para sua geração. São exemplos de tais funções a listagem de livros emprestados e usuários ativos.

Uma vez tendo identificado todas as funções, precisamos contar os elementos presentes. Para o caso de funções de dados é necessário contarmos os tipos de registros e os campos de dados envolvidos. Nas próximas seções iremos fazer isso para o exemplo da apostila. Para cada função de transação é necessário contar os arquivos referenciados e os campos de dados.

A partir dessa contagem podemos classificar cada função quanto à complexidade, que pode ser Baixa, Média ou Alta. Cada tipo de função

possui um peso, de acordo com sua complexidade. O peso corresponde à quantidade de PF não ajustados que a função agrega à aplicação. Somando-se os pesos de todas as funções, podemos obter o total de PF não ajustados.

A Tabela 7 exibe os pesos associados às funções. Nela temos a classificação por cada tipo de função e por complexidade. Uma vez que saibamos quais são as funções existentes e sua complexidade, basta obter o seu peso, que já representa o tamanho em PF.

Função	C. Baixa	C. Média	C. Alta
ALI	7	10	15
AIE	5	7	10
EE	3	4	6
SE	4	5	7
CE	3	4	6

Tabela 7: Pesos associados as funções na contagem de PF.

Contagem de funções de dados

As funções de dados são tipicamente as primeiras funções contadas em uma aplicação. Conforme já mencionado, são divididas em ALI e AIE, correspondendo a grupos de dados que podem ser atualizados e recuperados pela aplicação.

É importante ressaltar que os agrupamentos utilizados para a contagem não estão associados à sua implementação no sistema desenvolvido. Eles devem estar associados ao significado lógico reconhecível pelo usuário.

Os ALIs são grupos de dados ou informações de controle, identificáveis pelo usuário e mantidos dentro da fronteira da aplicação. Seu objetivo principal é armazenar dados, mantidos por meio de um ou mais processos elementares da aplicação que está sendo contada.

Para a contagem de um ALI é necessário identificar o número de agrupamentos existentes (TER) e número de campos (TED).

Para se identificar ALIs é necessário procurar por grupos de dados ou informações de controle correlatos. Um ALI deve necessariamente apresentar uma unidade lógica reconhecível pelo usuário e deve ser mantido por meio de um processo elementar da aplicação.

É importante ressaltar que não devemos considerar os arquivos lógicos que o usuário não tenha acesso, tais como arquivos internos do sistema (temporários ou de trabalho).

Para se atribuir complexidade a um ALI é necessário identificar os Tipos de Elementos de Registro (TERs) e os Tipos de Elementos de

Dados (TEDs). Um TER é um subgrupo de dados dentro de um ALI/AIE reconhecível pelo usuário. Essa definição normalmente traz muitas dúvidas aos contadores, uma vez que essa definição é imprecisa. Mas de modo geral, uma ALI/AIE possui apenas um TER.

O outro elemento associado à contagem de ALI/AIE são os TEDs. Eles representam um campo único, não repetitivo e reconhecível pelo usuário, mantido ou consultado no arquivo durante a execução de uma funcionalidade. No entanto, é fundamental ressaltar que devemos considerar apenas os TEDs que são efetivamente usados pela aplicação. Assim, se um ALI possui 20 TEDs, mas são utilizados apenas 10 na aplicação, temos que considerar apenas os 10 campos. Além desses campos, é necessário considerar os dados necessários para estabelecer relacionamentos entre os ALI/AIE.

TER TED	1-19	20-50	>50
1	Baixa	Baixa	Média
2-5	Baixa	Média	Alta
>5	Média	Alta	Alta

Tabela 8: Tabela para classificação de ALI/AIE.

Os AIEs são, de forma similar aos ALIs, grupo de dados ou informações de controle, identificáveis pelo usuário e referenciados pela aplicação. No entanto, eles são mantidos na fronteira de outra aplicação. Logicamente, por conta disso, também deverão ser contados de forma diferente.

Contagem de funções de transação

As funções de transação correspondem à funcionalidade oferecida ao usuário com o intuito de permitir o processamento dos dados contidos na aplicação. Isso inclui a entrada de dados, consultas a dados pré-armazenados, além da geração de dados derivados.

Conforme já comentado, existem três tipos de funções de transação: Entrada Externa (EE), Saída Externa (SE) e Consulta Externa (CE).

Entrada Externa

Uma Entrada Externa é um processo elementar da aplicação que processam dados ou informações de controle que entram na fronteira de

aplicação, ou seja, tem como principal objetivo manter um ou mais ALIs. Isso normalmente causa uma alteração no comportamento da aplicação.

O termo “processo elementar“ é utilizado para denotar a menor unidade de atividade que é significativa para o usuário. Essa função deve sempre levar a aplicação a um estado consistente, sob a ótica do negócio.

É importante frisar que para algo ser classificado como uma Entrada Externa é necessária receber dados de fora da aplicação. Isso significa que pelo menos um ALI deve ser mantido.

Para se identificar uma EE, pelo menos uma das seguintes condições deve ser satisfeita:

1. A lógica de processamento deve ser única, diferente das lógicas executadas por outras EEs;

2. O conjunto de elementos de dados identificado é diferente daqueles identificados para outras EEs;

3. O conjunto de ALIs ou AIEs referenciados é diferente daqueles referenciados por outras EEs.

São casos típicos de uma EE a inclusão, alteração ou exclusão de itens em um cadastro, obtenção de mensagens e atualização de arquivos. Não devemos considerar entradas necessárias apenas em função da tecnologia empregada e que não contribuem em nada para o usuário, nem parte da entrada das consultas que serve apenas para direcionar a recuperação de dados como são os casos de parâmetros de pesquisa e nem telas de entrada para fins de navegação como os menus.

É necessário contar apenas os dados que cruzam a fronteira da aplicação e que fazem sentido para o usuário. Nada de contar entradas dependentes de tecnologia e imperceptíveis na visão do usuário.

Em alguns casos, existem sistemas com métodos diferentes para executar a mesma função. Eles não devem ser considerados EEs. Da mesma forma, respostas às mensagens de confirmação não cruzam a fronteira da aplicação.

Uma vez identificada uma EE, é necessário atribuir sua complexidade. Isso está diretamente ligado à identificação dos Tipos de Arquivos Referenciados (TARs) e Tipos de Elementos de Dados (TEDs). OsTARs representam o número de ALIs/AIEs mantidos ou referenciados pela EE. Os TEDs representam os campos reconhecíveis pelo usuário que cruzam a fronteira da aplicação durante a EE. Uma vez conhecidos o número de TARs e TEDs, podemos chegar à complexidade da função.

A Tabela 9 exibe a tabela para atribuição de complexidade para uma EE.

TAR TED	1-4	5-15	>15
<2	Baixa	Baixa	Média
2	Baixa	Média	Alta
>2	Média	Alta	Alta

Tabela 9: Atribuição de complexidade para EES.

Saída Externa

Uma Saída Externa (SE) é um processo elementar da aplicação que geram dados ou informação de controle para fora da fronteira da aplicação. Seu objetivo principal é apresentar informações ao usuário da aplicação, fornecendo dados que auxiliem suas atividades.

Uma SE funciona a partir de uma lógica de processamento que envolve a geração de dados derivados, sendo a diferença de uma SE de uma CE. Uma SE envolve dados cuja obtenção requer algum tipo de processamento além da recuperação de campos de ALIs. Se isso não ocorrer, não se trata de uma SE.

Para a identificação de uma SE, é necessário procurar por dados ou informações de controle que devem sair da fronteira da aplicação. Ressaltando mais uma vez, a lógica de processamento de uma SE deve:

1. Conter pelo menos um cálculo ou fórmula matemática;
2. Criar dados derivados;
3. Manter pelo menos um ALI;
4. Alterar o comportamento da aplicação.

Outro detalhe importante é que a lógica de processamento utilizada na SE deve ser única, diferente daquela executada por outras SEs. Da mesma forma, o conjunto de elementos de dados identificados deve ser diferente daqueles identificados para outras SEs, assim como o conjunto de ALIs ou AIEs referenciados também devem ser diferentes daqueles referenciados por outras SEs. Se isso não se configurar, provavelmente temos uma única SE que possui parâmetros diferentes e que não deveria ser contada como outra SE. São exemplos típicos de SEs:

1. Relatórios que consolidam dados através de cálculos;
2. Telas de exibição de dados que possuam dados derivados;
3. Exibição de gráficos com dados calculados;
4. Transferência de dados, arquivos ou mensagens enviadas a outra aplicação, quando isso envolve geração de dados derivados ou atualização de arquivos.

É muito importante que não seja considerado como SEs

relatórios idênticos, com valores de dados diferentes. Uma mudança em um parâmetro, seja a introdução de um parâmetro adicional, não representa uma SE diferente. Da mesma forma, mensagens de erro ou validação, que fazem parte de outro processo elementar, não podem ser consideradas SEs. Assim, se no processo de inclusão de um elemento em uma aplicação, for necessário fazer uma validação que gere um dado calculado, que será apresentado e demonstrado como uma mensagem ao usuário, isso não pode ser considerado como uma SE.

Arquivos enviados a outras aplicações que não envolvam dados derivados ou manutenção de arquivos, também não podem ser consideradas SEs.

De forma similar a EE, a atribuição de complexidade a uma SE se dá a partir da identificação dos TARs e TEDs contidos na função. A regra também é bem parecida:

1. Contar um TAR para cada ALI ou AIE lido durante o processamento da SE de acordo com os requisitos do usuário;
2. Contar um TAR para cada ALI mantido pelo processo elementar da SE, lembrando que cada ALI conta uma vez, e não cada TER de um ALI;
3. Contar apenas uma vez os ALIs que são tanto lidos quanto mantidos;
4. Contar um TED para cada campo não repetitivo reconhecido pelo usuário, seja ele um campo que entra na fronteira da aplicação ou que saia da fronteira da aplicação. Se um mesmo TED entra e sai da fronteira da aplicação, ele deve ser contado apenas uma vez.

Alguns detalhes especiais com relação à contagem de TEDs devem ser especificados. Não devemos contar:

1. Numeração de páginas;
2. Campo de data/hora de emissão de relatório;
3. Literais (strings);
4. Títulos fixos, nomes de campos, títulos de colunas, etc;
5. Campos de ALI's mantidos ou consultados, que não cruzem a fronteira da aplicação;
6. Campos múltiplos que na visão do usuário tenham um significado único, como por exemplo, múltiplas linhas de um campo de endereço.

A Tabela 10 exibe a tabela para atribuição de complexidade para uma SE.

TAR TED	1-5	6-19	>19
<2	Baixa	Baixa	Média
2-3	Baixa	Média	Alta
>3	Média	Alta	Alta

Tabela 10: Atribuição de complexidade para SES.

Consulta Externa

Uma Consulta Externa (CE) é um processo elementar que recuperam dados ou informação de controle para fora da fronteira da aplicação. Seu objetivo principal é apresentar informações ao usuário, a partir da simples recuperação de dados de ALIs e/ou AIEs.

Um CE não envolve cálculos nem manutenção de ALIs. No entanto, é fundamental que existam dados ou informações de controle que saiam da fronteira da aplicação, recuperados de um ou mais ALIs ou AIEs associados à função.

A lógica de processamento envolvida em uma CE não pode conter cálculos ou fórmulas matemáticas, nem a geração de dados derivados ou a manutenção de ALIs.

O CE também segue as mesmas restrições associadas a uma SE:

1. A lógica de processamento é única, diferente daquela executada por outras CEs;

2. O conjunto de elementos de dados identificado é diferente daqueles identificados para outras CEs;

3. O conjunto de ALIs ou AIEs referenciados é diferente daqueles referenciados por outras CEs.

São exemplos típicos de CEs relatórios que exibem dados contidos em ALIs ou AIEs, sem cálculos, telas de exibição de dados que não possuam dados derivados e transferência de dados, arquivos ou mensagens enviadas a outra aplicação, sem que haja dados derivados ou atualização de ALIs.

Existem algumas características adicionais na contagem de CEs. São também classificados como CEs:

1. Telas de log-on para autenticação de usuários;

2. Ajuda on line. Nesse caso, cada nível de ajuda (tela, campo) conta como uma CE distinta;

3. Consultas do tipo lista e combo, normalmente utilizadas para facilitar o preenchimento de campos em outros processos elementares.

Também de forma similar às SEs, não devem ser consideradas

CEs relatórios idênticos com valores de dados diferentes, mensagens de erro ou confirmação que fazem parte de outro processo elementar e arquivos enviados a outras aplicações que envolvam dados derivados ou manutenção de arquivos.

A contagem também é feita de forma similar a uma SE: devem ser contado um TAR para cada ALI ou AIE lido durante o processamento da CE. A contagem para TEDs é similar ao de uma SE. A Tabela 11 exibe a tabela para atribuição de complexidade para uma CE.

TAR TED	1-5	6-19	>19
<2	Baixa	Baixa	Média
2-3	Baixa	Média	Alta
>3	Média	Alta	Alta

Tabela 11: Atribuição de complexidade para CES.

Contagem Estimativa de Pontos de Função

Nesta seção explicaremos como proceder com uma contagem estimativa de pontos de função. É importante ressaltar que uma contagem estimativa não tem como objetivo ir a fundo às regras e detalhes da contagem. Ela serve como uma aproximação do valor exato, para fins gerenciais, mas sem ser tão rigorosa quanto ao processo utilizado para obtenção desse valor.

Nosso objetivo neste capítulo não é tornar o leitor um especialista em APF, mas sim um conhecedor da técnica. Para se tornar um especialista será necessário muito mais investimento no estudo do manual do IFPUG, bem como na prática de contagem.

A Figura 43 exibe um resumo do roteiro para contagem estimativa em um projeto de software. De posse do modelo de dados para o projeto, normalmente contido nos diagramas UML que descrevem a aplicação, é possível realizar uma contagem das funções de dados existentes. Da mesma forma, com a descrição das funções, normalmente contidas nas descrições dos casos de uso que compõem o projeto, é possível contar as funções de transação. Assim, a Especificação de Requisitos, juntamente com o Modelo de Análise possuem todas as informações necessárias para procedermos com uma contagem.

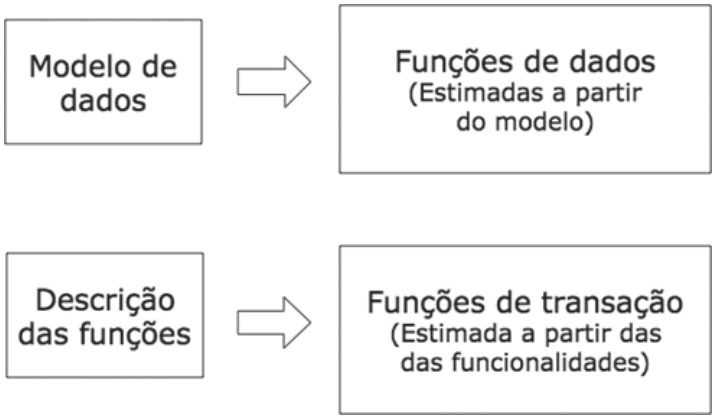


Figura 43: Roteiro para contagem estimativa em um projeto.

Agora vamos proceder com a contagem estimativa para o nosso exemplo. É importante ressaltar que em uma contagem, normalmente surgem diversas dúvidas. Precisamos anotar nossas decisões para que qualquer pessoa que tente realizar uma contagem possa chegar aos mesmos resultados.

Neste trabalho não discutimos nem utilizaremos o fator de ajuste, que é uma medida que pode influenciar em 35% do tamanho do sistema (para mais ou para menos), devido a algumas características a serem avaliadas. Como o fator de ajuste está muito defasado, por conta das características avaliadas, utilizaremos apenas o valor bruto calculado. Em uma contagem tradicional esse valor deveria ser multiplicado pelo fator de ajuste, para obtermos o cálculo em pontos de função ajustados.

Contagem de funções de dados

Esta seção apresenta um exemplo de contagem estimativa para funções de dados. Você deverá fazer o mesmo para um exemplo escolhido por você.

Nossa contagem se inicia pelas funções de dados. Nosso sistema exemplo, mesmo não sendo complexo, possui diversas classes. Temos que proceder com a contagem desses elementos. Utilizaremos como base o diagrama de classes exibido na Figura 36.

Iniciando pela classe Projeto, podemos perceber que ela é um ALI contendo 9 TEDs. Ela não possui nem um agrupamento interno, devendo contá-la como um ALI com apenas um TER. De acordo com a Tabela 8, Projeto é uma ALI simples.

Continuando pela classe Membro, começamos a decidir algumas questões para proceder com a contagem. Conforme pode ser visto na Figura 36 a classe Membro herda de Usuário, que por sua vez tem uma relação com Alteração. De um modo geral, um usuário da aplicação não perceberá essa divisão interna. Assim, essas 3 classes juntas

representam um único ALI que possui 2 TERs (e não 3, pois Usuário e Membro representam apenas um agrupamento). Como todo membro deve ter as informações de usuário, isso representa um único TER, ao invés de 2. A classe alteração representa outro TER. Com isso, temos que Membro é um ALI que possui 2 TERs. Com relação à quantidade de campo, possuímos as seguintes quantidades:

1. Membro possui 4 campos;
2. Usuário possui 2 campos;
3. Alteração possui 1 campo.

Nossa contagem levará em consideração 10 campos para Membro, contabilizando os 4 visíveis diretamente, 2 de usuário e 2 campos adicionais para permitir seus relacionamentos com projeto, além de 2 outros campos para a classe Revisão. Com isso teremos 7 TEDs. Com isso, a complexidade do ALI Membro pode ser considerada baixa.

A classe Critério representa um ALI simples, sem agrupamentos. Ela possui 3 campos, porém, devemos considerar um campo adicional para o relacionamento com projeto. Assim, a classe critério tem complexidade baixa.

A classe Requisito é um ALI que possui 2 TERs. Consideramos isso por conta do relacionamento com Alteração. Requisito possui 7 campos, mas consideraremos 8 por conta do relacionamento com Projeto. Alteração possui um campo, mas consideraremos 2 por conta do relacionamento com Requisito. Assim, temos Requisito como sendo um ALI com 2 TERs e 10 TEDs.

A classe Ator representa um ALI simples, contendo um agrupamento para alterações. Ela possui 2 campos, porém devemos considerar um campo adicional para o relacionamento com Caso de uso. Alteração possui 1 campo, mas vamos considerar 2 por conta do agrupamento com Ator. Assim, a classe Ator é uma ALI com 2 TERs e 5 TEDs com complexidade baixa.

A classe Caso de uso é provavelmente o item mais complexo do exemplo. Ela possui diversos TERs:

1. Caso de uso com 6 campos;
2. Alteração com 1 campo;
3. Protótipo com 2 campos;
4. Fluxos com 2 campos;
5. Fluxos alternativos com 1 campo.

No total, Caso de uso é um ALI com 5 TERs que possui 18 campos, pois foram contabilizados 8 campos para a classe caso de uso

(2 adicionais para o relacionamento com requisitos e atores), 2 campos para Alteração (1 campo adicional para o relacionamento com Caso de uso), 3 campos para Protótipo (1 campo adicional para o relacionamento com Caso de uso), 3 campos para Fluxo (1 campo adicional para o relacionamento com Caso de uso) e 2 campos para Fluxo Alternativa (1 campo adicional para o relacionamento com Caso de uso). Assim, Caso de uso é considerado um ALI com complexidade baixa. Observe que embora bem mais complexo que os demais ALIs analisados aqui, ele ainda possui a mesma complexidade. Essa é uma das críticas a contagem em PF.

Finalizando, a classe Revisão possui um agrupamento para a avaliação, denominada Item de revisão. A classe Revisão possui 4 campos e Item de revisão possui 2 campos. No entanto, existem 3 relacionamentos adicionais, que Revisão é responsável (2 com Membro e um com Projeto). Item de revisão possui 4 campos adicionais, por conta dos relacionamentos com Revisão, Critério, Requisito e Caso de uso, contabilizando 4 campos adicionais. Assim, Revisão é um ALI com 2 TERs e 13 TEDs. Com isso, Revisão também possui complexidade baixa.

A Tabela 12 resume a atribuição de complexidade para os elementos presentes em nosso exemplo.

Seq.	Nome	ALI	TER	TED	Complexidade
1	Projeto	1	1	9	Baixa
2	Membro (Usuário, Alteração)	1	2	10	Baixa
3	Requisito (Alteração)	1	2	10	Baixa
4	Ator (Alteração)	1	2	5	Baixa
5	Caso de uso (Fluxo, Fluxo Alternativo, Protótipo, Alteração)	1	5	18	Baixa
6	Revisão (item de revisão)	1	2	13	Baixa

Tabela 12: Resumo das complexidades para os ALI do exemplo.

Cada ALI de complexidade baixa conta 7 PF, conforme descrito na Tabela 7. Assim, temos 6 ALIs simples, que nos resulta em 6 x 7 = 42 pontos de função de dados.

Contagem de funções de transação

Agora procederemos com a contagem das funções de transação. Vale ressaltar que uma contagem detalhada deve levar em consideração todas as regras, incluindo campos chaves, comandos, ajuda, previsão de relacionamentos, etc. Por conta disso, faremos uma contagem bastante

simplificada. Utilizaremos com base para contagem de TEDs apenas os campos visíveis aos usuários, esquecendo-se de outros detalhes da contagem. Assim, contaremos os ALIs/AIEs referenciados na função (que serão os TARs) e os TEDs envolvidos. Depois disso, faremos a atribuição de complexidade baseado nas tabelas de atribuição de complexidade para EE, SE e CE (Tabela 9, Tabela 10, Tabela 11).

De forma similar à seção anterior, apresentamos um exemplo de contagem estimativa para funções de transação. Você deverá fazer o mesmo para um exemplo escolhido por você.

A contagem das funções de transação vai seguir a ordem dos casos de uso na Especificação de Requisitos do ReqG. O primeiro caso de uso é o Cadastro de Projeto. A pesquisa por projeto faz com que 2 campos cruzem a fronteira do sistema. Esses campos são de dois ALIs diferentes: Projeto e Membro (gerente). Assim, a pesquisa é uma CE que possui 2 TARs e 2 TEDs referenciados. Por conta disso, ela é uma CE de complexidade baixa, conforme Tabela 11. Ainda no Cadastro de Projeto é possível visualizar os dados, exibindo os três campos relacionados (sigla, descrição e gerente). A visualização também possui 2 TARs (Projeto e Gerente) e 3 TEDs. Ela também é uma CE de complexidade baixa. A alteração é uma EE que possui 2 TARs e 3 TEDs. Ela é uma EE de complexidade baixa. A exclusão é um caso diferenciado. Para a exclusão, normalmente apenas o identificador do elemento a ser excluído cruza a fronteira da aplicação. Por conta disso, utilizaremos sempre nas exclusões o padrão de termos 1 TAR e 1 TED. Isso funciona dessa forma na contagem detalhada, porém existem algumas outras regras envolvidas.

O segundo caso de uso considerado é Gestão de Gerentes. Podemos realizar uma pesquisa, a visualização dos dados de um gerente, uma inclusão, alteração e exclusão. A pesquisa é uma CE que referencia dois ALIs: Gerente e Projeto. Além disso, a pesquisa exibe 4 campos. O restante das funções lida apenas com o ALI Gerente. A visualização, inclusão e alteração lidam com 5 campos. A exclusão, conforme nossa padronização, trata de apenas 1 campo.

O próximo caso de uso, Gestão de Membros, é parecido com o caso de uso Gestão de Gerentes. Podemos realizar uma pesquisa, a visualização dos dados de um membro, uma inclusão, alteração e exclusão. Todas as funções referenciam 2 ALIs: Membro e Projeto. O restante é contar os campos referenciados e atribuir a complexidade.

O caso de uso Controle de Projetos possui apenas 3 funções: pesquisa, alteração e visualização. A pesquisa é um CE que referencia 2 ALIs, Projeto e Membro. A visualização e alteração também referenciam 2 ALIs, mas exibem vários campos. Observe que alguns campos aparecem

2 vezes na tela, por isso não são contados duas vezes. São exemplos disso o nome do projeto e nome de um membro, que aparecem tanto nos dados do projeto, como também nos dados dos Membros.

O caso de uso Gestão de Requisitos é um CRUD (acrônimo para Create, Read, Update, Delete) que referencia 3 ALIs: Projeto, Requisito e Membro. No entanto, na pesquisa são referenciados apenas 2 ALIs.

O caso de uso Gestão de Atores também é um CRUD. Ele possui 3 ALIs referenciados: Projeto, Caso de Uso e Ator. A pesquisa referencia 4 campos, enquanto que as outras funções (com exceção da exclusão) referenciam 8 campos.

A Gestão de Casos de Uso possui muitos ALIs referenciados. Na pesquisa são referenciados 3 ALIs: Caso de uso, Requisito e Projeto. Nas demais funções ainda têm referência a Membro e Ator. Observe que existem campos na tela utilizados para entrada de dados, mas que não cruzam a fronteira do sistema. Um exemplo disso é a tabela Dados do Fluxo Alternativo. Os campos são usados para criar a lista de Fluxos Alternativos da tela. Essa lista é enviada e cruza a fronteira da aplicação. Assim, não devemos contar os campos que não cruzam a fronteira da aplicação.

A Gestão de Critérios é um caso de uso simples que referencia os ALIs Projeto e Critério, além de usar 4 campos.

O caso de uso Revisão possui 5 funções de transação: pesquisa, visualização, inclusão, alteração e reabertura. A pesquisa referencia 2 ALIs: Revisão e Projeto. As funções de visualização, inclusão e alteração referenciam Projeto, Revisão, Critério, Requisitos, Caso de uso e Membro. A função Reabertura apenas altera um campo de Revisão, indicando se ela foi fechada ou não.

A Geração da Especificação possui duas funções de transação: pesquisa e geração. A pesquisa referencia 2 ALIs: Projeto e Membro. A geração de especificação é uma CE que referencia todos os ALIs, com exceção de Revisão e Critério. São 5 ALIs. Todos os campos desses ALIs são referenciados na função.

O Relatório de acompanhamento possui duas funções: uma CE relacionada à pesquisa e uma SE relacionada à geração do relatório de acompanhamento. A pesquisa referencia 2 ALIs: Projeto e Membro. A SE referencia os ALIs Projeto, Membro, Requisito e Caso de uso. Existem campos adicionais para informar o percentual de conclusão do requisito, do caso de uso e do projeto como um todo. Esses são campos adicionais do relatório precisam ser contabilizados. Por isso a contagem referencia 14 campos: são 11 campos normais e 3 calculados.

A Tabela 13 exibe um resumo da contagem realizada, detalhando as

funções identificadas, número de TARs e TEDs envolvidos, complexidade identificada e valor da função em Pontos de Função.

É importante frisar que devemos ter um cuidado especial com o número de ALIs referenciados nas funções. Se existir pelo menos um campo de um ALI envolvido na função, então esse ALI é referenciado. Além disso, temos que contar o número de campos associados à função. Conforme comentamos, utilizamos uma contagem simplificada: contamos apenas os campos visíveis. Não trataremos aqui de outros detalhes como previsão de comandos, mensagens, relacionamentos entre elementos, etc. Isso simplifica o processo de entendimento da APF. No entanto, frisamos que existem muitos detalhes a serem considerados em uma contagem detalhada.

Seq.	Nome	Tipo	ALI	TED	Complexidade	PF
1	Cadastro de Projetos - Pesquisa	CE	2	2	Baixa	3
2	Cadastro de Projetos - Visualização	CE	2	3	Baixa	3
3	Cadastro de Projetos - Inclusão	EE	2	3	Baixa	3
4	Cadastro de Projetos - Alteração	EE	2	3	Baixa	3
5	Cadastro de Projetos - Exclusão	EE	1	1	Baixa	3
6	Gestão de Gerentes - Pesquisa	CE	2	4	Baixa	3
7	Gestão de Gerentes - Visualização	CE	1	5	Baixa	3
8	Gestão de Gerentes - Inclusão	EE	1	5	Baixa	3
9	Gestão de Gerentes - Alteração	EE	1	5	Baixa	3
10	Gestão de Gerentes - Exclusão	EE	1	1	Baixa	3
11	Gestão de Membros - Pesquisa	CE	2	5	Baixa	3
12	Gestão de Membros - Visualização	CE	2	7	Média	4
13	Gestão de Membros - Inclusão	EE	2	7	Média	4
14	Gestão de Membros - Alteração	EE	2	7	Média	4
15	Gestão de Membros - Exclusão	EE	1	1	Baixa	3
16	Controle de Projetos - Pesquisa	CE	2	2	Baixa	3
17	Controle de Projetos - Visualização	CE	2	12	Média	6
18	Controle de Projetos - Alteração	EE	2	12	Média	6
19	Gestão de Requisitos - Pesquisa	CE	2	4	Baixa	3
20	Gestão de Requisitos - Visualização	CE	3	12	Média	4
21	Gestão de Requisitos - Inclusão	EE	3	12	Alta	6
22	Gestão de Requisitos - Alteração	EE	3	12	Alta	6
23	Gestão de Requisitos - Exclusão	EE	1	1	Baixa	3
24	Gestão de Atores - Pesquisa	CE	3	4	Baixa	3
25	Gestão de Atores - Visualização	CE	3	8	Média	4
26	Gestão de Atores - Inclusão	EE	3	8	Alta	6
27	Gestão de Atores - Alteração	EE	3	8	Alta	6
28	Gestão de Atores - Exclusão	EE	1	1	Baixa	3
29	Gestão de Casos de uso - Pesquisa	CE	3	4	Baixa	3
30	Gestão de Casos de uso - Visualização	CE	5	20	Alta	6
31	Gestão de Casos de uso - Inclusão	EE	5	20	Alta	6
32	Gestão de Casos de uso - Alteração	EE	5	20	Alta	6
33	Gestão de Casos de uso - Exclusão	EE	1	1	Baixa	3
34	Gestão de Critérios - Pesquisa	CE	2	4	Baixa	3
35	Gestão de Critérios - Visualização	CE	2	4	Baixa	3
36	Gestão de Critérios - Inclusão	EE	2	4	Baixa	3
37	Gestão de Critérios - Alteração	EE	2	4	Baixa	3
38	Gestão de Critérios - Exclusão	EE	1	1	Baixa	3
39	Revisão - Pesquisa	CE	2	3	Baixa	3
40	Revisão - Visualização	CE	6	10	Alta	6
41	Revisão - Inclusão	EE	6	10	Alta	6
42	Revisão - Alteração	EE	6	10	Alta	6
43	Revisão - Reabertura	EE	1	1	Baixa	3
44	Geração da Especificação - Pesquisa	CE	2	3	Baixa	3
45	Geração da Especificação - Geração	CE	5	33	Alta	6
46	Relatório de Acompanhamento - Pesquisa	CE	2	3	Baixa	3
47	Relatório de Acompanhamento - Relatório	SE	4	14	Alta	7
TOTAL						189

Tabela 13: Resumo da contagem de funções de transação.

Planejamento e Gestão de Projetos com PF

Pontos de Função podem ser utilizados para auxiliar a estimativa de esforço necessário ao desenvolvimento ou evolução de um produto. Isso pode ser feito a partir do cálculo de produtividade associada a um projeto e sua equipe.

O uso de APF permite o acompanhamento da produtividade, bem como seu uso para a realização de estimativas de custo e esforço.

A produtividade é uma medida que indica a capacidade de produção. No caso de software, ela indica a capacidade de produzir software por unidade de tempo. Uma função para produtividade, baseada em pontos de função é: Produtividade = PF/PM (ponto de função por pessoa-mês). Mas como calcular isso? Basta contar quantos pontos de função foram desenvolvidos por uma equipe em um mês de trabalho. Se descobrirmos isso, podemos usar para fazer estimativas.

De posse da produtividade de uma pessoa ou de uma equipe, é fácil obter uma estimativa de tempo para realização do desenvolvimento de um sistema. Por exemplo, se uma equipe tem média de 200 PF/mês, um software de 600 PF poderia ser construído em 3 meses.

No entanto, existem diversos fatores que podem influenciar os dados de produtividade. As diferenças entre linguagens utilizadas, tecnologias e capacitação da equipe podem influenciar muito os dados de produtividade. Imagine por exemplo programadores COBOL e programadores que utilizem Delphi. Certamente a produtividade de programadores Delphi deve ser bem superior a de programadores COBOL, uma vez que Delphi possui elementos que favorecem a produtividade em relação ao COBOL.

Na verdade, uma mesma equipe pode ter produtividade bastante diferente de um projeto para outro. As pessoas podem perder produtividade quando estão com problemas pessoais.

Outro dado interessante é que o tamanho dos projetos também pode influenciar na produtividade. Projetos muito grandes tendem a ter produtividade menor que projetos pequenos. A explicação para isso é simples: quando um projeto é muito grande, coisas simples podem crescer em dimensão, gerando um problema. Um exemplo é a comunicação: grandes equipes têm muitos problemas para uma reunião, por exemplo. Pequenas equipes resolvem isso facilmente.

Outros fatores que podem influenciar no cálculo de produtividade são as coletas de medidas não padronizadas ou não confiáveis. Se isso ocorrer, as estimativas podem ser muito imprecisas. Outro ponto importante é que as medidas sejam feitas utilizando produtos que tenham sua qualidade controlada. Uma empresa pode gerar um sistema

rapidamente e com isso ter uma produtividade alta. Porém, se a qualidade dele não é medida, podem ser descobertos tantos erros que não justifiquem considerar o produto concluído. Assim, é fundamental o controle da qualidade para que tenhamos uma medida de produtividade efetiva.

Normalmente, a obtenção de dados de produtividade segue alguns passos. Inicialmente, é preciso identificar projetos similares, uma vez que a produtividade pode ser dependente do que se faz. Outro ponto importante é caracterizar o tamanho, linguagem, aplicação, experiência da equipe e principalmente o processo usado para guiar o trabalho. Com isso feito, pode-se obter a produtividade para cada projeto.

Assim, pontos de função é uma boa métrica para ajudar no cálculo de produtividade. No entanto, é necessário ter cuidado com as medições para não gerarmos dados inconfiáveis.

Visão crítica da APF

A APF possui muitas vantagens e desvantagens associadas. É importante conhecer cada um desses pontos para que tenhamos subsídios suficientes para decidir ou não pelo seu uso.

As vantagens do PF são:

1. A especificação de requisitos é suficiente para o cálculo. Essa característica é muito interessante para o uso do APF em projetos. Com a ER de um produto, podemos estimar seu tamanho e assim estimar custo e esforço para seu desenvolvimento.

2. Método de grande difusão. APF é utilizado no mundo inteiro. Além disso, mesmo estando defasado em alguns pontos, ainda possui muitos usuários. Além disso, em projetos de terceirização no Brasil é indicado como métrica para cálculo da produção e definição de pagamentos.

3. Muitos dados publicados. Existem muitos dados de empresas relativos ao uso de APF. Esses dados podem até ser adquiridos por outras empresas. São bases históricas de projetos com diversas informações disponíveis que podem ser utilizadas por outras organizações para ajudar em suas estimativas.

4. Podem ser usados para comparações. Com o uso do APF podemos comparar a efetividade de tecnologias diferentes (Java x Delphi), além de permitir a comparação de empresas (A x B), analisando quem produz mais em menos tempo.

5. Grupo internacional de usuários (IFPUG). O grupo de usuários de APF é grande, existente em boa parte do mundo e bastante atuante.

Questões enviadas ao grupo são respondidas em pouco tempo.

6. Fórum para troca de informação. Existem muitos fóruns para eliminação de dúvidas associados com respostas efetivas e em tempos reduzidos.

No entanto, mesmo tendo essas vantagens, existem diversas desvantagens associadas.

1. Não têm nenhuma interpretação concreta. Não faz parte do nosso dia-a-dia o uso de APF. Assim, dizer que algo tem 200PF ou 600PF não representa uma informação associada. Não é a mesma coisa de dizer que uma casa tem 1.000m2. No entanto, apenas difundindo ainda mais o seu uso é que essa medida pode ter uma interpretação concreta.

2. Contagem não pode ser facilmente automatizada. Existe muita interpretação associada à contagem de PF. Isso dificulta sua automação por completo, visto que essas interpretações são difíceis de serem automatizadas por completa.

3. Existem variantes do método. Existem algumas variações do método que utilizam características diferentes de contagem. Isso pode gerar diferenças em trabalhos e contradições. Existem valores diferentes para os pesos usados, além de regras diferentes de contagem.

4. São orientados para sistemas de informação. APF foi criada para contagem de sistemas de informação, sendo apropriadas para esses casos. No entanto, seu uso para contagem de tamanho de jogos, aplicações para celulares, recuperação da informação, não são adequadas. Assim, qualquer aplicação em contextos diferentes requer adaptações para os produtos a serem contados.

A APF possui várias vantagens e desvantagens associadas. No entanto, ela ainda é uma forma muito utilizada no mundo.

5. A contagem rigorosa é difícil. Existem muitas regras associadas. Por conta disso fizemos uma contagem bastante simplificada. As regras são muito extensas e difíceis de serem seguidas em detalhes.

6. Requer profissional especializado. Uma contagem detalhada exige um profissional bastante especializado, que é caro e difícil de ser encontrado no mercado de trabalho.

AAPF ainda possui algumas limitações que devem ser consideradas em qualquer projeto de contagem:

1. Não leva em conta a reutilização. A APF não possui um mecanismo de contabilizar o reuso de partes de um projeto, visto que isso pode reduzir o esforço necessário para sua construção. Não é previsto o reuso de componentes, de código legado, de código herdado de versões anteriores.

2. É difícil seu uso para contagem de alterações de requisitos.

3. APF é inadequada para contar a complexidade de ações de manutenção corretiva, pois é difícil expressar as diferenças de uma versão para outra.

De modo geral, APF possui vantagens e desvantagens, mas mesmo assim é um método bastante utilizado. De qualquer forma, independente de utilizar APF ou outro método, é importante as medidas para controlar nosso trabalho. Como foi dito pelo famoso escritor Tom DeMarco: “Não se consegue controlar o que não se consegue medir”.

Exercícios Propostos

1. O que é a mensuração em software?

2. O que pode ser medido em um software?

3. O que é a produtividade?

4. Quais são as características de uma boa medida de tamanho?

5. Por que linhas de código não são utilizadas como medida para controlar estimativas de desenvolvimento?

6. O que é APF?

7. Quando surgiu a APF?

8. Quais foram os dois objetivos principais da APF?

9. Quais são os cinco elementos considerados em uma contagem? Explique cada um deles.

10. Quais são os passos incluídos no procedimento de contagem de PF?

11. Quais são os 3 tipos de contagem existente? Explique cada uma.

12. O que é o escopo da contagem em PF?

13. Quais são as duas categorias de funções consideradas em um cálculo de PF?

14. O que é um ALI? E um AIE? Qual a diferença entre eles?

15. Quais são os 3 tipos de funções de transação?

16. O que pode influenciar a contagem de um ALI? Explique esses elementos.

17. Quais campos devem ser considerados em uma tela para a contagem do tamanho de uma EE?

18. Dê exemplos de SE e CE ressaltando a diferença entre os dois.

19. O que pode ser usado como insulo para a contagem das funções de dados em um projeto? O que pode ser usado para contar as funções de transação?

20. Realize uma contagem de funções de dados de um projeto diferente do exemplo da apostila, explicando as decisões existentes e detalhando os números associados, de forma similar ao exemplo detalhado.

21. De forma similar a questão anterior, realize uma contagem de funções de transação de um projeto diferente do exemplo da apostila, explicando as decisões existentes e detalhando os números associados de forma similar ao exemplo detalhado.

22. Como podemos utilizar APF para estimar um projeto?

23. Cite vantagens e desvantagens do uso de APF.

REFERÊNCIAS DA WEB

Página da Universidade Aberta do Piauí - UAPI
http://www.ufpi.br/uapi

Página da Universidade Aberta do Brasil- UAB
http://www.uab.gov.br

Sítio de apoio ao livro de Wilson de Pádua
http://homepages.dcc.ufmg.br/~wilson/

Sítio oficial da UML
http://www.uml.org/

International Function Points User Group
http://www.ifpug.org/

Brazilian Function Points User Group
http://www.bfpug.com.br/

UNIDADE 3
Exemplo de Artefatos

Resumo

Nesta unidade apresentamos exemplos completos dos artefatos utilizados para o levantamento de requisitos, registro de uma reunião e para revisão de uma ER ou de um modelo de análise. Eles são exemplos completos para o sistema exemplo abordado no trabalho.

3 Exemplo de Especificação de Requisitos

ANEXO

Fictícia Soluções em Informática

Especificação dos Requisitos

do Software ReqG 1.0

Autores: Equipe Fictícia

Teresina - PI
Dezembro de 2010

Aprovação

Aprovamos o documento de Especificação de Requisitos do projeto ReqG 1.0.

RESPONSÁVEL	INSTITUIÇÃO	DATA	ASSINATURA
Gildásio Guedes Fernandes	UAPI	08/12/2010	
Luís Cláudio Demes da Mata Souza	UAPI	08/12/2010	
Pedro de Alcântara dos S. Neto	FICTÍCIA	08/12/2010	

Versões revisadas anteriores

RESPONSÁVEL	VERSÃO	DATA	DESCRIÇÃO
Pedro de Alcântara dos S. Neto	1.0	07/12/2010	Finalização da versão inicial.

Especificação dos Requisitos do Software Gerenciador de Requisitos 1.0

Sumário

Introdução 119
Nome do produto 119
Escopo do produto 119
Limites do produto 119
Definições e siglas 120
Definição dos Requisitos 120
Requisitos Funcionais 142
Requisitos Não-Funcionais 121
Detalhamento dos Requisitos 121
Diagrama de contexto 121
Casos de uso 122
Atores 123

Especificação dos Casos De Uso 123
Cadastro de Projeto..... 123
Gestão de Gerentes 125
Gestão de Membros 127
Controle de Projetos..... 130
Gestão de Requisitos 131
Gestão de Atores..... 134
Gestão de Casos de Uso 135
Gestão de Critérios..... 139
Revisão..... 141
Geração da Especificação..... 145
Relatório de Acompanhamento 146

Introdução

Nome do produto

ReqG: Sistema Gerenciador de Requisitos de Software.

Escopo do produto

Gerenciar os requisitos de um projeto de software via Web, registrando todos os dados associados à sua definição, de forma a cobrir todas as atividades previstas em um fluxo de requisitos.

Limites do produto

1. O ReqG não controlará a execução de tarefas no projeto, apenas registrará os dados relacionados aos requisitos.
2. O ReqG não gerará documentos editáveis com a especificação de requisitos. Os dados ficam registrados no sistema e só podem ser alterados por ele.
3. O ReqG não controlará qualquer aspecto do custo ou do cronograma de execução.
4. O backup e a recuperação das bases de dados do sistema ficam a cargo da administração de dados e não serão providas pelo ReqG.
5. O ReqG não terá ajuda on line.

Definições e siglas

Nr.	Sigla	Definição
1.	JSF	Java Server Faces. Tecnologia para construção da camada de apresentação de um sistema Web.
2.	Hibernate	Framework para persistência de dados em Java.
3.	MySQL	Banco de dados livre bastante popular no mundo.

Definição dos Requisitos

Requisitos Funcionais

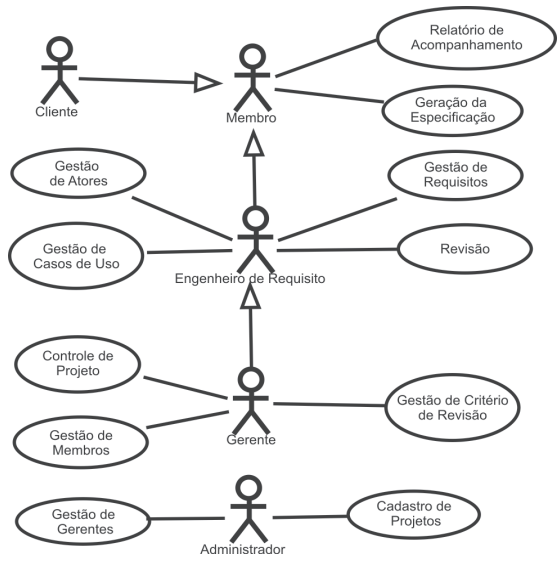
ID	Requisito	Descrição
RF1	Requisitos	O sistema deve permitir cadastrar e controlar todos os aspectos relacionados aos requisitos de um projeto, permitindo visualizar isso e acompanhar sua evolução, incluindo as pessoas que trabalharam no projeto, os analistas e gerente do projeto.
RF2	Casos de uso	O sistema deve possibilitar a especificação dos casos de uso, registrando sua descrição, atores, protótipos de tela associados, relacionando aos requisitos que deram origem ao caso de uso.
RF3	Revisão	Deve ser possível realizar uma revisão dos requisitos e casos de uso, utilizando critérios definidos pelos próprios gerentes de projetos, de forma independente aos demais projetos.
RF4	Acompanhamento dos clientes	Deve ser possível que os clientes possam acompanhar a evolução do projeto a qualquer momento, consultando tudo o que foi feito.
RF5	Liberação de acesso por projeto	Deve ser possível liberar o acesso ao sistema por projeto, indicando o seu gerente. Assim, para se ter acesso ao sistema, deverá ser adquirido uma licença para um projeto. A partir disso, o gerente ficará responsável por definir quem deverá usar o sistema, seja para trabalhar na especificação de requisitos como um dos Engenheiros de Requisitos, seja como cliente consultando o resultado do trabalho realizado.
RF6	Geração da documentação	Deve ser possível gerar a especificação na forma de um documento, contendo todos os dados registrados do projeto.

Requisitos Não-Funcionais

ID	Requisito	Descrição
RNF1	Ambiente	O sistema deve funcionar em ambiente Web, sendo compatível com os principais navegadores do momento: Internet Explorer, Firefox, Safari e Chroma.
RNF2	Linguagem	O sistema deve ser desenvolvido utilizando-se a linguagem Java com as tecnologias JSF e Hibernate.
RNF3	Banco de dados	O sistema deve utilizar o banco de dados MySQL.
RNF4	Desempenho	O sistema deve ser construído para funcionar com 100 usuários simultâneos com respostas de até 5s quando utilizado em rede local.
RNF5	Segurança	O sistema deve restringir o acesso por meio de senhas. Deve-se ter um controle no registro de senha de forma a impedir o uso de senhas consideradas fáceis.
RNF6	Usabilidade	Um usuário com conhecimento básico em informática e conhecimento nos conceitos de requisitos deveria ser capaz de operar o sistema com um curso de 30 minutos.

Detalhamento dos Requisitos

Diagrama de contexto



Casos de uso

ID	Caso de uso	Requisito associado	Descrição
UC1	Gestão de Requisitos	RF1	Cadastro de requisitos funcionais e não funcionais associados ao projeto.
UC2	Gestão de Membros	RF1	Cadastro de todos os envolvidos no projeto.
UC3	Gestão de Casos de Uso	RF2	Definição dos casos de uso do projeto.
UC4	Gestão de Atores	RF2	Definição dos atores que irão interagir no projeto.
UC5	Revisão	RF3	Revisão de um requisito ou de um caso de uso, observando os critérios pré-estabelecidos no projeto para a revisão.
UC6	Gestão de Critérios de Revisão	RF3	Cadastro de critérios a serem utilizados em uma revisão.
UC7	Cadastro de Projeto	RF5	Cadastro de um projeto com definição do seu gerente feito pelo administrador do sistema.
UC8	Gestão de Gerentes	RF5	Cadastro de um gerente de projeto feito pelo administrador do sistema.
UC9	Controle de Projetos	RF5	Controle do projeto, com detalhamento de informações sobre o mesmo feito pelo gerente do projeto.
UC10	Geração da especificação	RF6, RF4	Geração da especificação de requisitos, utilizando um formato pré-definido, contendo todos os dados registrados no projeto.
UC11	Relatório de Acompanhamento	RF4, RF6	Emissão de um relatório contendo uma indicação do estado do projeto, a partir do estado de cada caso de uso que o compõe.

Atores

Nº	Ator	Descrição
4.	Cliente	Clientes de um projeto, normalmente responsável pelo fornecimento de informações para moldagem do produto.
5.	Administrador	Responsável pelo controle do uso do sistema, liberando acesso aos Gerentes a partir do cadastramento de um projeto.
6.	Gerente	Responsável pelo controle de um projeto, definindo a equipe e suas tarefas.
7.	Membro	Pessoa que faz parte da equipe que trabalha no projeto.
8.	Engenheiro de Requisito	Técnico com formação em Engenharia de Software, capaz de identificar e descrever requisitos, utilizando as prescrições de Engenharia de Requisitos existentes e implementadas no sistema.

Especificação dos Casos De Uso
Cadastro de Projeto
Protótipo de Tela de Cadastro de Projetos

Cadastro de Projetos	
Pesquisa por Projetos	
Nome	SystemG (texto com até 30 caracteres)
Gerente	Silio Silvestre Ferreira Freitas (Texto com até 100 caracteres)
<Pesquisar>	
Projetos recuperados	
Sigla	Gerente
SystemG	Silio Silvestre Ferreira Freitas
Frigo	Alberto Sobrinho Araújo
TecnoComp	Silio Silvestre Ferreira Freitas
<Novo><Alterar><Visualizar><Excluir>	
Dados do Projeto	
Sigla*	ReqG ReqG 1.0 (texto com até 30 caracteres)
Descrição	Sistema de Gerenciamento de Requisitos do Projeto (texto com até 255 caracteres)
Gerente*	[Lista de Gerentes de Projeto cadastrados no sistema]
<Salvar>	

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

1. O **ReqG** exibe a Tela de Cadastro de Projetos. 2. O Administrador informa os dados para pesquisa por projetos. 3. O Administrador aciona o comando Pesquisar. 4. O **ReqG** recupera e exibe na lista Projetos Recuperados os projetos que atendem aos parâmetros de pesquisa informados, ordenados pela Sigla em ordem crescente.

Fluxo alternativo Inclusão de novo projeto

Pré-condições	1- O Administrador acionou o comando Novo.
Passos	1. O Administrador preenche os Dados do Projeto. 2. O Administrador aciona o comando Salvar. 3. O **ReqG** verifica que não existe um projeto com sigla informada. 4. O **ReqG** salva os dados do projeto. 5. O **ReqG** envia um email ao gerente do projeto informando o cadastramento do projeto.

Fluxo alternativo Alteração de um Projeto

Pré-condições	1. O Administrador selecionou um Projeto na lista de Projetos Recuperados. 2. O Administrador acionou o comando *Alterar.*
Passos	1. O **ReqG** recupera e exibe Dados do Projeto. 2. O Administrador altera os Dados do Projeto que desejar. 3. O Administrador aciona o comando Salvar. 4. O **ReqG** salva os dados do projeto. 5. O **ReqG** verifica que não existe um projeto com sigla informada. 6. O **ReqG** salva os dados do projeto. 7. O **ReqG** envia um email ao gerente do projeto informando a alteração no projeto.

Fluxo alternativo Visualização de Dados de um Projeto

Pré-condições	1. O Administrador selecionou um projeto na lista de Projetos Recuperados. 2. O Administrador acionou o comando Visualizar.
Passos	1. 1. O **ReqG** recupera e exibe os Dados do Projeto somente para leitura.

Fluxo alternativo Exclusão de um Projeto

Pré-condições	1. O Administrador selecionou um projeto na lista de *Projetos Recuperados.* 2. O Administrador acionou o comando *Excluir.*
Passos	1. O **ReqG** verifica que não há dados associado ao projeto selecionado, como requisitos, casos de uso, membros e revisões. 2. O **ReqG** exclui o projeto selecionado. 3. O **ReqG** envia um email ao gerente do projeto informando a exclusão do projeto.

Gestão de Gerentes

Protótipo de Tela de Gestão de Gerentes

Gestão de Gerentes			
Pesquisa por Gerentes			
Nome	Silio Silvestre Ferreira Freitas (texto com até 100 caracteres)		
Projeto	Ferrare 1.0 (texto com até 30 caracteres)		
<Pesquisar>			
Gerentes recuperados			
Nome	**Login**	**Email**	**Projeto**
Silio Silvestre Ferreira Freitas	Silio	siliosffreitas@gmail.com	Ferrare 1.0, ReqG 1.0
João José Almeida Cavalcante	jj	jjacavalcante@hotmail.com	Genesis 2.0
<Novo><Alterar><Visualizar><Excluir>			
Dados do Gerente			
Nome*	Silio Silvestre Ferreira Freitas (texto com até 100 caracteres)		
Email*	siliosffreitas@gmail.com (email válido)		
Telefone	(86) 3224-3678 (texto até 100 caracteres)		

Login*	siliosffreitas (texto até 20 caracteres)
Senha*	12345678 (texto até 20 caracteres)
<Salvar>	

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

4. O **ReqG** exibe a Tela de Gestão de Gerentes. 5. O Administrador informa os dados para pesquisa por Gerentes. 6. O Administrador aciona o comando Pesquisar. 7. O **ReqG** recupera e exibe na lista Gerentes recuperados os Gerentes que atendem aos parâmetros de pesquisa informados, ordenados pelo Nome em ordem crescente.

Fluxo alternativo Inclusão de um Novo Gerente

Pré-condições	8. O Administrador acionou o comando Novo.
Passos	9. O Administrador preenche os *Dados do Gerente.* 10. O Administrador aciona o comando *Salvar.* 11. O **ReqG** verifica que não existe um Gerente com *email* e *login* informados. 12. O **ReqG** salva os Dados do *Gerente.*

Fluxo alternativo Alteração de um Gerente

Pré-condições	1. O Administrador selecionou um Gerente na lista de *Gerentes recuperados.* 2. O Administrador acionou o comando *Alterar.*
Passos	1. O **ReqG** recupera e exibe Dados do *Gerente.* 2. O Administrador altera os Dados do Gerente que desejar. 3. O Administrador aciona o comando Salvar. 4. O **ReqG** verifica que não existe um Gerente com *email e login* informados. 5. O **ReqG** salva os Dados do Gerente.

Fluxo alternativo Visualização de Dados de um Gerente

Pré-condições	1. O Administrador selecionou um Gerente na lista de *Gerentes recuperados.* 2. O Administrador acionou o comando *Visualizar.*
Passos	1. O **ReqG** recupera e exibe os *Dados do Gerente* somente para leitura.

1. Fluxo alternativo Exclusão de um Gerente

Pré-condições	1. O Administrador selecionou um Gerente na lista de *Gerentes recuperados.* 2. O Administrador acionou o comando *Excluir.*
Passos	1. O **ReqG** verifica que não há nenhum projeto que tenha o Gerente selecionado como Gerente do projeto. 2. O **ReqG** exclui o Gerente selecionado.

Gestão de Membros

Protótipo de Tela de Gestão de Membros

Gestão de Membros				
Pesquisa por Membros				
Nome	Pedro da Silva Castro (texto com até 50 caracteres)			
Cargo	[Analista, Programador, Designer, Arquiteto de Software, Fornecedor de Requisitos]			
<Pesquisar>				
Membros do Projeto				
Nome	**Email**	**Projeto**	**Telefone**	**Papel**
Pedro	Pedro@email.com	Ferrare 1.0	(12) 1212-1212	Analista
Lucas	lucassld@gmail.com	Quantum 2.0	(23) 3443-3433	Design
Adriana	Adriana12@hotmail.com	XMen 1.1	(34) 4554-5445	Programador
<Novo><Alterar><Visualizar><Excluir>				
Dados do Membro				
Nome*	Thiago Alves (texto com até 100 caracteres)			
Email*	tt@gmail.com (email válido)			
Telefone	(086) 9999-9999 (telefone válido)			
Papel*	Cliente; Engenheiro de Requisitos			
Login	Thiago (texto com até 20 caracteres)			

Dados do Membro	
Senha*	Senha (texto com até 20 caracteres)

Projeto*	[Lista de projetos pré-cadastrados no sistema e associados ao usuário corrente]
	...
	[Lista de projetos pré-cadastrados no sistema e associados ao usuário corrente
<Salvar>	

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

1. O **ReqG** exibe a Tela de Gestão de Membros.
2. O Gerente informa os dados para pesquisa por Membros.
3. O Gerente aciona o comando Pesquisar.
4. O **ReqG** recupera e exibe na lista Membros Recuperados os Membros que atendem aos parâmetros de pesquisa informados, ordenados pelo Nome em ordem crescente.

Fluxo alternativo Inclusão de um Novo Membro

Pré-condições	1.O Gerente acionou o comando Novo.
Passos	1. O Gerente preenche os Dados do Membro. 2.O Gerente aciona o comando Salvar. 3. O **ReqG** verifica que não existe um membro cadastrado com login e email informados. 4. O **ReqG** salva os dados do Membro. 5. O **ReqG** envia um email ao Membro informando sobre sua inclusão nos projetos selecionados, detalhando seu cargo em cada projeto.

Fluxo alternativo Alteração de um Membro

Pré-condições	1.O Gerente selecionou um Membro na lista de *Membros Recuperados.* 2. O Gerente acionou o comando *Alterar.*
Passos	1. O **ReqG** recupera e exibe *Dados do Membro.* 2. O Gerente altera os *Dados do Membro* que desejar. 3. O Gerente aciona o comando *Salvar.* 4. O **ReqG** verifica que não existe um membro com *login e email* informados. 5. O **ReqG** salva os dados do Membro. 6. O **ReqG** envia um *email* para o membro informando as alterações.

Fluxo alternativo Visualização de Dados de um Membro

Pré-condições	1. O Gerente selecionou um Membro na lista de *Membros Recuperados* 2. O Gerente acionou o comando *Visualizar.*
Passos	1.O **ReqG** recupera e exibe os Dados do Membro somente para leitura.

Fluxo alternativo Exclusão de um Membro

Pré-condições	1. O Gerente selecionou um Membro na lista de *Membros Recuperados* 2. O Gerente acionou o comando *Excluir.*
Passos	1. O **ReqG** verifica se que o Membro não trabalhou no projeto, fornecendo informações a respeito do levantamento de requisitos tais como registrando requisitos, detalhando casos de uso, protótipos, realizando revisões, etc.. 2. O **ReqG** exclui o Membro selecionado.

Controle de Projetos

Protótipo de Tela de Controle de Projetos

Controle do Projeto	
Pesquisa por projetos	
Projeto	Estocando (texto com até 30 caracteres)
<Pesquisar>	
Projetos recuperados	
Sigla	**Gerente**
SystemG	Silio Silvestre Ferreira Freitas
Frigo	Alberto Sobrinho Araújo
TecnoComp	Silio Silvestre Ferreira Freitas
<Alterar><Visualizar>	
Dados do Projeto	
Projeto	SystemG
Gerente	Silio Silvestre Ferreira Freitas
Escopo*	Sistema de apoio e controle de requisitos (texto com até 2000 caracteres)
Diagrama de Contexto	SystemG_DiagramaContexto.jpg (Imagem)
Limites do produto	O sistema não terá monitoramento on line (área de texto com até 2000 caracteres)
Instituição responsável pela execução	Fictítica Soluções em Informática (texto com até 100 caracteres)

Instituição cliente	Biriba Comércio (texto com até 100 caracteres)
Data de início*	12/05/2009 (data passada ou atual válida no formato dd/mm/aaaa)
Data de Término	12/05/2011 (data futura válida no formato dd/mm/aaaa)

Membros do Projeto				
Nome	**Email**	**Projeto**	**Celular**	**Cargo**
Pedro	**Pedro@email.com**	**Ferrare 1.0**	**(12) 1212-1212**	**Analista**
Lucas	**lucassld@gmail.com**	**Quantum 2.0**	**(23) 3443-3433**	**Design**
Adriana	**Adriana12@hotmail.com**	**XMen 1.1**	**(34) 4554-5445**	**Programador**
<Salvar>				

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

O **ReqG** exibe a Tela de Controle do Projeto.
O Gerente informa os dados para pesquisa por Projetos.
O Gerente aciona o comando Pesquisar.
O **ReqG** recupera e exibe na lista Projetos Recuperados os Projetos que atendem aos parâmetros de pesquisa informados e que tenham como Gerente o usuário corrente, ordenados pelo Nome em ordem crescente.
O Gerente seleciona o projeto a controlar e aciona o comando Alterar.
O Gerente altera os dados de controle do projeto.
O Gerente aciona o comando Salvar.
O **ReqG** salva os dados do projeto.
O **ReqG** envia um email a todos os membros do projeto relatando as alterações realizadas.

Gestão de Requisitos

Protótipo de Tela de Gestão de Requisitos

Gestão de Requisitos	
Pesquisa por requisitos	
Projeto	Estocando (texto com até 30 caracteres)
Requisito	Linguagem (texto com até 30 caracteres)
<Pesquisar>	
Requisitos recuperados	

Nome	Descrição	Prioridade	Projeto
Linguagem Java	Desenvolvido em Java	Alta	Ferrare 1.0
Gestão de Projetos	Gestão de Projetos	Média	Quantum 2.0
Revisão	Revisão dos Requisitos	Baixa	XMen 1.0
<Novo><Alterar><Visualizar><Excluir>			
Dados do Requisito			
Identificador*	RF1 (formato RFn para funcionais ou RNFn para não funcionais, onde n é o próximo número natural não associado a um requisito)		
Nome*	Linguagem Java (texto até 100 caracteres)		
Descrição*	O Sistema deve ser desenvolvido em Java para a Web (texto até 2000 caracteres)		
Tipo*	[Funcional; Não Funcional]		
Prioridade*	[Alta; Média; Baixa]		
Complexidade*	[Alta; Média; Baixa]		
Situação*	[Identificado; Detalhado; Implementado; Homologado]		
<Salvar>			
Histórico de Alteração			
Data	**Responsável**	**Papel**	**Email**
10/05/2010	Joaquim Parente	Analista	joça@gmail.com
15/05/2010	**Astulfo Alves**	**Analista**	**astoa@gmail.com**
10/07/2010	**Silio Silvestre**	**Gerente**	**silio@gmail.com**

Detalhamento do Caso de Uso

Pré-condições

O usuário corrente deve ser um Engenheiro de Requisitos ou o Gerente do Projeto.

Fluxo principal

O **ReqG** exibe a Tela de Gestão de Requisitos.
O Engenheiro de Requisitos informa os dados para pesquisa por Requisitos.
O Engenheiro de Requisitos aciona o comando Pesquisar.
O **ReqG** recupera e exibe na lista Requisitos Recuperados os requisitos que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo Nome em ordem crescente.

Fluxo alternativo Inclusão de Novo Requisito

Pré-condições	1. O Engenheiro de Requisitos acionou o comando Novo.
Passos	1. O Engenheiro de Requisitos preenche os Dados do Requisito. 2. O Engenheiro de Requisitos aciona o comando Salvar. 3. O **ReqG** gera o identificador do requisito sendo RF + n para requisitos funcionais ou RNF + n para requisitos não funcionais, onde n é o próximo numero natural não associado a um requisito. 4. O **ReqG** verifica que não há requisito para o projeto com o nome e tipo informado. 5. O **ReqG** verifica que a situação do requisito é Identificado. 6. O ReqG salva os dados do Requisito. 7. O **ReqG** registra no histórico de alteração do requisito a data atual, o usuário corrente como responsável pela inclusão.

Fluxo alternativo de Alteração do Requisito

Pré-condições	1. O Engenheiro de Requisitos selecionou um Requisito na lista de Requisitos recuperados. 2. O Engenheiro de Requisitos acionou o comando Alterar.
Passos	1. O **ReqG** recupera e exibe Dados do Requisito. 2. O Engenheiro de Requisitos altera os Dados do Requisito que desejar. 3. O Engenheiro de Requisitos aciona o comando Salvar. 4. **ReqG** verifica que todos os casos de uso relacionados ao requisito estão na mesma situação do requisito. 5. O **ReqG** salva os dados do Requisito. 6. O **ReqG** registra no histórico de alteração do requisito a data atual e o usuário corrente como responsável pela alteração.

Fluxo alternativo Visualização de Dados de Requisito

Pré-condições	1. O Engenheiro de Requisitos selecionou um Requisito na lista de Requisitos recuperados. 2. O Engenheiro de Requisitos acionou o comando *Visualizar.*
Passos	1. O **ReqG** recupera e exibe os Dados do Requisito somente para leitura.

Fluxo alternativo Exclusão de Requisito

Pré-condições	1. O Engenheiro de Requisitos selecionou um Requisito na lista de Requisitos recuperados. 2. O Engenheiro de Requisitos acionou o comando *Excluir.*

Passos	1. O **ReqG** verifica que não existe nenhum caso de uso ou revisão associado ao Requisito. 2. O **ReqG** exclui o requisito selecionado.

Gestão de Atores

Protótipo de Tela de Gestão de Atores

Gestão de Atores			
Pesquisa por Ator			
Projeto	Estocando (texto com até 30 caracteres)		
Ator	Gerente(texto com até 30 caracteres)		
<Pesquisar>			
Atores recuperados			
Nome	**Descrição**	**Caso de Uso**	**Projeto**
Caixeiro	Usuário que abre e fecha compra	UC6,UC8	Estocando
Fiscal de ponto	Fiscaliza entrada e saída de	UC1	Estocando
Revisão	Revisão dos Requisitos	UC2	Ferrare 1.0
<Novo><Alterar><Visualizar><Excluir>			
Dados do Ator			
Nome*	Caixeiro (texto até 50 caracteres, único)		
Descrição*	Responsável pela venda de mercadorias (texto até 100 caracteres)		
Caso de uso associado*	[Lista de caso de uso cadastrado no projeto]		
	...		
	[Lista de caso de uso cadastrado no projeto]		
<Salvar>			
Histórico de Alteração			
Data	**Responsável**	**Papel**	**Email**
10/05/2010	Joaquim Parente	Analista	joça@gmail.com
15/05/2010	Astulfo Alves	Analista	astoa@gmail.com
10/07/2010	Silio Silvestre	Gerente	silio@gmail.com

Detalhamento do Caso de Uso

Pré-condições

O usuário corrente deve ser um Engenheiro de Requisitos ou o Gerente do Projeto.

Fluxo principal

O **ReqG** exibe a Tela de Gestão de Atores. O Engenheiro de Requisitos informa os dados para pesquisa por Atores. O Engenheiro de Requisitos aciona o comando Pesquisar. O **ReqG** recupera e exibe na lista Atores Recuperados os atores que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo Nome em ordem crescente.

Fluxo alternativo de Alteração do Ator

Pré-condições	1. O Engenheiro de Requisitos selecionou um Ator na lista de *Atores recuperados.* 2. O Engenheiro de Requisitos acionou o comando *Alterar.*
Passos	1. O **ReqG** recupera e exibe *Dados do Ator.* 2. O Engenheiro de Requisitos altera os *Dados do Ator* que desejar. 3. O Engenheiro de Requisitos aciona o comando *Salvar.* 4. O **ReqG** verifica que não existe um ator com mesmo nome para o projeto. 5. O **ReqG** salva os dados do Ator. 6. O **ReqG** registra no histórico de alteração do ator a data atual, o usuário corrente como responsável pela alteração.

Fluxo alternativo Visualização de Dados de Ator

Pré-condições	1. O Engenheiro de Requisitos selecionou um Ator na lista de *Atores recuperados.* 2. O Engenheiro de Requisitos acionou o comando *Visualizar.*
Passos	1. O **ReqG** recupera e exibe os *Dados do Ator* somente para leitura.

Fluxo alternativo Exclusão de Ator

Pré-condições	1. O Engenheiro de Requisitos selecionou um Ator na lista de *Atores recuperados.* 2. O Engenheiro de Requisitos acionou o comando *Excluir.*
Passos	1. O **ReqG** verifica que o ator não está associado a nenhum caso de uso. 2. O **ReqG** exclui o ator selecionado. .

Gestão de Casos de Uso

Protótipo de Tela de Gestão de Casos de Uso

Gestão de Casos de Uso
Pesquisa por Casos de Uso

Projeto	Estocando (texto com até 30 caracteres)		
Caso de uso	Criação de Projetos (texto até 30 caracteres)		
<Pesquisar>			
Casos de Uso recuperados			
Nome	Descrição	Requisitos	Projeto
Gestão de Projetos	Criação de Projetos	RF01,RF03	Estocando
Gestão de Gerentes	Controle de Gerentes por um projeto.	RF01	Estocando
Gestão de Membros	Controle de membros de uma equipe	RF02	Ferrare 1.0
<Novo><Alterar><Visualizar><Excluir>			
Dados do Caso de Uso			
Identificador*	UC1 (formato UCn, onde n é o próximo número natural não associado a um caso de uso)		
Nome*	Cadastrar Projeto (texto até 100 caracteres)		
Descrição*	Cria a conta no sistema para o gerenciamento dos requisitos de um projeto (texto até 2000 caracteres)		
Ator*	[Lista de atores pré-cadastrados para o projeto]		
	...		
	[Lista de atores pré-cadastrados para o projeto]		
Prioridade*	[Alta; Média; Baixa]		
Complexidade*	[Alta; Média; Baixa]		
Situação*	[Identificado; Detalhado; Implementado; Homologado]		
Fluxo Principal			
Pré-condição*	Usuário logado no Sistema como Gerente (Texto com até 500 caracteres)		
Passos*	O ReqG exibe a Tela de Gestão de Casos de Uso. O ReqG recupera e exibe todos os casos de uso cadastrados. (área de texto com até 2000 caracteres)		
Fluxo Alternativos			
Nome	**Pré-condições**	**Passos**	
Inclusão de caso de uso	O usuário acionou o comando novo.	O ReqG faz isso. O ReqG faz aquilo. ReqG salva tudo.	
Exclusão de caso de uso	O usuário acionou o comando excluir.	O ReqG remove tudo.	
<Adicionar fluxo><Excluir fluxo>			
Subfluxos			
Nome	**Passos**		
Atualização de algo	O ReqG faz isso. O ReqG faz aquilo. ReqG salva tudo.		

Atualização de algo	O ReqG faz isso. O ReqG faz aquilo. ReqG salva tudo.
Dados Subfluxos	
Nome*	Inclusão de usuário (texto com até 100 caracteres, único por projeto)
Passos*	O ReqG exibe a Tela de Gestão de Casos de Uso. O ReqG recupera e exibe todos os casos de uso cadastrados. (área de texto com até 2000 caracteres)
<Adicionar subfluxo><Excluir Subfluxo>	
Protótipos de interfaces	
Nome	**Arquivo**
Tela de Inclusão de caso de uso	Tela.jpg
Conexão com sistema de tarefas	-
Dados de Protótipos de interfaces	
Nome*	Tela de inclusão de usuário (texto com até 100 caracteres, único por projeto)
Arquivo*	Tela.jpg (arquivo com imagem)
<Adicionar protótipo><Excluir protótipo>	
<Salvar>	
Histórico de Alteração	

Data	Responsável	Papel	Email
10/05/2010	Joaquim Parente	Analista	joça@gmail.com
15/05/2010	Astulfo Alves	Analista	astoa@gmail.com
10/07/2010	Silio Silvestre	Gerente	silio@gmail.com

Detalhamento do Caso de Uso

Pré-condições

O usuário corrente deve ser um Engenheiro de Requisitos ou o Gerente do Projeto.

Fluxo principal

O **ReqG** exibe a Tela de Gestão de Casos de uso. O Engenheiro de Requisitos informa os dados para pesquisa por Casos de uso. O Engenheiro de Requisitos aciona o comando Pesquisar. O **ReqG** recupera e exibe na lista Casos de uso Recuperados os casos de uso que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo Nome em ordem crescente.

Fluxo alternativo Inclusão de Novo Caso de Uso

Pré-condições	1. O Engenheiro de Requisitos acionou o comando Novo.
Passos	1. O Engenheiro de Requisitos preenche os Dados do Caso de Uso. 2.Se desejar, o Engenheiro de Requisitos inclui fluxos alternativos, subfluxos e protótipos de interfaces. 3.Se desejar, o Engenheiro de Requisitos exclui fluxos alternativos, subfluxos e protótipos de interfaces. 4.O Engenheiro de Requisitos aciona o comando Salvar. 5.O **ReqG** verifica que não existe um caso de uso com mesmo nome para o projeto. 6.O **ReqG** salva os dados do caso de uso. 7.O **ReqG** registra no histórico de alteração do caso de uso a data atual e o usuário corrente como responsável pela inclusão.

Fluxo alternativo de Alteração do Caso de Uso

Pré-condições	1. O Engenheiro de Requisitos acionou o comando Novo. 2. O Engenheiro de Requisitos acionou o comando Alterar.

Passos	1. O **ReqG** recupera e exibe Dados do Caso de Uso. 2. O Engenheiro de Requisitos altera os Dados do Caso de Uso. 3. Se desejar, o Engenheiro de Requisitos inclui fluxos alternativos e protótipos de interfaces. 4. O Engenheiro de Requisitos aciona o comando Salvar. 5. **O ReqG** verifica que não existe um caso de uso com mesmo nome para o projeto. 6. O **ReqG** salva os dados do Caso de Uso. 7. O **ReqG** registra no histórico de alteração do caso de uso a data atual e o usuário corrente como responsável pela alteração.

Fluxo alternativo Visualização de Caso de Uso

Pré-condições	1. O Engenheiro de Requisitos selecionou um Caso de Uso na lista de Casos de Uso recuperados. 2. O Engenheiro de Requisitos acionou o comando Visualizar.
Passos	1. O **ReqG** recupera e exibe os Dados do Caso de Uso somente para leitura.

Fluxo alternativo Exclusão de Caso de Uso

Pré-condições	1. O Engenheiro de Requisitos selecionou um Caso de Uso na lista de Caso de Uso recuperado. 2. O Engenheiro de Requisitos acionou o comando Excluir.
Passos	1. O **ReqG** solicita confirmação da exclusão. 2. O **ReqG** exclui o caso de uso.

Gestão de Critérios

Protótipo de Tela de Critérios

Gestão de Critérios			
Pesquisa por Critérios			
Projeto	Estocando (texto até 30 caracteres)		
Critério	Ambiguidade (texto até 50 caracteres)		
Tipo	[Requisito; Caso de uso]		
<Pesquisar>			
Critérios recuperados			
Projeto	**Nome**	**Descrição**	**Tipo**
Ferrare 1.0	Ambiguidade	Apresenta mais de um sentido semântico	Requisito
Ferrare 1.0	Clareza	Apresenta uma interpretação direta e clara	Requisito
ReqG 1.0	Completude	O Caso de uso está completo	Caso de uso
<Novo><Alterar><Visualizar><Excluir>			
Dados do Critério			
Projeto	[Lista de Projetos cadastrados e que o usuário corrente é Membro]		
Nome*	Ambigüidade (texto até 50 caracteres)		
Descrição*	Apresenta mais de um sentido (texto até 2000 caracteres)		
Tipo*	[Requisito; Caso de Uso]		
<Salvar>			

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

O **ReqG** exibe a Tela de Gestão de Critérios.
O Gerente informa os dados para pesquisa por Critérios.
O Gerente aciona o comando Pesquisar.
O **ReqG** recupera e exibe na lista Critérios Recuperados os critérios que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo Nome em ordem crescente.

Fluxo alternativo Inclusão de Novo Critério

Pré-condições	1. O Gerente acionou o comando Novo.

Passos	1. O Gerente preenche os Dados do Critério. 2. O Gerente aciona o comando Salvar. 3. O ReqG verifica que não existe um critério cadastrado com o nome informado. 4. O ReqG salva os dados do Critério.

Fluxo alternativo de Alteração de Critério

Pré-condições	1. O Gerente selecionou um Critério na lista de Critérios recuperados. 2. O Gerente acionou o comando Alterar.
Passos	1. O **ReqG** recupera e exibe Dados do Critério. 2. O Gerente altera os Dados do Critério que desejar. 3. O Gerente aciona o comando Salvar. 4. O **ReqG** verifica que não existe um critério cadastrado com o nome informado. 5. O **ReqG** salva os dados do Critério.

Fluxo alternativo Visualização de Dados de Critério

Pré-condições	1. O Gerente selecionou um Critério na lista de Critérios recuperados. 2. O Gerente acionou o comando Visualizar.
Passos	1. O **ReqG** recupera e exibe os Dados do Critério somente para leitura.

Fluxo alternativo Exclusão de Critério

Pré-condições	1. O Gerente selecionou um Critério na lista de Critérios recuperados. 2. O Gerente acionou o comando Excluir.
Passos	1. O **ReqG** verifica que não existe nenhuma revisão cadastrada e que utilize o critério. 2. O **ReqG** exclui o Critério selecionado.

Revisão

Protótipo de Tela de Revisão

Revisão	
Pesquisar por Revisões	
Revisão	Revisão preliminar de levantamento (texto com até 30 caracteres).
Projeto	SystemG (texto com até 30 caracteres)
<Pesquisar>	

Revisões recuperadas		
Projeto	**Identificador da Revisão**	**Data**
SystemG 1.0	Revisão preliminar de levantamento de requisitos	31/10/2009
SystemG 1.0	Revisão final de detalhamento de requisitos	31/11/2009
SystemG 2.0	Revisão preliminar de levantamento de requisitos	17/10/2010
SystemG 2.0	Revisão final de detalhamento de requisitos	14/11/2010
ProjetoX 1.0	Revisão inicial de requisitos	12/12/2010
<Nova Revisão><Alterar Revisão><Visualizar Revisão><Reabrir revisão>		
Dados da Revisão		
Projeto*	[Lista de Projetos que possuem o usuário corrente como membro]	
Identificador*	Revisão preliminar de requisitos (texto até 50 caracteres)	
Descrição*	Revisão preliminar de requisitos (texto até 50 caracteres)	
Data*	12/12/2010 (data no formato dd/mm/aaaa)	
Participantes dos desenvolvedores*	[Lista de Membros do Projeto que não sejam clientes]	
	...	
	[Lista de Membros do Projeto que não sejam clientes]	
Participantes dos clientes	[Lista de Membros do Projeto que sejam clientes]	
	...	
	[Lista de Membros do Projeto que sejam clientes]	
Situação*	Situação* [Aberta; Fechada]	
Requisitos/Casos de uso Avaliados		
Requisito/Caso de uso	**Critério**	**Atende?**	**Observações**
Banco de dados	Ambuiguidade	Sim	-
	Clareza	Não	Não ficou claro o banco a ser utilizado.
	Precisão	Não	Não foi definida com precisão a versão do banco a ser utilizado.
Gestão de membros	Ambuiguidade	Sim	-
	Clareza	Sim	-
	Precisão	Sim	-
Revisão	Ambuiguidade	Sim	-
	Clareza	Não	Não ficou claro se a revisão é apenas de requisitos ou se pode ser de casos de uso também.
	Precisão	Sim	-

<Excluir Avaliação de Requisito/Caso de uso>		
Avaliação de Critérios para Requisitos		
Requisito a ser avaliado	[Lista de requisitos cadastrados no projeto]	
Critério	Atende?	Observações
Ambiguidade	[sim; não]	(texto com até 100 caracteres)
Clareza	[sim; não]	(texto com até 100 caracteres)
Precisão	[sim; não]	(texto com até 100 caracteres)
<Adicionar Avaliação de Requisito>		
Avaliação de Critérios para Casos de Uso		
Caso de uso a ser avaliado	[Lista de Casos de uso cadastrados no projeto]	
Critério	Atende?	Observações
Ambiguidade	[sim; não]	(texto com até 100 caracteres)
Clareza	[sim; não]	(texto com até 100 caracteres)
Precisão	[sim; não]	(texto com até 100 caracteres)
<Adicionar Avaliação de Caso de uso>		
<Salvar>		

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

O **ReqG** exibe a Tela de Revisão.

O Engenheiro de Requisito informa os dados para pesquisa por Revisão.

O Engenheiro de Requisito aciona o comando Pesquisar.

O **ReqG** recupera e exibe na lista Revisões Recuperados as revisões que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo Projeto e Identificador da revisão em ordem crescente.

Fluxo alternativo Inclusão de Nova Revisão

Pré-condições	1. O Engenheiro de Requisito acionou o comando Nova Revisão.
Passos	1. O Gerente preenche os Dados da Revisão 2. Para cada Requisito a ser avaliado: 1. O **ReqG** executa o Subfluxo Avaliação de Requisito. 3. Para cada Caso de uso a ser avaliado: 1. O **ReqG** executa o Subfluxo Avaliação de Caso de uso. 4. Se desejar excluir alguma avaliação de requisito ou caso de uso: 1. O **ReqG** executa o Subfluxo 5. O Engenheiro de Requisitos aciona o comando Salvar. 6. O **ReqG** verifica que não existe uma revisão cadastrada para o projeto com o mesmo identificador. 7. O **ReqG** salva os dados da Revisão.

Subfluxo Avaliação de Requisito

1. O Engenheiro de Requisitos informa o Requisito a ser avaliado.
2. O **ReqG** recupera e exibe na lista de Avaliação de Critérios para Requisitos os critérios associados a requisitos cadastrados no projeto.
3. O Engenheiro de Requisitos informa se o critério foi atendido e, se desejar, preenche alguma informação adicional no campo observações.
4. O Engenheiro de Requisitos aciona o comando Adicionar Avaliação de Requisito.

Subfluxo Avaliação de Caso de uso

1. O Engenheiro de Requisitos informa o Caso de uso a ser avaliado.
2. O **ReqG** recupera e exibe na lista de Avaliação de Critérios para Casos de uso os critérios associados a casos de uso cadastrados no projeto.
3. O Engenheiro de Requisitos informa se o critério foi atendido e, se desejar, preenche alguma informação adicional no campo observações.
4. O Engenheiro de Requisitos aciona o comando Adicionar Avaliação de Caso de uso.

Subfluxo Exclusão de Avaliação

1. O Engenheiro de Requisitos seleciona uma avaliação de Requisito ou Caso de uso na lista de Requisitos/Casos de uso avaliados.
2. O Engenheiro de Requisitos aciona o comando Excluir Avaliação de Requisito/Caso de uso.
3. O **ReqG** remove a avaliação do requisito/caso de uso..

Fluxo alternativo Alteração de Revisão

Pré-condições	1. O Engenheiro de Requisitos selecionou uma Revisão da Lista de Revisões recuperadas. 2. O Engenheiro de Requisitos acionou o comando Alterar Revisão. 3. A Revisão não se encontra no estado fechado.

Passos	1. O **ReqG** recupera e exibe cada avaliação de requisito e caso de uso associada à revisão na lista de Requisitos/Casos de uso avaliados. 2. O Gerentepreenche os Dados da Revisão. 3. Para cada Requisito a ser avaliado: 1. O **ReqG** executa o Subfluxo Avaliação de Requisito. 4. Para cada Caso de uso a ser avaliado: 1. O **ReqG** executa o Subfluxo Avaliação de Caso de uso. 5. Se desejar excluir alguma avaliação de requisito ou caso de uso: 1. O **ReqG** executa o Subfluxo 6. O Engenheiro de Requisitos aciona o comando Salvar. 7. O **ReqG** verifica que não existe uma revisão cadastrada para o projeto com o mesmo identificador. 8. O **ReqG** salva os dados da Revisão.

Fluxo alternativo Visualização de Revisão

Pré-condições	1. O Engenheiro de Requisitos selecionou uma Revisão da Lista de Revisões recuperadas. 2. O Engenheiro de Requisitos acionou o comando Visualizar Revisão.
Passos	1. O **ReqG** recupera e exibe os Dados da Revisão. 2. O **ReqG** recupera e exibe cada avaliação de requisito e caso de uso associada à revisão na lista de Requisitos/Casos de uso avaliados.

Fluxo alternativo Reabertura de Revisão

Pré-condições	1. O Engenheiro de Requisitos selecionou uma Revisão da Lista de Revisões recuperadas. 2. O Engenheiro de Requisitos acionou o comando Reabrir Revisão. 3. A Revisão encontra-se no estado fechado.
Passos	1. O **ReqG** altera o status da revisão para Aberta. 2. **O ReqG** salva os dados da Revisão.

Geração da Especificação

Protótipo de Tela de Geração da Especificação

Geração da Especificação	
Informações do Projeto	
Geração da Especificação	SystemG (texto com até 30 caracteres)
Informações do Projeto	Silio Silvestre Ferreira Freitas (Texto com até 100 caracteres)
<Pesquisar>	
Projetos recuperados	

Projeto	Descrição	Gerente
SystemG	Criação de um sistema X	Silio Silvestre Ferreira Freitas
Frigo	Projeto muito interessante	Alberto Sobrinho Araújo
TecnoComp	Manutenção de algo	Silio Silvestre Ferreira Freitas
<Gerar Especificação>		

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

O **ReqG** exibe a Tela de Geração da Especificação.
O Membro informa os dados para pesquisa por Projetos.
O Membro aciona o comando Pesquisar.
O **ReqG** recupera e exibe na lista Projetos Recuperados os Projetos que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo nome do Projeto em ordem crescente.
O Membro aciona o comando Gerar Especificação.
O **ReqG** gera um documento no formato especificado no arquivo Modelo-ERSw.doc.

Relatório de Acompanhamento

Protótipo da Tela de Relatório de Acompanhamento

Relatório de Acompanhamento		
Informações do Projeto		
Projeto	SystemG (texto com até 30 caracteres)	
Gerente	Silio Silvestre Ferreira Freitas (Texto com até 100 caracteres)	
<Pesquisar>		
Projetos recuperados		
Projeto	**Descrição**	**Gerente**
SystemG	Criação de um sistema X	Silio Silvestre Ferreira Freitas
Frigo	Projeto muito interessante	Alberto Sobrinho Araújo
TecnoComp	Manutenção de algo	Silio Silvestre Ferreira Freitas
<Gerar Relatório>		

Protótipo do Relatório de Acompanhamento

Relatório de Acompanhamento
Projeto: SystemG
Gerente: Silio Silvestre

ID	Requisito	Descrição	Tipo	Estado
RF1	Linguagem Java	O sistema deve ser feito em Java.	Não-funcional	Identificado (10%)
RF2	Acompanhamento	O sistema deve permitir o acompa- acompanhamento do projeto pelos membros	Funcional	Identificado (20%)

ID	Caso de uso	Descrição	Estado
UC4	Geração da especificação	Relatório com a descrição do projeto	Detalhado (25%)
UC6	Relatório de Acomp.	Relatório com o estado do projeto	Detalhado (25%)
UC8	Gestão de Membros	Cadastro de membros	Identificado (10%)

ID	Requisito	Descrição	Tipo	Estado
RF3	Gestão de projeto	O sistema deve permitir o acompa- acompanhamento do projeto pelos membros	Funcional	Detalhado (50%)

ID	Caso de uso	Descrição	Estado
UC2	Gestão de projetos	Cadastro de projetos	Detalhado (25%)
UC3	Controle de projetos	Controle do projeto	Implementado (75%)

Percentual de conclusão do projeto: 27%

Detalhamento do Caso de Uso

Pré-condições

Não aplicável.

Fluxo principal

O ReqG exibe a Tela de Relatório de Acompanhamento.
O Membro informa os dados para pesquisa por Projetos.
O Membro aciona o comando Pesquisar.
O ReqG recupera e exibe na lista Projetos Recuperados os Projetos que atendem aos parâmetros de pesquisa informados e que tenham como Membro no projeto o usuário corrente, ordenados pelo nome do Projeto em ordem crescente.
O Membro aciona o comando Gerar Relatório.
Para cada requisito contido no projeto:
O ReqG imprime uma linha com o ID, nome, descrição, tipo e estado do requisito, calculado a partir do estado ou a partir dos casos de uso associados.
Para cada caso de uso associado ao requisito:
O ReqG imprime uma linha com o ID, nome, descrição e estado do caso de uso.
O ReqG soma o percentual de conclusão de cada caso de uso, de acordo com o seu estado, sendo Identificado (10%), Detalhado (25%), Implementado (75%) e Homologado (100%).
Se o requisito possui casos de uso associados:
O ReqG calcula o percentual de conclusão do requisito a partir da soma de todos os percentuais dos casos de uso dividido pela quantidade de casos de uso.
Se não:
O ReqG calcula o percentual de conclusão do requisito a partir do estado do requisito.
O ReqG calcula o percentual de conclusão do projeto a partir da soma de todos os percentuais dos requisitos divididos pela quantidade de requisitos existentes.

ANEXO II – Exemplo de Agenda de Oficina de Levantamento de Requisitos

Agenda de Oficina de Levantamento de Requisitos Projeto NomeDoProjeto		
Data:	Horário:	Local:
10/05/10	11:00 - 11:30h	Nome Local:
Participante do Cliente:		Nome do Desenvolvedor 1 Nome do Desenvolvedor 2 Nome do Desenvolvedor 3
Participante do Desenvolvedor:		Nome do Cliente 1 Nome do Cliente 2 Nome do Cliente 3
Pauta Prevista:		
Nº	Assunto	Descrição
1.	Apresentação inicial	Apresentação do calendário (programação de datas e pautas) das oficinas;

1	Apresentação inicial	Apresentação do cronograma do levantamento inicial e detalhamento dos requisitos.
2	Seções iniciais da ERSw	Determinação do Escopo do produto Descrição Geral do Produto Levantamento das Funções
Pauta adicional:		
Nº	Assunto	Descrição
1	Assunto 1	Pauta adicional, para ser discutida caso haja tempo na reunião, apenas uma forma de prevenção.
Documentos necessários para a realização da oficina		
Pelo Cliente:		Algum documento necessário para se chegar a alguma função do sistema, como normas, leis, exemplos, etc.
Pelos Desenvolvedores:		1. Algum documento ou artefato que pode ajudar no levantamento de requisitos.
Pendências a serem checadas:		
Nº	Descrição	Responsável
1	1. -	-
Observações:		
A Pauta Adicional será discutida apenas se houver disponibilidade de tempo. A impossibilidade de participação de algum membro convocado deverá ser comunicada, no mínimo, com um dia de antecedência da reunião. Caso algum documento a ser apresentado ou pendência a ser checada não tenha sido preparada (o) em tempo, tal fato deve ser comunicado, no mínimo, com um dia de antecedência da reunião.		
Convocado por:		Data:
Nome da Organização Desenvolvedora – Nome da Equipe		10/05/10

ANEXO III – Exemplo de Ata de Oficina de Levantamento de Requisitos

Ata de Reunião de Levantamento de Requisitos Projeto Nome Do Projeto			
Data:	Horário:	Local:	
10/05/10	11:00 - 11:30h	Nome do Local	
Participantes do Cliente:		Nome do Cliente 1 Nome do Cliente 2 Nome do Cliente 3	
Participantes do Desenvolvedor:		Nome do Desenvolvedor 1 Nome do Desenvolvedor 2 Nome do Desenvolvedor 3	
Tópicos discutidos			
Nº	Assunto	Descrição	
1.	Apresentação inicial	1. O planejamento inicial foi apresentado aos usuários.	
2.	Seções iniciais da ERSw	1. Foi solicitada a alteração da data de início do período de revisão da ERSw pelos usuários. O fornecedor ficou de avaliar se será possível. 2. Foi esclarecido que os resultados precisarão ser acompanhados por outros membros da PLAG. 3. Foi sugerida uma apresentação do trabalho que está sendo feito para as pessoas envolvidas do cliente para que as mesmas tomem conhecimento. 4. O cronograma inicial das oficinas e pautas sugeridas para cada uma delas foi apresentado aos usuários. Foi colocado que caso alguma oficina seja cancelada, esse cancelamento terá repercussão em todo o planejamento comprometendo os prazos de entrega da ERSw e fases posteriores a especificação.	
Erratas			
Data	Tópico incorreto	Correção	
-	-	-	
Pendências			
Nº	Descrição	Responsável	Prazo
5.	Agendar uma data para apresentação do trabalho que está sendo feito para os envolvidos do cliente.	UFPI	Em aberto
6.	Rever período de revisão da ERSw pelos usuários.	UFPI	
7.	Rever Pautas previstas das oficinas.	UFPI	
Data de envio da ata para o cliente			

Anexo IV Revisão da Especificação de Requisitos

Objetivos do Documento

Registrar os problemas encontradas durante a revisão da Especificação de Requisitos (ER) do projeto ReqG juntamente com as providências adotadas para solucionar tais problemas.

Revisores

Nome	email	Telefone
Pedro	pasn@ufpi.edu.br	8816-7070

Data	13.01.11

Critérios

Ambiguidade	Cada requisito ou caso de uso especificado não possui mais de um sentido, permitindo assim uma interpretação dúbia sobre o seu significado.
Clareza	As regras de negócio e informações associadas aos requisitos são claras, permitindo seu entendimento.
Completude	Os requisitos estão completamente descritos. Não há lacunas que impossibilitem o entendimento.
Não conflitante	Existem requisitos com especificação de conceitos conflitantes com outros requisitos, deixando claro que existe algo errado na especificação.

Nr.	Requisito	Caso de Uso	Ambiguidade	Clareza	Completude	Não conflitante	Conflitos	Defeito	Providência
1	RF1	-	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
2	RF2	-	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
3	RF3	-	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
4	RF4	-	Não aprovado	Não aprovado	Aprovado	Aprovado	Em relação a ambiguidade, o acompanhamento dos clientes e geração da ER parecem ter muitos aspectos em comum. Em relação à clareza, não ficou claro o que é acompanhamento do projeto, por conta disso a superposição com o RF6.	Parece que o RF4 e RF6 tratam da mesma coisa, embora não haja uma clara definição do que realmente se trata.	O requisito foi reescrito para detalhar o que é o acompanhamento. Isso o diferenciou da geração da especificação, evitando a ambiguidade e deixando claro o que está associado a tal requisito.
5	RF5	-	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
6	RF6	-	Não aprovado	Aprovado	Aprovado	Aprovado	O requisito parece ter superposição com o RF4. Aparentemente os dois descrevem a mesma coisa.	A especificação do requisito, quando comparado ao RF4 parece demonstrar que os dois tratam da mesma coisa, gerando uma duplicidade de interpretação.	A alteração do RF4 o deixou diferente do RF6, resolvendo o problema da ambiguidade existente entre os dois.

Nr.	Requisito	Caso de Uso	Ambiguidade	Clareza	Completude	Não conflitante	Conflitos	Defeito	Providência
7	RNF1		Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
8	RNF2		Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
9	RNF3		Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
10	RNF4		Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
11	RNF5		Aprovado	Não aprovado	Aprovado	Aprovado	Não foi possível entender o que significa uma senha fácil.	Falta a descrição do que é uma senha fácil.	Foi detalhado como deveria ser o padrão de senhas para evitar senhas consideradas fáceis.
12	RNF6		Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
13	RF1	UC1					-	-	-
14	RF1	UC2	Aprovado	Aprovado	Não aprovado	Aprovado	Não foi especificada a ordem em que os resultados devem ser exibidos.	Não foi especificada a ordem em que os resultados devem ser exibidos.	
15	RF2	UC3	Aprovado	Aprovado	Não aprovado	Aprovado	-	-	-
16	RF2	UC4	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
17	RF3	UC5	Aprovado	Aprovado	Aprovado	Aprovado	Não existe a especificação do projeto associado ao requisito.	Identificar o projeto associado ao requisito.	Adicionado ao protótipo a informação do projeto a ter um requisito cadastrado.

Nr.	Requisito	Caso de Uso	Ambiguidade	Clareza	Completude	Não conflitante	Conflitos	Defeito	Providência
18	RF3	UC6	Aprovado	Aprovado	Aprovado	Aprovado	Não existe a especificação do projeto associado ao Ator.	Identificar o projeto associado ao ator.	Adicionado ao protótipo a informação do projeto a ter um ator cadastrado.
19	RF5	UC7	Aprovado	Aprovado	Aprovado	Aprovado	Não existe a especificação do projeto associado ao Caso de uso.	Identificar o projeto associado ao caso de uso.	Adicionado ao protótipo a informação do projeto a ter um caso de uso cadastrado.
20	RF5	UC8	Não aprovado	Aprovado	Aprovado	Aprovado	-	-	
21	RF5	UC9	Aprovado	Aprovado	Aprovado	Aprovado	-		
22	RF4, RF6	UC10	Aprovado	Aprovado	Aprovado	Aprovado	-		
23	RF4, RF6	UC11	Aprovado	Aprovado	Aprovado	Aprovado	-		

Anexo V
Revisão do Modelo de Análise

Objetivos do Documento

Registrar os problemas encontradas durante a revisão do Modelo de Análise do projeto ReqG juntamente com as providências adotadas para solucionar tais problemas.

Revisores

Nome	email	Telefone
Pedro	pasn@ufpi.edu.br	8819-7070

Data	13.01.11

Critérios

Ambiguidade	Cada classe identificada ou realização não possui mais de um sentido, nem existem diferentes representações para coisas similares, que permitam uma interpretação dúbia sobre o seu significado.
Clareza	As classes e realizações são claras permitindo seu entendimento.
Completude	Os atributos dos protótipos que devam ser armazenados estão completamente descritos nas entidades. As realizações possuem todos os objetos utilizados. Não há lacunas que impossibilitem o entendimento.
Não conflitante	As realizações não possuem conceitos conflitantes com outras realizações. As classes não possuem atributos conflitantes. Diagramas em diferentes pacotes não contêm modelagens diferentes.

Nr.	Elemento	Ambiguidade	Clareza	Completude	Não conflitante	Conflitos	Defeito	Providência
1	Classe Alteracao	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
2	Classe Usuario	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
3	Classe Membro	Aprovado	Não aprovado	Aprovado	Aprovado	Os atributos de usuário estavam dentro da classe membro, dificultando o entendimento.	Não estava claro o suficiente que os membros do projeto eram usuários.	Foi extraída uma classe usuário, separando-a de membro. Isso tornou mais claro o diagrama.
4	Classe Projeto	Aprovado	Aprovado	Não aprovado	Aprovado	Não havia a modelagem dos membros de um projeto.	Não havia a modelagem dos membros de um projeto.	Foi incluído o relacionamento de projeto com membros, representando os participantes, além do gerente.
5	Classe Ator	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
6	Classe Caso de Uso	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
7	Classe Fluxo	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
8	Classe Fluxo Alternativo	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
9	Classe Subfluxo	Não aprovado	Aprovado	Aprovado	Aprovado	Subfluxo não acrescentava nenhuma informação adicional a fluxo, sua classe genérica.	Não havia necessidade de termos uma classe subfluxo, uma vez que um subfluxo é um fluxo e não possui qualquer característica adicional.	Foi removida a classe e criada uma nova associação entre caso de uso e fluxo, com o nome subfluxos.

Nr.	Elemento	Ambiguidade	Clareza	Completude	Não conflitante	Conflitos	Defeito	Providência
10	Classe Pro-totipo	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
11	Classe Requsiito	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
12	Classe Criterio	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
13	Classe Item de Revisao	Aprovado	Aprovado	Não aprovado	Aprovado	Não havia um atributo para as observações relacionadas a avalia-ção de um critério.	Falta do campo observação na avaliação.	Foi adicionado o campo à classe Item de Revisão.
14	Classe Revisao	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
15	Diagrama de Entidades	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-
16	Realização Controle de Projetos	Aprovado	Aprovado	Aprovado	Aprovado	-	-	-

Referências

ALISTAIR, COCKBURN, **Escrevendo Casos De Uso Eficazes,** Editora Bookman, 2003.

BERTRAND MEYER, Object-oriented Software Construction, Prentice-Hall, 2a edição, 1997.

DAVID GARMUS, DAVID HERRON, **Function Point Analysis: Measurement Practices for Successful Software Projects,** Addison-Wesley, 2000.

IAN SOMMERVILLE, **Engenharia de Software,** 6a. Edição, Addison-Wesley, 2005.

J. RUMBAUGH, I. JACOBSON, G. BOOCH.The **Unified Modeling Language Reference Manual,** Addison Wesley, 1999.

JANE WOOD, SILVER DENISE, **Joint Application Development,** John Wiley & Sons Inc, ISBN 0-47104-299-4.

KENDALL SCOTT, **Processo Unificado Explicado,** Editora Bookman, 2003.

KENT BECK, **Extreme Programming Explained:** embrace change. Boston: Addison–Wesley/Longman, 1999.

ROGER S. PRESSMAN, **Engenharia de Software,** 5a. Edição, McGraw-Hill, São Paulo, 2002.

WILSON DE PÁDUA PAULA FILHO, **Engenharia de Software: Fundamentos, Métodos e Padrões,** Editora LTC, 2ª Edição, 2003.

WILSON DE PÁDUA PAULA FILHO, **Engenharia de Software: Fundamentos, Métodos e Padrões,** Editora LTC, 2ª Edição, 2003.

Minicurrículo

Pedro de Alcântara dos Santos Neto

CV. http://lattes.cnpq.br/3452982259415951



Possui graduação em Bacharelado em Ciência da Computação pela Universidade Federal do Piauí (1995). Mestrado em Ciência da Computação pela Universidade Federal de Pernambuco (1999) e doutorado em Ciência da Computação pela Universidade Federal de Minas Gerais (2006). Atualmente, é professor da Universidade Federal do Piauí. Tem experiência na área de Engenharia de Software, atuando principalmente na automação de testes, desenvolvimento de software utilizando metodologias ágeis e engenharia de software experimental.





www.uapi.ufpi.br